

Java Virtual Machine

Arquitectura Básica

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta

Java Virtual Machine

Consideraciones

- Es una máquina que solo maneja **enteros**
 - IJVM (Integer Java Virtual Machine)
- No se contemplan operaciones de E/S, **no corresponden a la máquina virtual**
 - Ej: `System.out.println()`
- La instrucción ISA (assembler de JVM) `INVOKEVIRTUAL (CALL)`, está simplificada para que solo pueda invocar métodos de su propio objeto
 - Por lo tanto convierte a esta versión de Java en un lenguaje **no orientado a objetos**, termina siendo un C simplificado.

Java Virtual Machine

Ejemplo de programa

Fragmento Java		Assembler IJVM	Programa IJVM
<code>i=j+k;</code>	1	<code>ILOAD j // i=j+k</code>	<code>0x15 0x02</code>
<code>if(i==3)</code>	2	<code>ILOAD k</code>	<code>0x15 0x03</code>
<code> k=0;</code>	3	<code>IADD</code>	<code>0x60</code>
<code>else</code>	4	<code>ISTORE i</code>	<code>0x36 0x01</code>
<code> j=j-1;</code>	5	<code>ILOAD i // if(i<3)</code>	<code>0x15 0x01</code>
	6	<code>BIPUSH 3</code>	<code>0x10 0x03</code>
	7	<code>IF_ICMPEQ L1</code>	<code>0x9F 0x00 0x0D</code>
	8	<code>ILOAD j // j=j-1</code>	<code>0x15 0x02</code>
	9	<code>BIPUSH 1</code>	<code>0x10 0x01</code>
	10	<code>ISUB</code>	<code>0x64</code>
	11	<code>ISTORE j</code>	<code>0x36 0x02</code>
	12	<code>GOTO L2</code>	<code>0xA7 0x00 0x07</code>
	13	<code>L1: BIPUSH 0 // k=0</code>	<code>0x10 0x00</code>
	14	<code>ISTORE k</code>	<code>0x36 0x03</code>
	15	<code>L2:</code>	

Fig 4-14

Java Virtual Machine

Ejemplo de programa

Fragmento Java

```
i=j+k;  
if(i==3)  
    k=0;  
else  
    j=j-1;
```

Assembler IJVM

```
1      ILOAD j      // i=j+k  
2      ILOAD k  
3      IADD  
4      ISTORE i  
5      ILOAD i      // if(i<3)  
6      BIPUSH 3  
7      IF_ICMPEQ L1  
8      ILOAD j      // j=j-1  
9      BIPUSH 1  
10     ISUB  
11     ISTORE j  
12     GOTO L2  
13 L1: BIPUSH 0      // k=0  
14     ISTORE k  
15 L2:
```

Programa IJVM

```
0x15 0x02  
0x15 0x03  
0x60  
0x36 0x01  
0x15 0x01  
0x10 0x03  
0x9F 0x00 0x0D  
0x15 0x02  
0x10 0x01  
0x64  
0x36 0x02  
0xA7 0x00 0x07  
0x10 0x00  
0x36 0x03
```

Java Virtual Machine

Ejemplo de programa

Fragmento Java

Assembler IJVM

Programa IJVM

<code>i=j+k;</code>	1	<code>ILOAD j</code>	<code>// i=j+k</code>	<code>0x15 0x02</code>
<code>if(i==3)</code>	2	<code>ILOAD k</code>		<code>0x15 0x03</code>
<code> k=0;</code>	3	<code>IADD</code>		<code>0x60</code>
<code>else</code>	4	<code>ISTORE i</code>		<code>0x36 0x01</code>
<code> j=j-1;</code>	5	<code>ILOAD i</code>	<code>// if(i<3)</code>	<code>0x15 0x01</code>
	6	<code>BIPUSH 3</code>		<code>0x10 0x03</code>
	7	<code>IF_ICMPEQ L1</code>		<code>0x9F 0x00 0x0D</code>
	8	<code>ILOAD j</code>	<code>// j=j-1</code>	<code>0x15 0x02</code>
	9	<code>BIPUSH 1</code>		<code>0x10 0x01</code>
	10	<code>ISUB</code>		<code>0x64</code>
	11	<code>ISTORE j</code>		<code>0x36 0x02</code>
	12	<code>GOTO L2</code>		<code>0xA7 0x00 0x07</code>
	13	<code>L1: BIPUSH 0</code>	<code>// k=0</code>	<code>0x10 0x00</code>
	14	<code>ISTORE k</code>		<code>0x36 0x03</code>
	15	<code>L2:</code>		

Fig 4-14

Java Virtual Machine

Ejemplo de programa

Fragmento Java	Assembler IJVM	Programa IJVM
<code>i=j+k;</code>	1 <code>ILOAD j</code> <code>// i=j+k</code>	<code>0x15 0x02</code>
<code>if(i==3)</code>	2 <code>ILOAD k</code>	<code>0x15 0x03</code>
<code>k=0;</code>	3 <code>IADD</code>	<code>0x60</code>
<code>else</code>	4 <code>ISTORE i</code>	<code>0x36 0x01</code>
<code>j=j-1;</code>	5 <code>ILOAD i</code> <code>// if(i<3)</code>	<code>0x15 0x01</code>
	6 <code>BIPUSH 3</code>	<code>0x10 0x03</code>
	7 <code>IF_ICMPEQ L1</code>	<code>0x9F 0x00 0x0D</code>
	8 <code>ILOAD j</code> <code>// j=j-1</code>	<code>0x15 0x02</code>
	9 <code>BIPUSH 1</code>	<code>0x10 0x01</code>
	10 <code>ISUB</code>	<code>0x64</code>
	11 <code>ISTORE j</code>	<code>0x36 0x02</code>
	12 <code>GOTO L2</code>	<code>0xA7 0x00 0x07</code>
	13 <code>L1:</code> <code>BIPUSH 0</code> <code>// k=0</code>	<code>0x10 0x00</code>
	14 <code>ISTORE k</code>	<code>0x36 0x03</code>
	15 <code>L2:</code>	

Fig 4-14

Java Virtual Machine

Ejemplo de programa

Fragmento Java	Assembler IJVM	Programa IJVM
<code>i=j+k;</code>	1 <code>ILOAD j</code> <i>// i=j+k</i>	<code>0x15 0x02</code>
<code>if(i==3)</code>	2 <code>ILOAD k</code>	<code>0x15 0x03</code>
<code>k=0;</code>	3 <code>IADD</code>	<code>0x60</code>
<code>else</code>	4 <code>ISTORE i</code>	<code>0x36 0x01</code>
<code>j=j-1;</code>	5 <code>ILOAD i</code> <i>// if(i<3)</i>	<code>0x15 0x01</code>
	6 <code>BIPUSH 3</code>	<code>0x10 0x03</code>
	7 <code>IF_ICMPEQ L1</code>	<code>0x9F 0x00 0x0D</code>
	8 <code>ILOAD j</code> <i>// j=j-1</i>	<code>0x15 0x02</code>
	9 <code>BIPUSH 1</code>	<code>0x10 0x01</code>
	10 <code>ISUB</code>	<code>0x64</code>
	11 <code>ISTORE j</code>	<code>0x36 0x02</code>
	12 <code>GOTO L2</code>	<code>0xA7 0x00 0x07</code>
	13 <code>L1:</code> <code>BIPUSH 0</code> <i>// k=0</i>	<code>0x10 0x00</code>
	14 <code>ISTORE k</code>	<code>0x36 0x03</code>
	15 <code>L2:</code>	

Fig 4-14

Java Virtual Machine

Ejemplo de programa

Fragmento Java	Assembler IJVM	Programa IJVM
<code>i=j+k;</code>	1 <code>ILOAD j</code> <i>// i=j+k</i>	<code>0x15 0x02</code>
<code>if(i==3)</code>	2 <code>ILOAD k</code>	<code>0x15 0x03</code>
<code>k=0;</code>	3 <code>IADD</code>	<code>0x60</code>
<code>else</code>	4 <code>ISTORE i</code>	<code>0x36 0x01</code>
<code>j=j-1;</code>	5 <code>ILOAD i</code> <i>// if(i<3)</i>	<code>0x15 0x01</code>
	6 <code>BIPUSH 3</code>	<code>0x10 0x03</code>
	7 <code>IF_ICMPEQ L1</code>	<code>0x9F 0x00 0x0D</code>
	8 <code>ILOAD j</code> <i>// j=j-1</i>	<code>0x15 0x02</code>
	9 <code>BIPUSH 1</code>	<code>0x10 0x01</code>
	10 <code>ISUB</code>	<code>0x64</code>
	11 <code>ISTORE j</code>	<code>0x36 0x02</code>
	12 <code>GOTO L2</code>	<code>0xA7 0x00 0x07</code>
	13 <code>L1:</code> <code>BIPUSH 0</code> <i>// k=0</i>	<code>0x10 0x00</code>
	14 <code>ISTORE k</code>	<code>0x36 0x03</code>
	15 <code>L2:</code>	

Fig 4-14

Java Virtual Machine

Instrucciones Assembler JVM (Macro-lenguaje)

Hex	Mnemónico	Significado
0x10	BIPUSH <i>byte</i>	Pone “byte” a la pila
0x59	DUP	Copia la palabra superior en la pila y la pone en la pila
0xA7	GOTO <i>offset</i>	Salto incondicional
0x60	IADD	Saca dos palabras de la pila; pone su suma
0x7E	IAND	Saca dos palabras de la pila; pone el AND booleano
0x99	IFEQ <i>offset</i>	Saca una palabra de la pila y salta si es cero
0x9B	IFLT <i>offset</i>	Saca una palabra de la pila y salta si es menor que cero
0x9F	IF_ICMPEQ <i>offset</i>	Saca dos palabras de la pila; salta si son iguales
0x84	IINC <i>varnum const</i>	Añade una constante a una variable local
0x15	ILOAD <i>varnum</i>	Pone la variable local a la pila
0xB6	INVOKEVIRTUAL <i>disp</i>	Invoca un método
0x80	IOR	Saca dos palabras de la pila; Pone el OR booleano
0xAC	IRETURN	Retorna de un método con valor entero
0x36	ISTORE <i>varnum</i>	Saca una palabra de la pila y la almacena en una variable local
0x64	ISUB	Saca dos palabras de la pila; empuja su diferencia
0x13	LDC_W <i>index</i>	Empuja una constante del pool de constantes a la pila
0x00	NOP	No hace nada
0x57	POP	Elimina la palabra superior de la pila
0x5F	SWAP	Intercambia las dos palabras superiores de la pila
0xC4	WIDE	Prefijo; la siguiente instrucción tiene un índice de 16 bits

Los operandos *byte*, *const* y *varnum* son de **1 byte** y los operandos *disp*, *index* y *offset* son de **2 bytes**

Java Virtual Machine

Instrucciones Assembler Java (Macro-lenguaje)

Hex	Mnemónico	Significado
0x10	BIPUSH <i>byte</i>	Pone "byte" a la pila
0x59	DUP	Copia la palabra superior en la pila y la pone en la pila
0xA7	GOTO <i>offset</i>	Salto incondicional
0x60	IADD	Saca dos palabras de la pila; pone su suma
0x7E	IAND	Saca dos palabras de la pila; pone el AND booleano
0x99	IFEQ <i>offset</i>	Saca una palabra de la pila y salta si es cero
0x9B	IFLT <i>offset</i>	Saca una palabra de la pila y salta si es menor que cero
0x9F	IF_ICMPEQ <i>offset</i>	Saca dos palabras de la pila; salta si son iguales
0x84	IINC <i>varnum const</i>	Añade una constante a una variable local
0x15	ILOAD <i>varnum</i>	Pone la variable local a la pila
0xB6	INVOKEVIRTUAL <i>disp</i>	Invoca un método
0x80	IOR	Saca dos palabras de la pila; Pone el OR booleano
0xAC	IRETURN	Retorna de un método con valor entero
0x36	ISTORE <i>varnum</i>	Saca una palabra de la pila y la almacena en una variable local
0x64	ISUB	Saca dos palabras de la pila; empuja su diferencia
0x13	LDC_W <i>index</i>	Empuja una constante del pool de constantes a la pila
0x00	NOP	No hace nada
0x57	POP	Elimina la palabra superior de la pila
0x5F	SWAP	Intercambia las dos palabras superiores de la pila
0xC4	WIDE	Prefijo; la siguiente instrucción tiene un índice de 16 bits

Los operandos *byte*, *const* y *varnum* son de **1 byte** y los operandos *disp*, *index* y *offset* son de **2 bytes**

Java Virtual Machine

Instrucciones Assembler Java (Macro-lenguaje)

Hex	Mnemónico	Significado
0x10	BIPUSH <i>byte</i>	Pone "byte" a la pila
0x59	DUP	Copia la palabra superior en la pila y la pone en la pila
0xA7	GOTO <i>offset</i>	Salto incondicional
0x60	IADD	Saca dos palabras de la pila; pone su suma
0x7E	IAND	Saca dos palabras de la pila; pone el AND booleano
0x99	IFEQ <i>offset</i>	Saca una palabra de la pila y salta si es cero
0x9B	IFLT <i>offset</i>	Saca una palabra de la pila y salta si es menor que cero
0x9F	IF_ICMPEQ <i>offset</i>	Saca dos palabras de la pila; salta si son iguales
0x84	IINC <i>varnum const</i>	Añade una constante a una variable local
0x15	ILOAD <i>varnum</i>	Pone la variable local a la pila
0xB6	INVOKEVIRTUAL <i>disp</i>	Invoca un método
0x80	IOR	Saca dos palabras de la pila; Pone el OR booleano
0xAC	IRETURN	Retorna de un método con valor entero
0x36	ISTORE <i>varnum</i>	Saca una palabra de la pila y la almacena en una variable local
0x64	ISUB	Saca dos palabras de la pila; empuja su diferencia
0x13	LDC_W <i>index</i>	Empuja una constante del pool de constantes a la pila
0x00	NOP	No hace nada
0x57	POP	Elimina la palabra superior de la pila
0x5F	SWAP	Intercambia las dos palabras superiores de la pila
0xC4	WIDE	Prefijo; la siguiente instrucción tiene un índice de 16 bits

Los operandos *byte*, *const* y *varnum* son de **1 byte** y los operandos *disp*, *index* y *offset* son de **2 bytes**

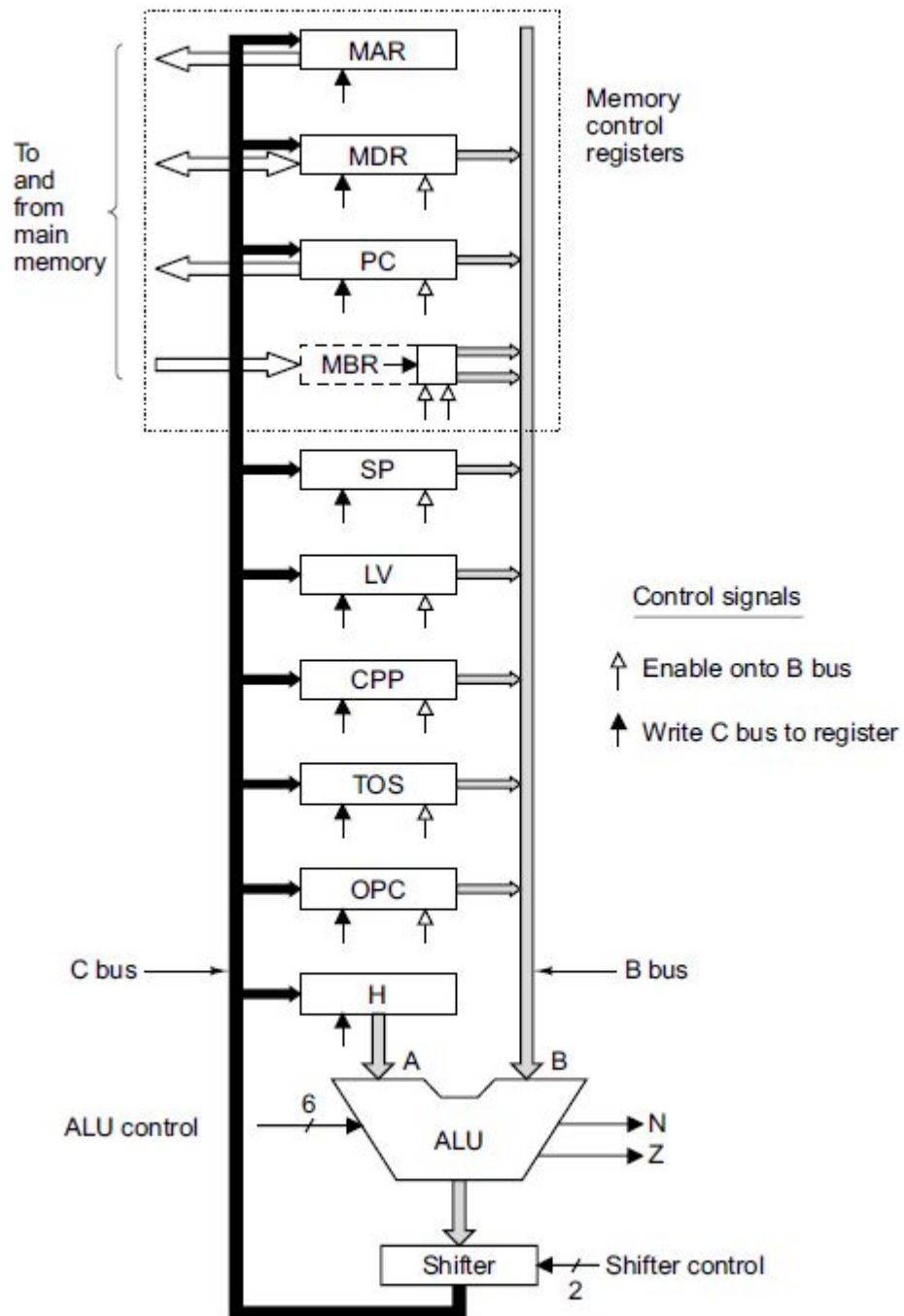
Java Virtual Machine

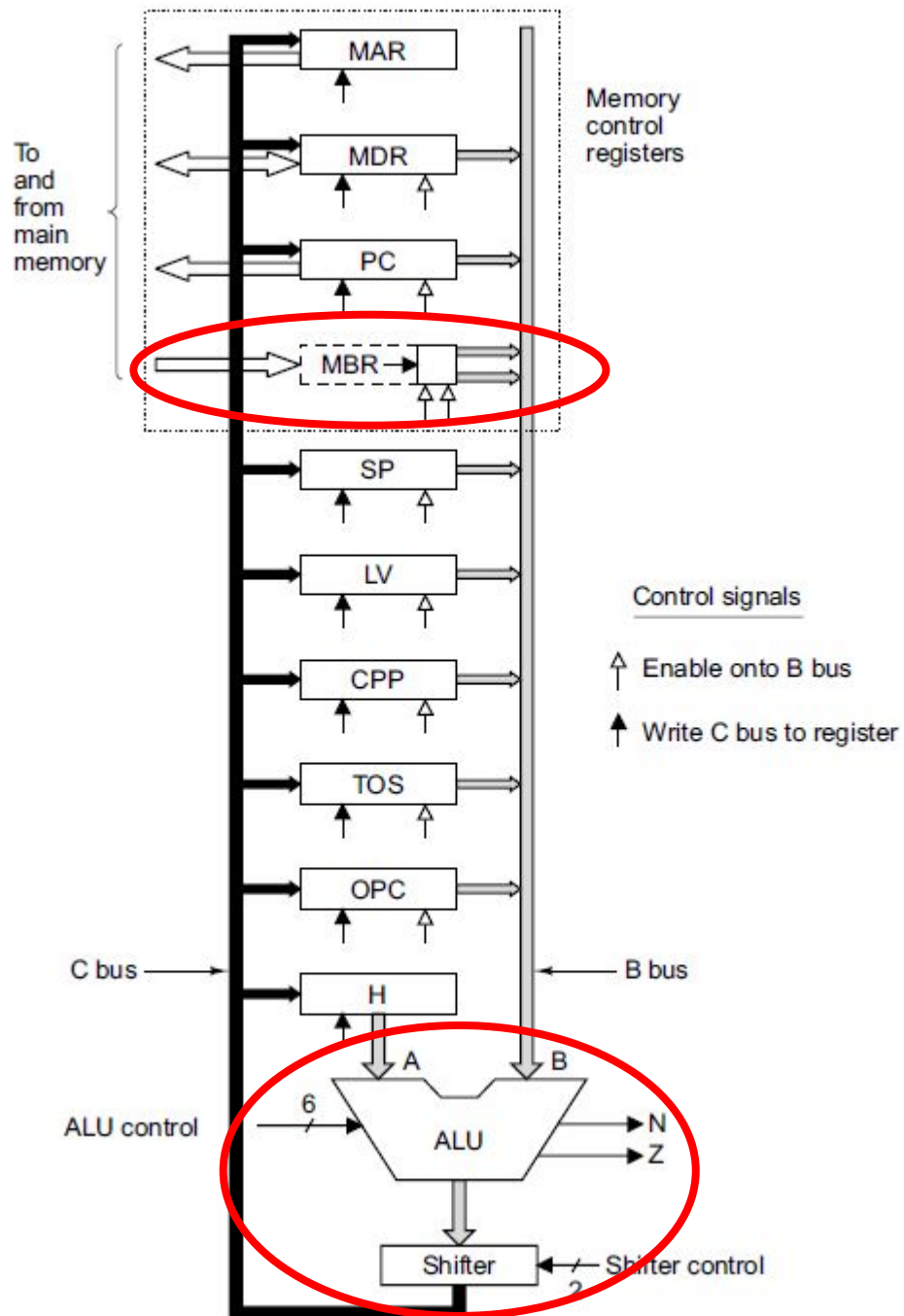
Instrucciones Assembler Java (Macro-lenguaje)

Hex	Mnemónico	Significado
0x10	BIPUSH <i>byte</i>	Pone "byte" a la pila
0x59	DUP	Copia la palabra superior en la pila y la pone en la pila
0xA7	GOTO <i>offset</i>	Salto incondicional
0x60	IADD	Saca dos palabras de la pila; pone su suma
0x7E	IAND	Saca dos palabras de la pila; pone el AND booleano
0x99	IFEQ <i>offset</i>	Saca una palabra de la pila y salta si es cero
0x9B	IFLT <i>offset</i>	Saca una palabra de la pila y salta si es menor que cero
0x9F	IF_ICMPEQ <i>offset</i>	Saca dos palabras de la pila; salta si son iguales
0x84	IINC <i>varnum const</i>	Añade una constante a una variable local
0x15	ILOAD <i>varnum</i>	Pone la variable local a la pila
0xB6	INVOKEVIRTUAL <i>disp</i>	Invoca un método
0x80	IOR	Saca dos palabras de la pila; Pone el OR booleano
0xAC	IRETURN	Retorna de un método con valor entero
0x36	ISTORE <i>varnum</i>	Saca una palabra de la pila y la almacena en una variable local
0x64	ISUB	Saca dos palabras de la pila; empuja su diferencia
0x13	LDC_W <i>index</i>	Empuja una constante del pool de constantes a la pila
0x00	NOP	No hace nada
0x57	POP	Elimina la palabra superior de la pila
0x5F	SWAP	Intercambia las dos palabras superiores de la pila
0xC4	WIDE	Prefijo; la siguiente instrucción tiene un índice de 16 bits

Los operandos *byte*, *const* y *varnum* son de **1 byte** y los operandos *disp*, *index* y *offset* son de **2 bytes**

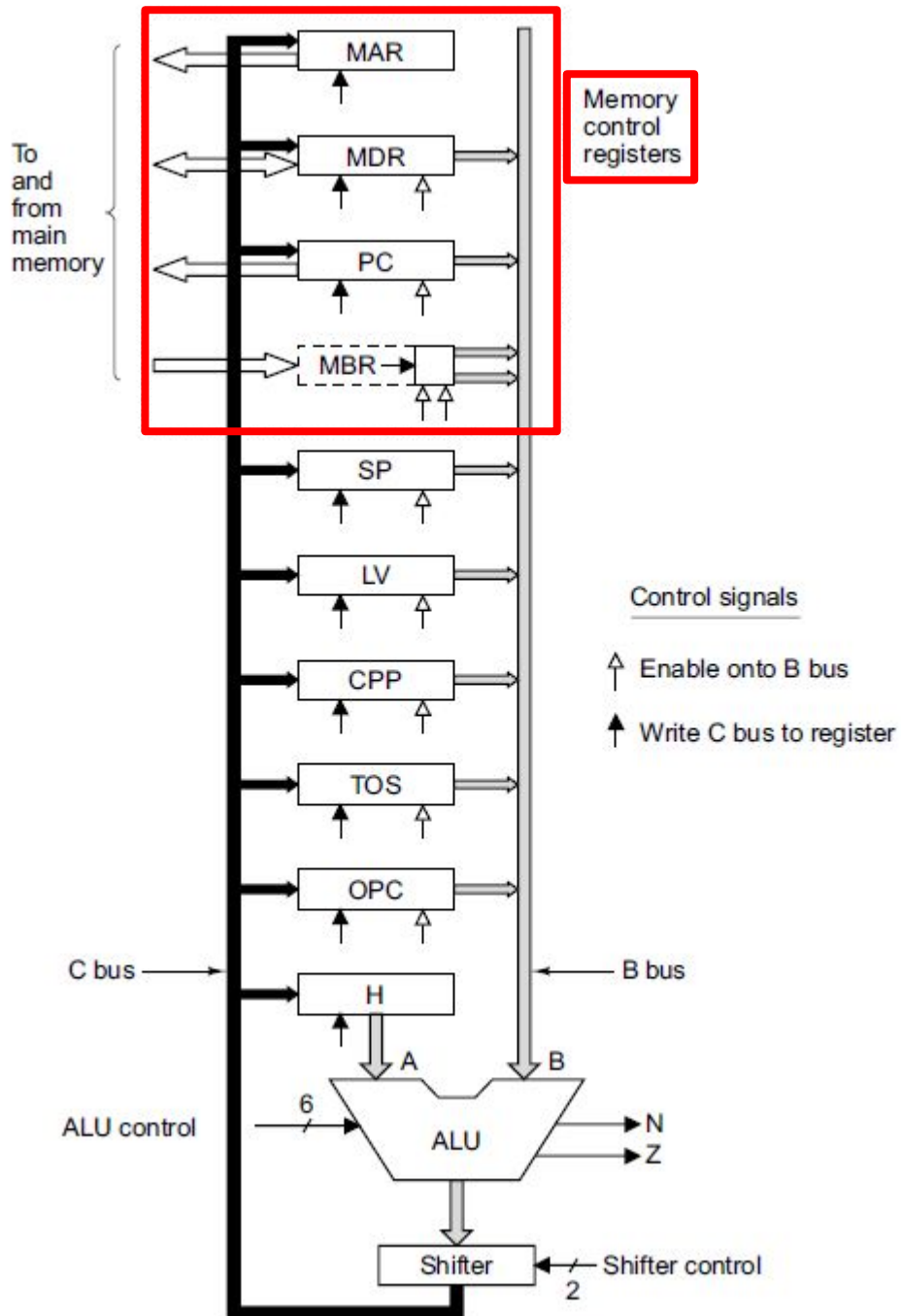
Flujo de datos de la JVM





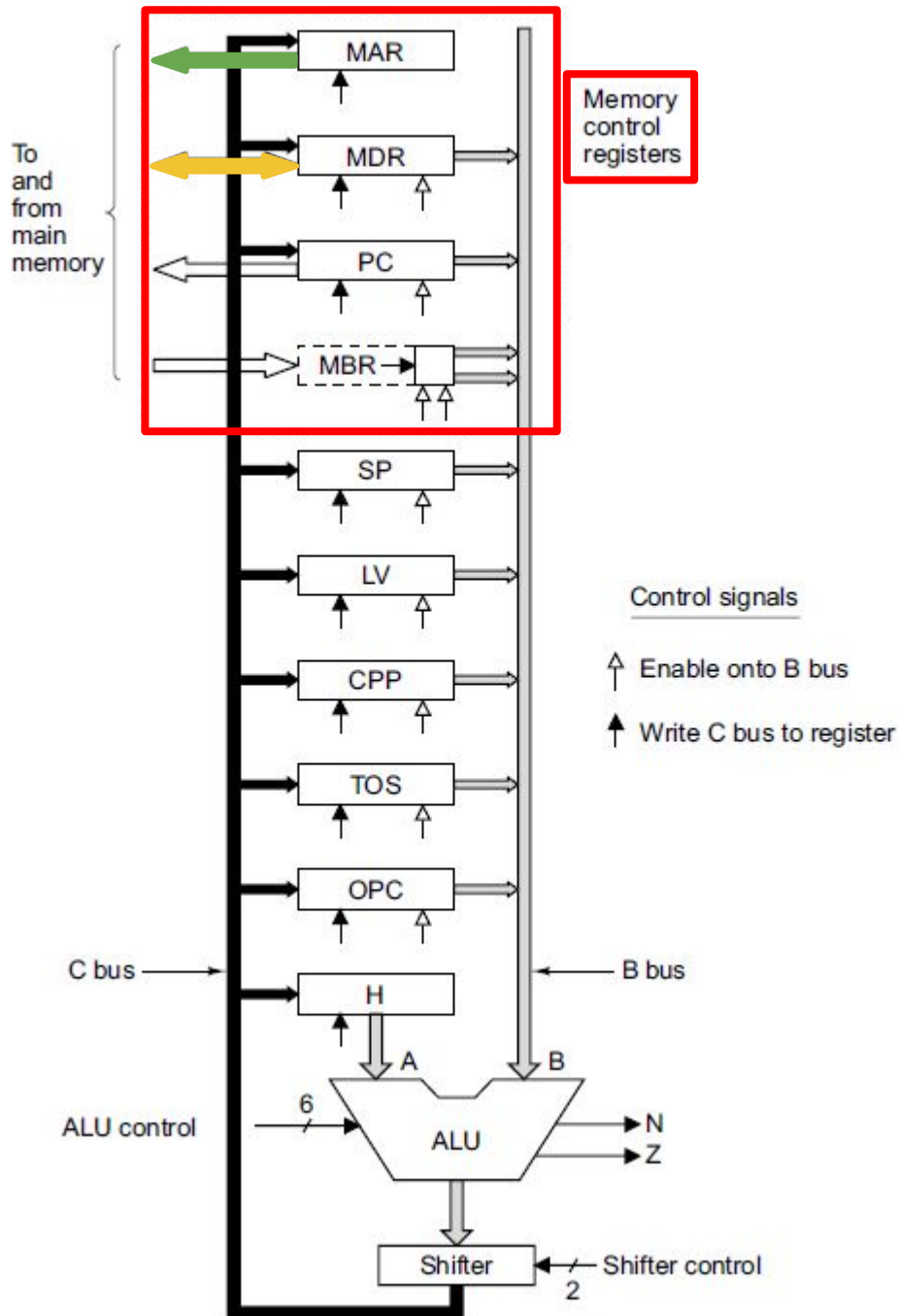
Flujo de datos de la JVM

- Registros de 32 bits
- excepto **MBR** de 8 bits
- ALU y Shifter



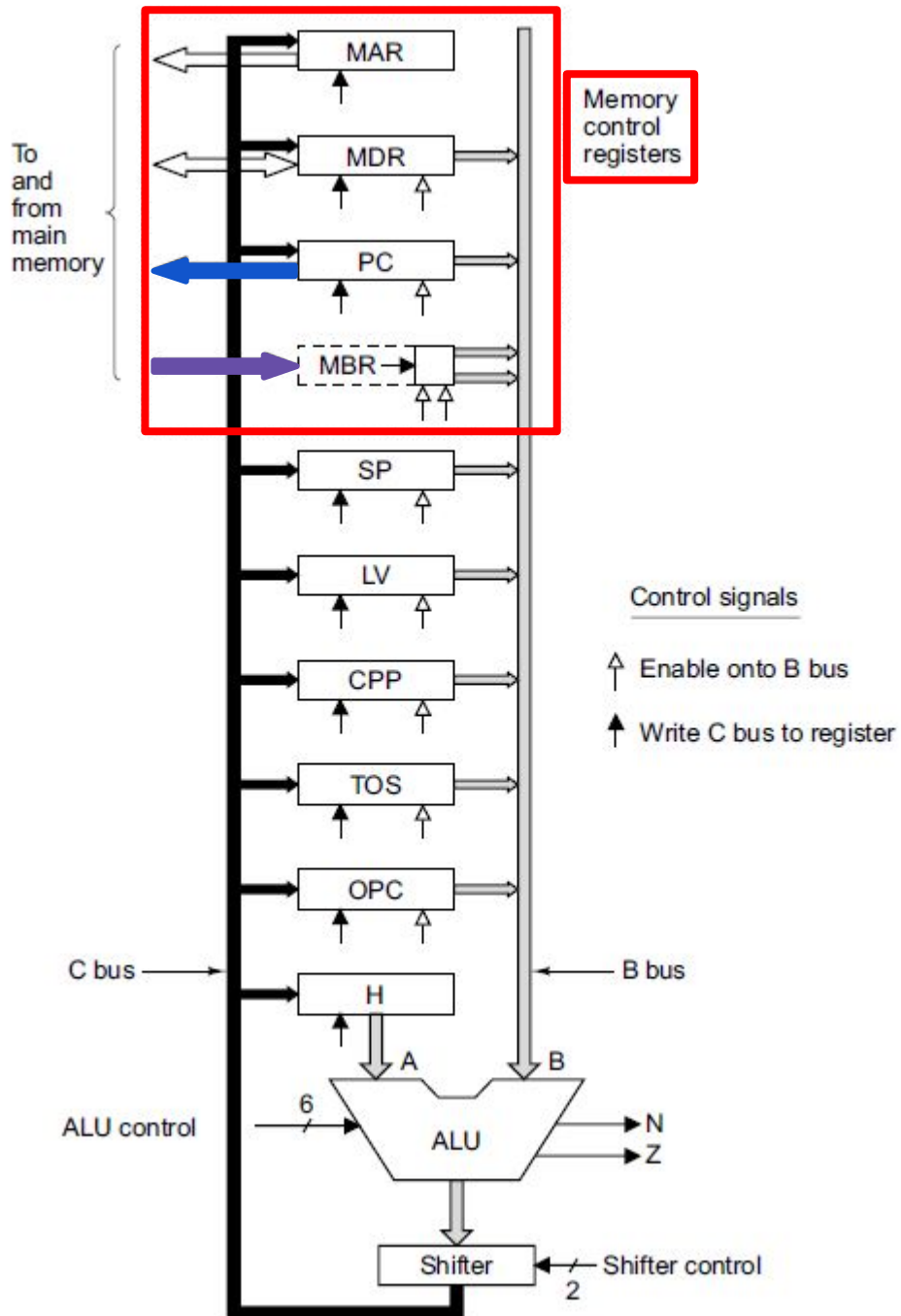
Flujo de datos de la JVM

- Registros que interactúan con la memoria RAM



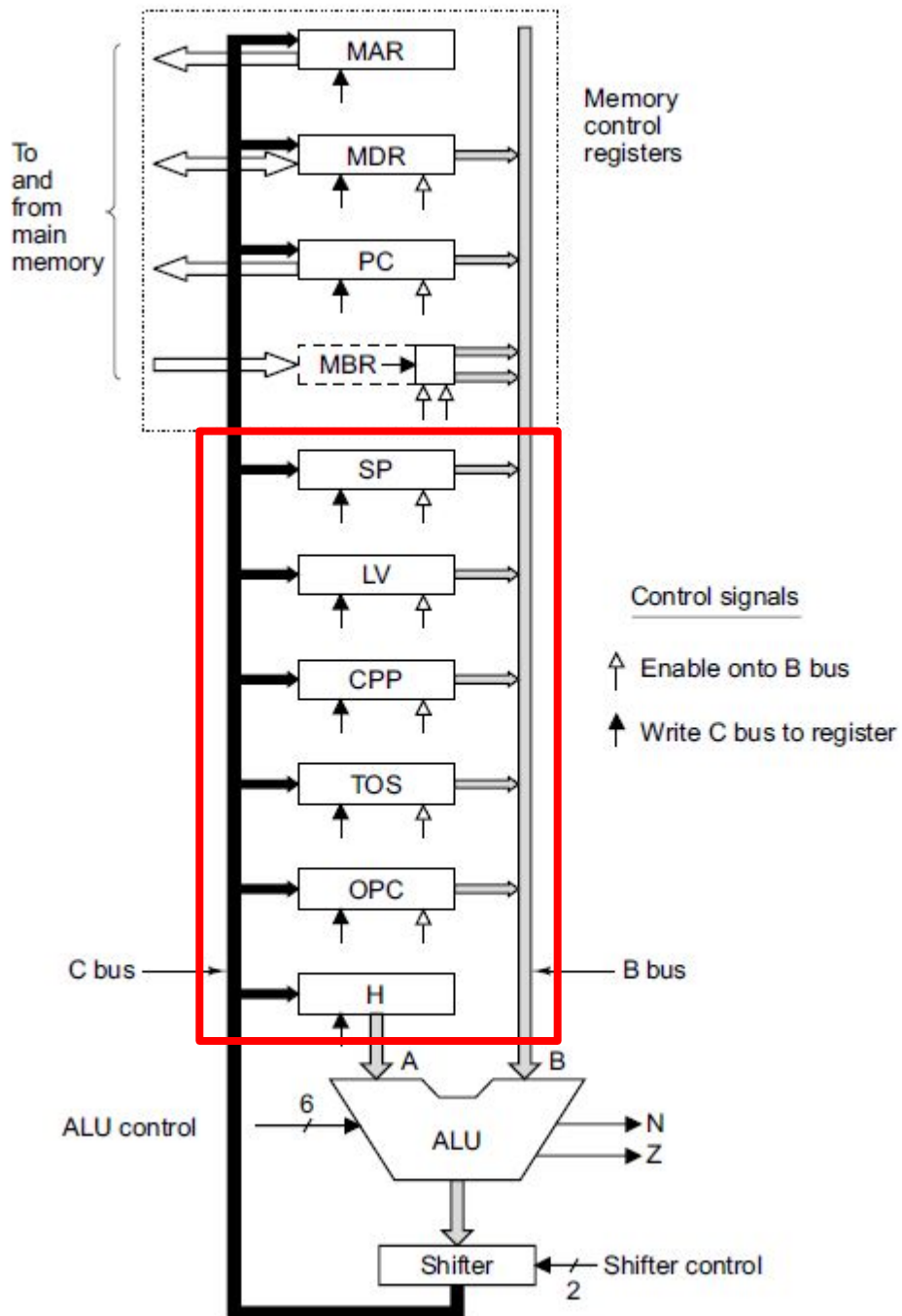
Flujo de datos de la JVM

- MAR y MDR para R/W datos a memoria RAM
- **Memory Address Register**
- **Memory Data Register**



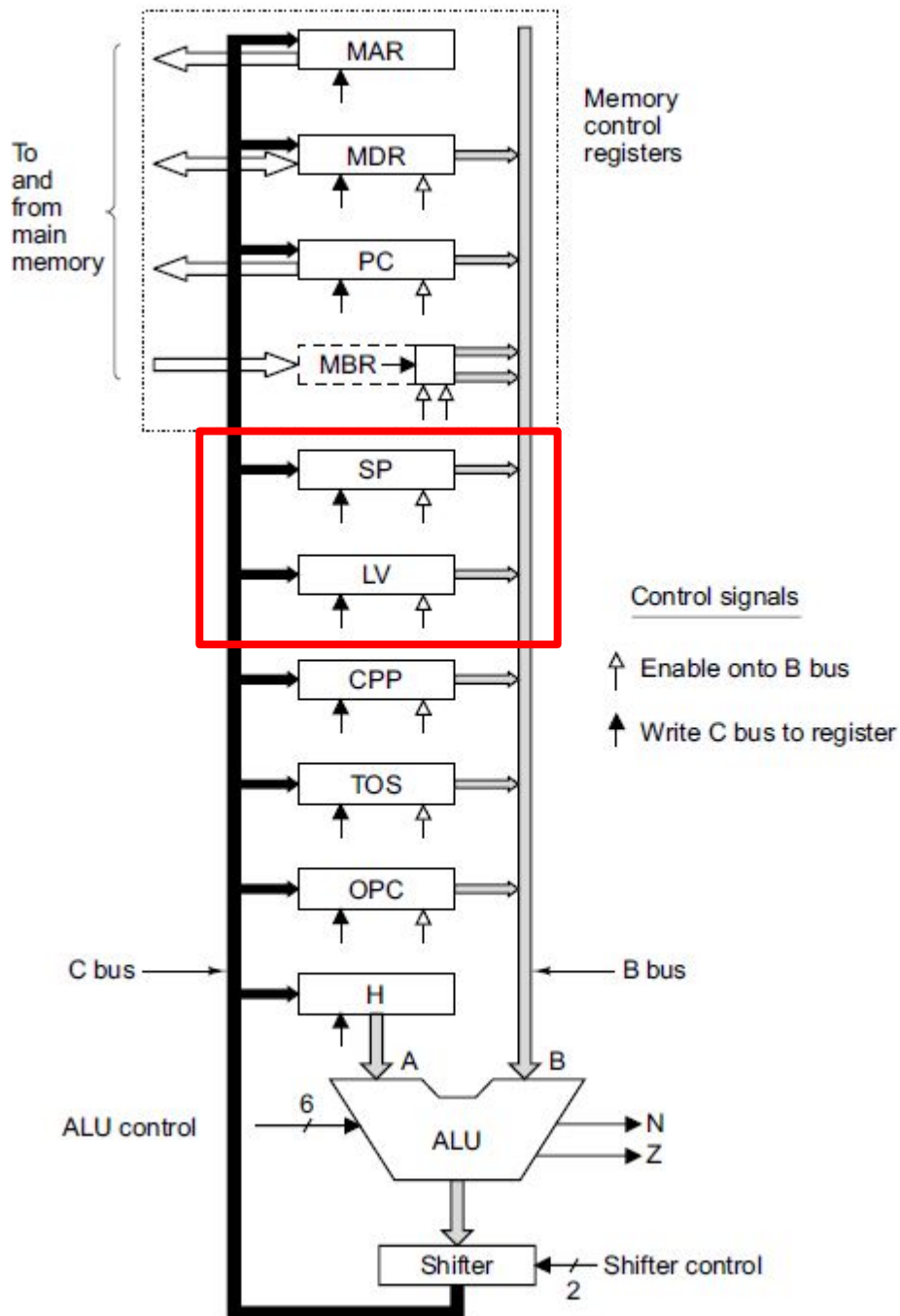
Flujo de datos de la JVM

- PC y MBR para Fetch instrucciones de la RAM
- **P**rogram **C**ounter
- **M**emory **B**uffer **R**egister



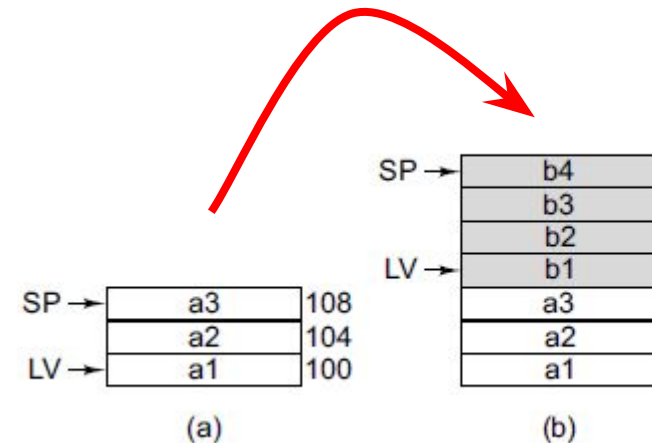
Flujo de datos de la JVM

- Registros de usos específicos
- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held

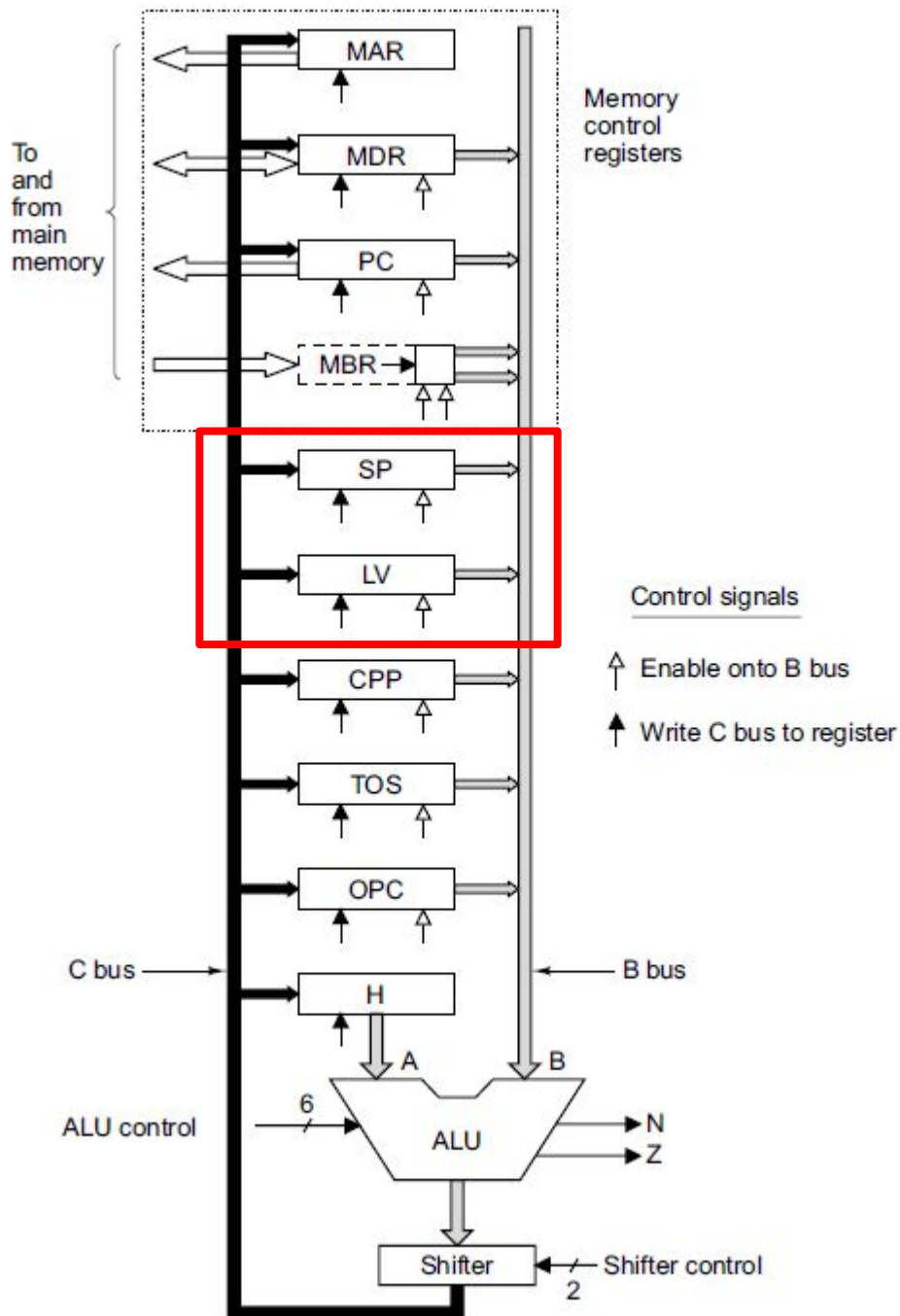


Flujo de datos de la JVM

- Registros de usos específicos
- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held

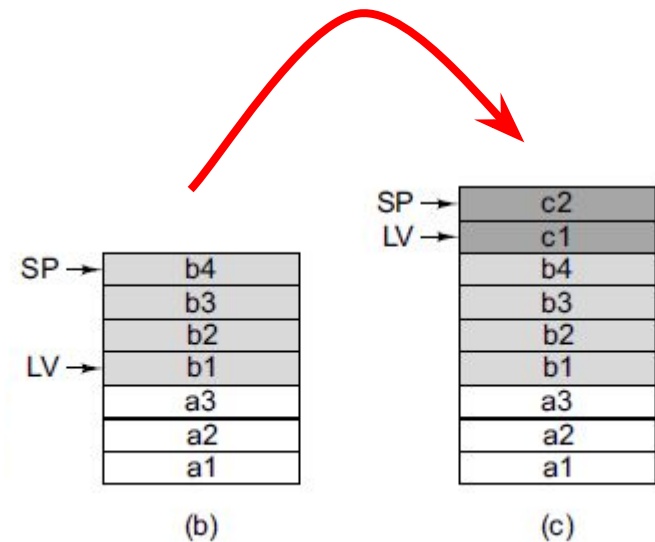


El proc. A invoca al proc. B

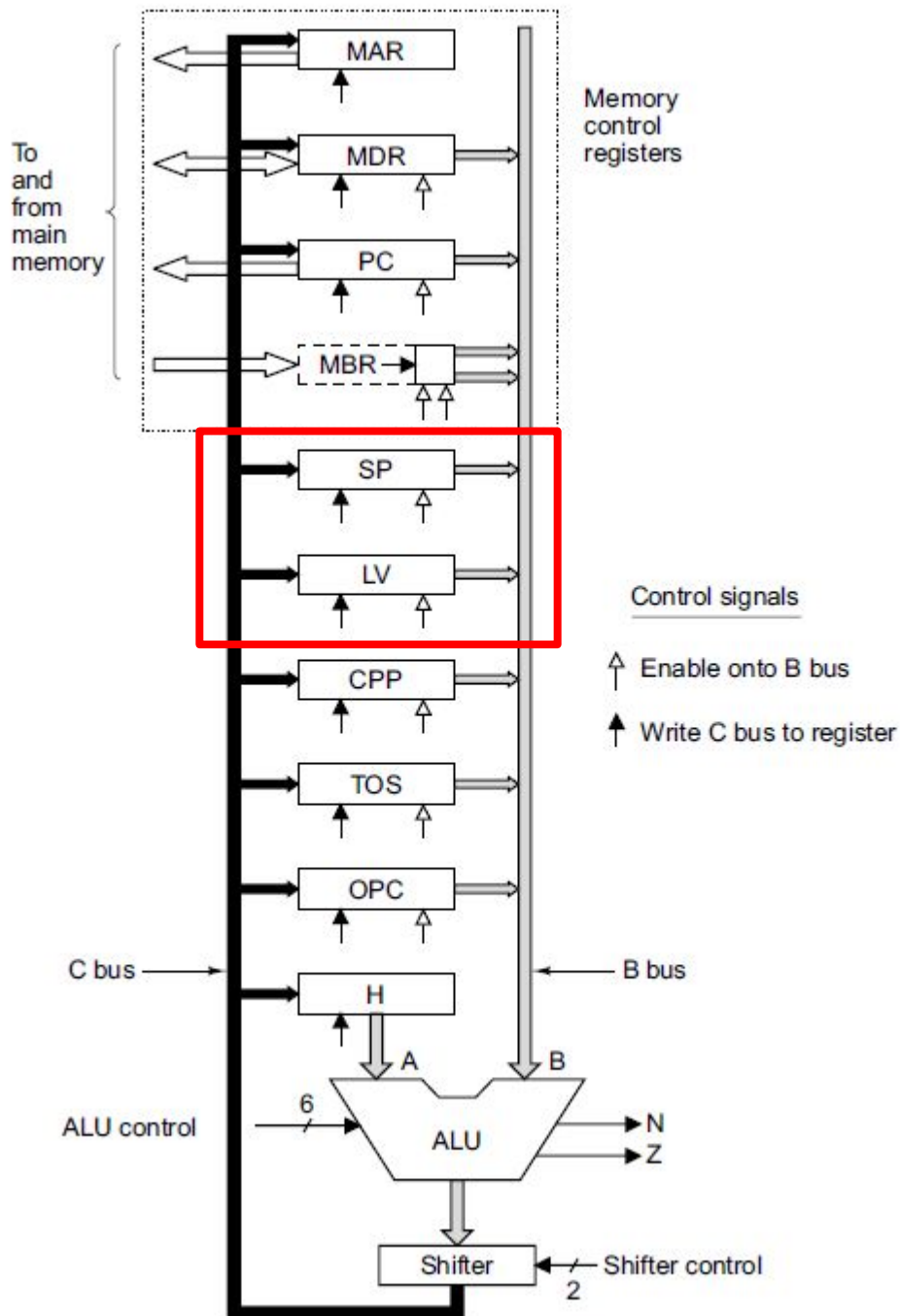


Flujo de datos de la JVM

- Registros de usos específicos
- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held

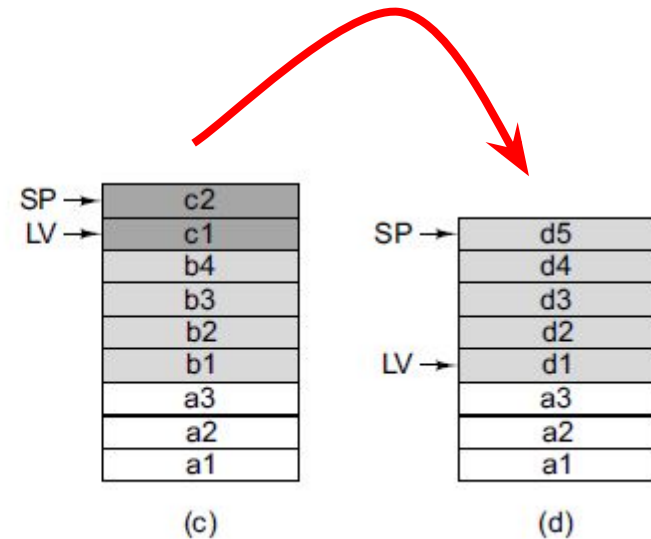


El proc. B invoca al proc. C

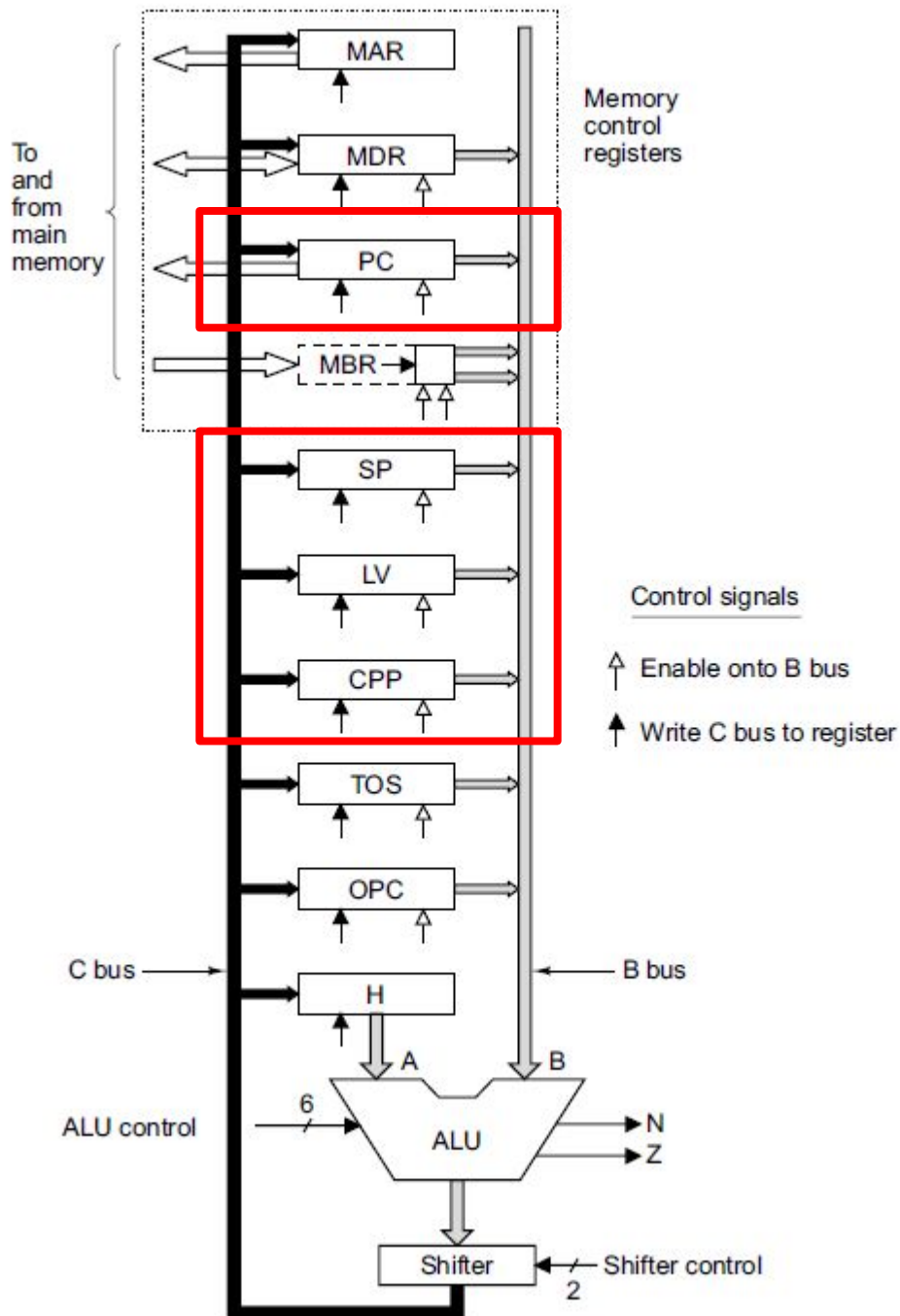


Flujo de datos de la JVM

- Registros de usos específicos
- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held

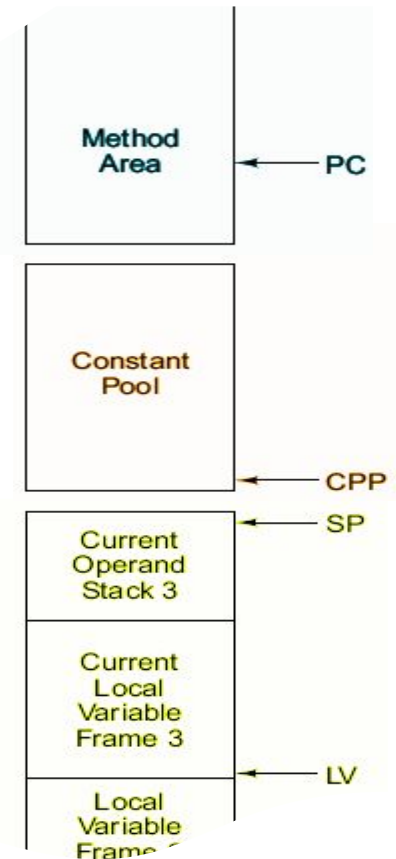


Retorna al proc. A e invoca al proc. D

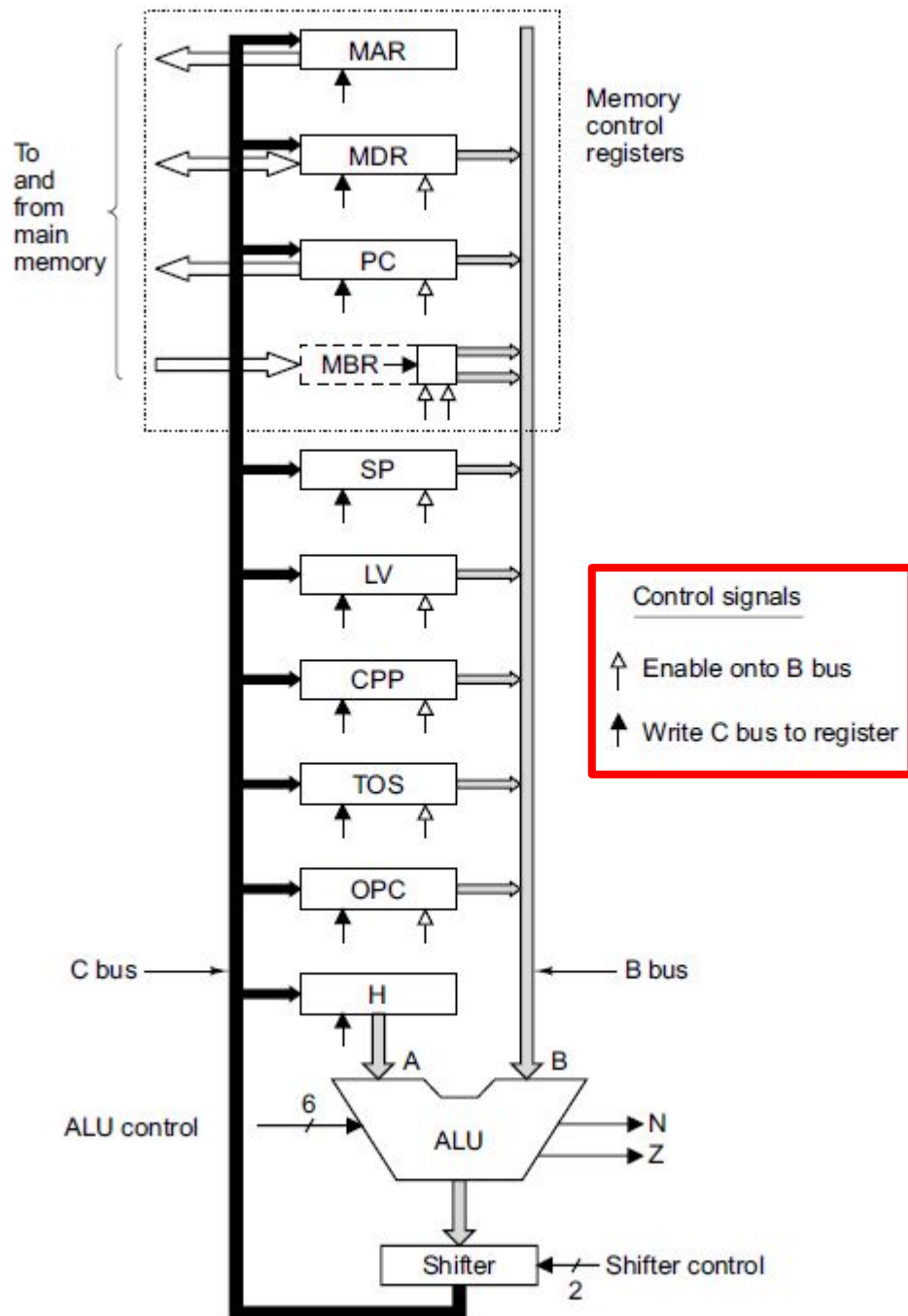


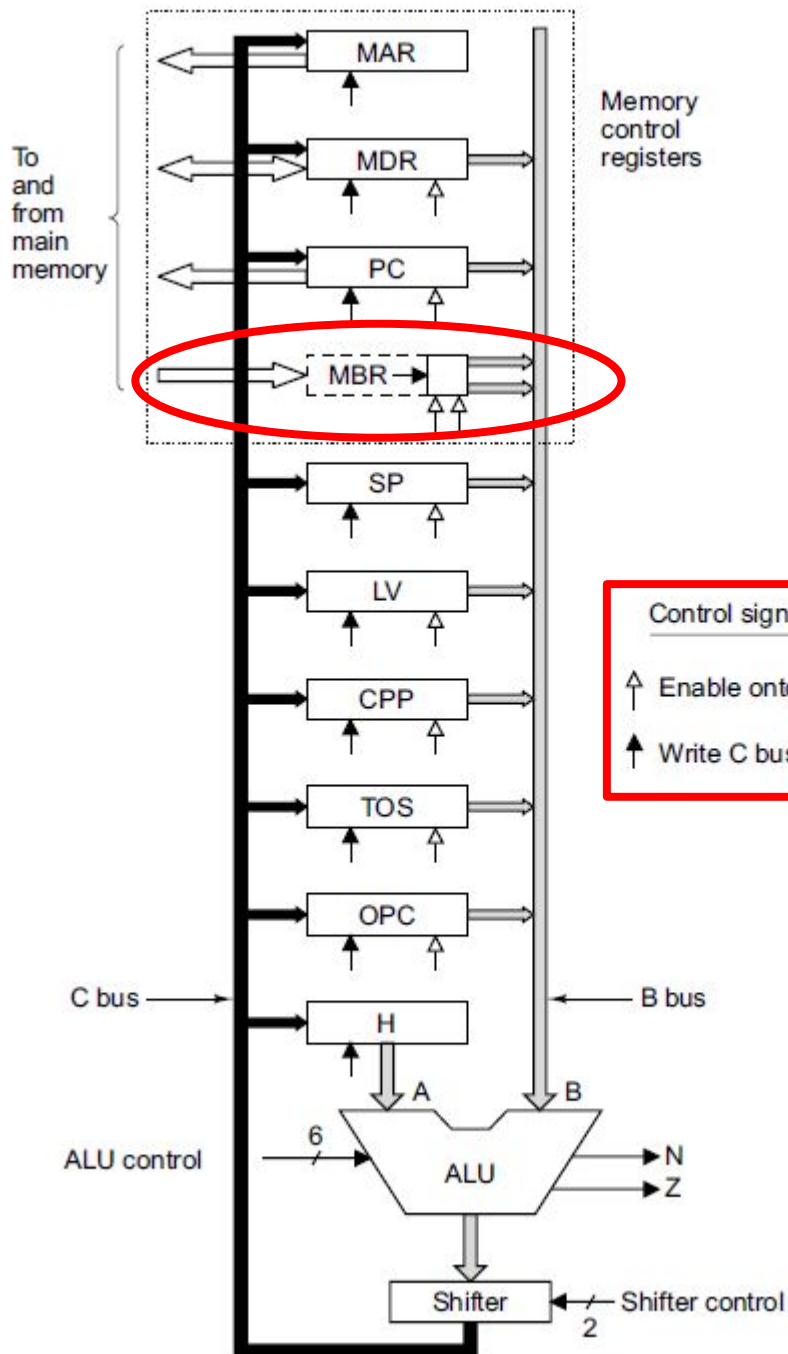
Flujo de datos de la JVM

- Registros de usos específicos
- Stack Pointer
- Local Variables
- Constant Pool Pointer
- Top Of Stack
- Old PC / Auxiliar
- Held



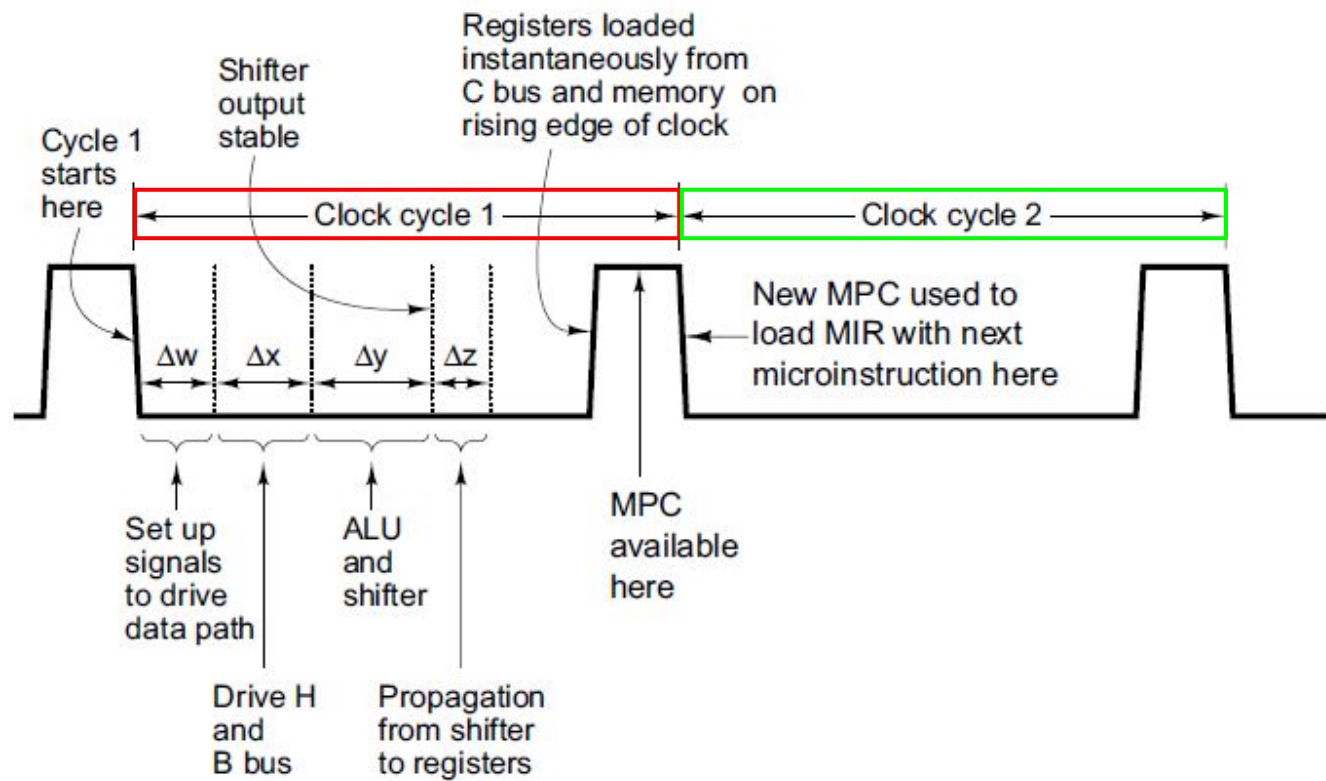
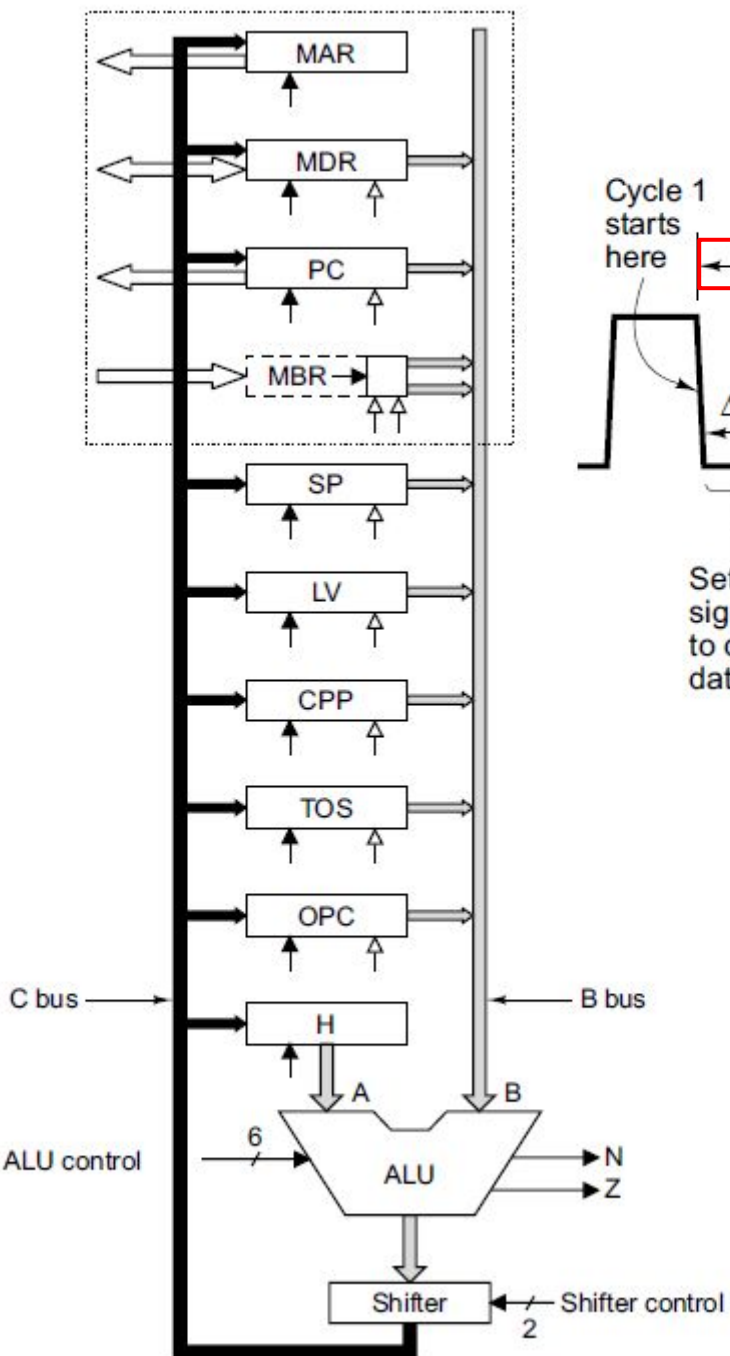
Flujo de datos de la JVM

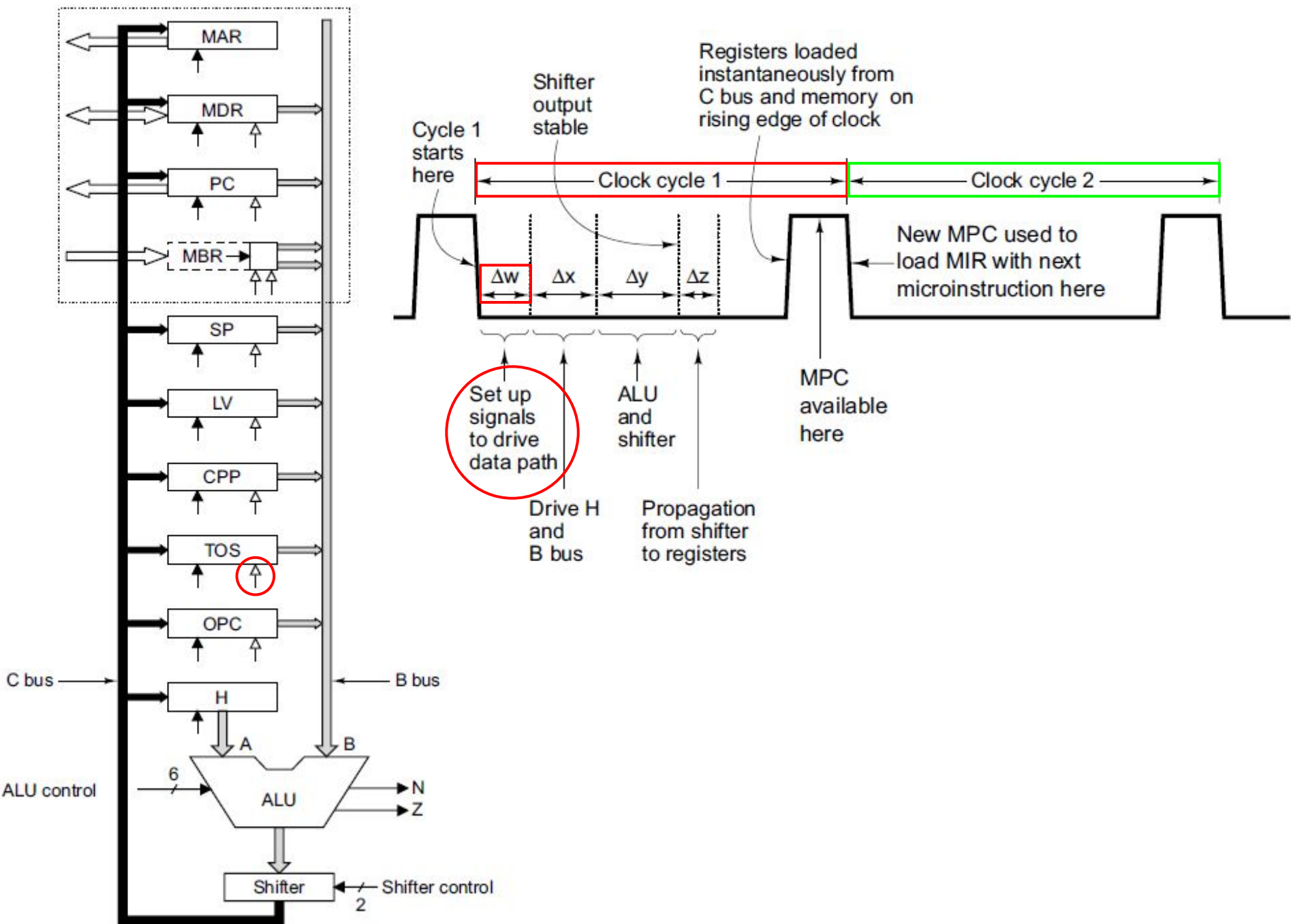


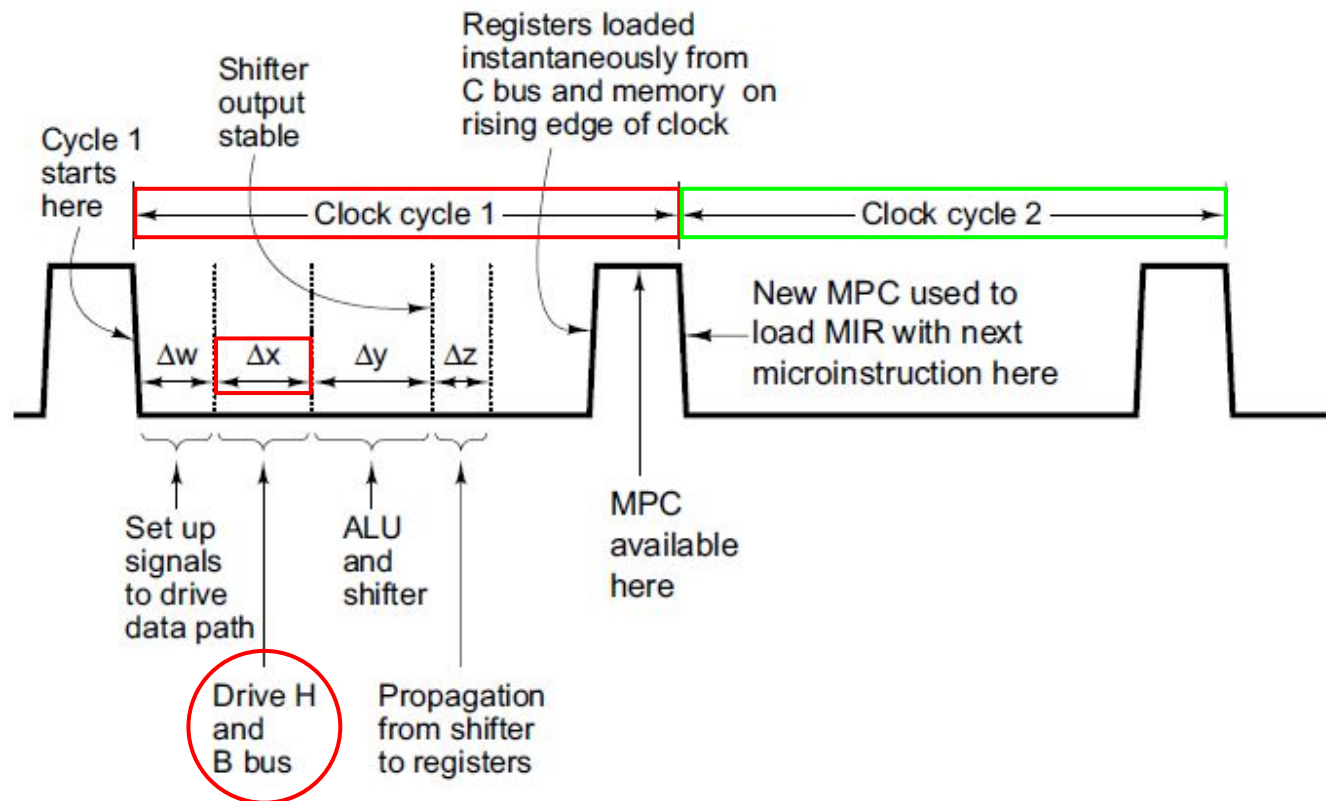
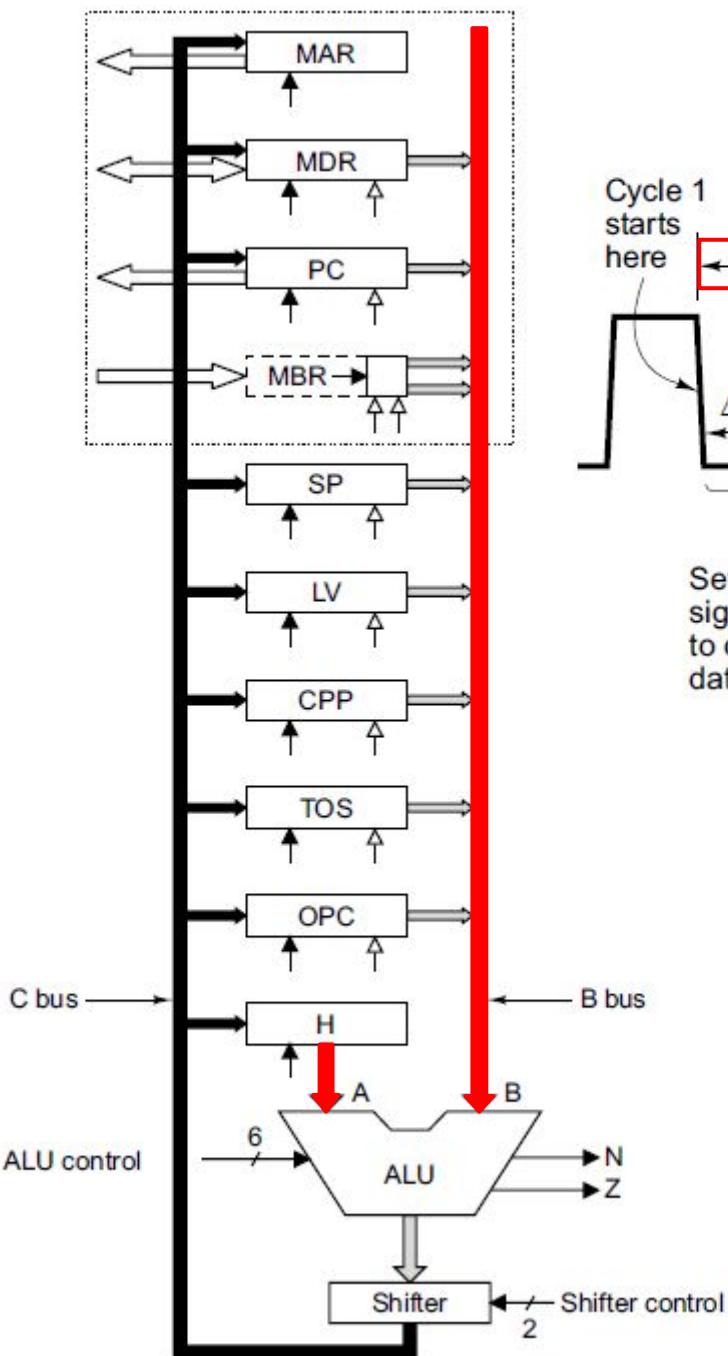


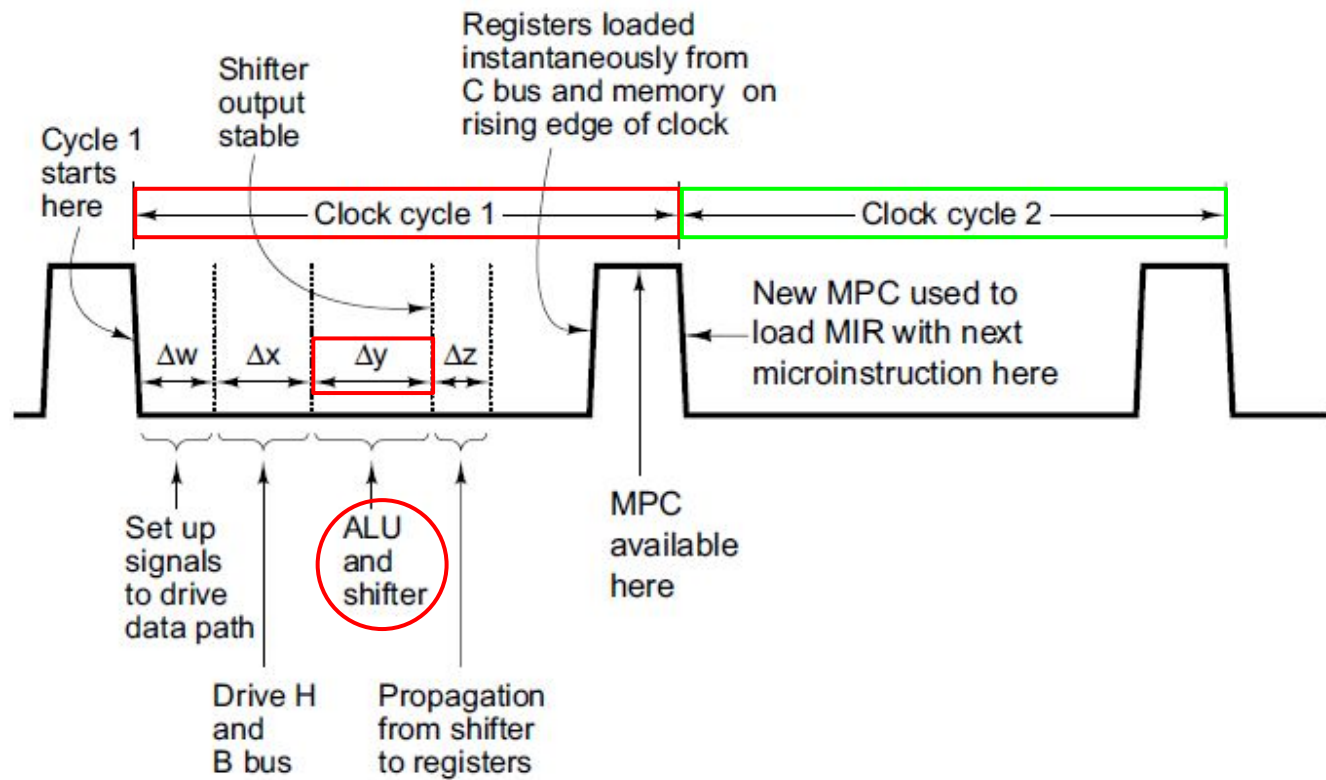
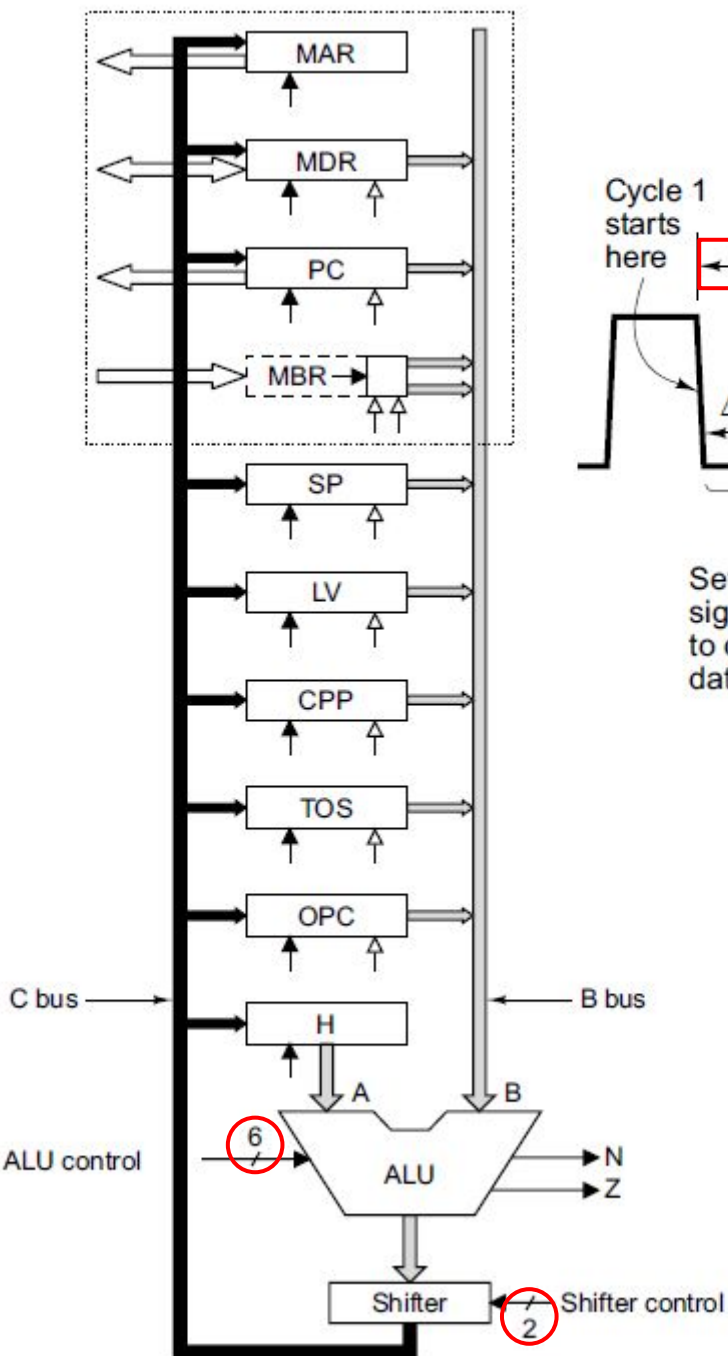
Flujo de datos de la JVM

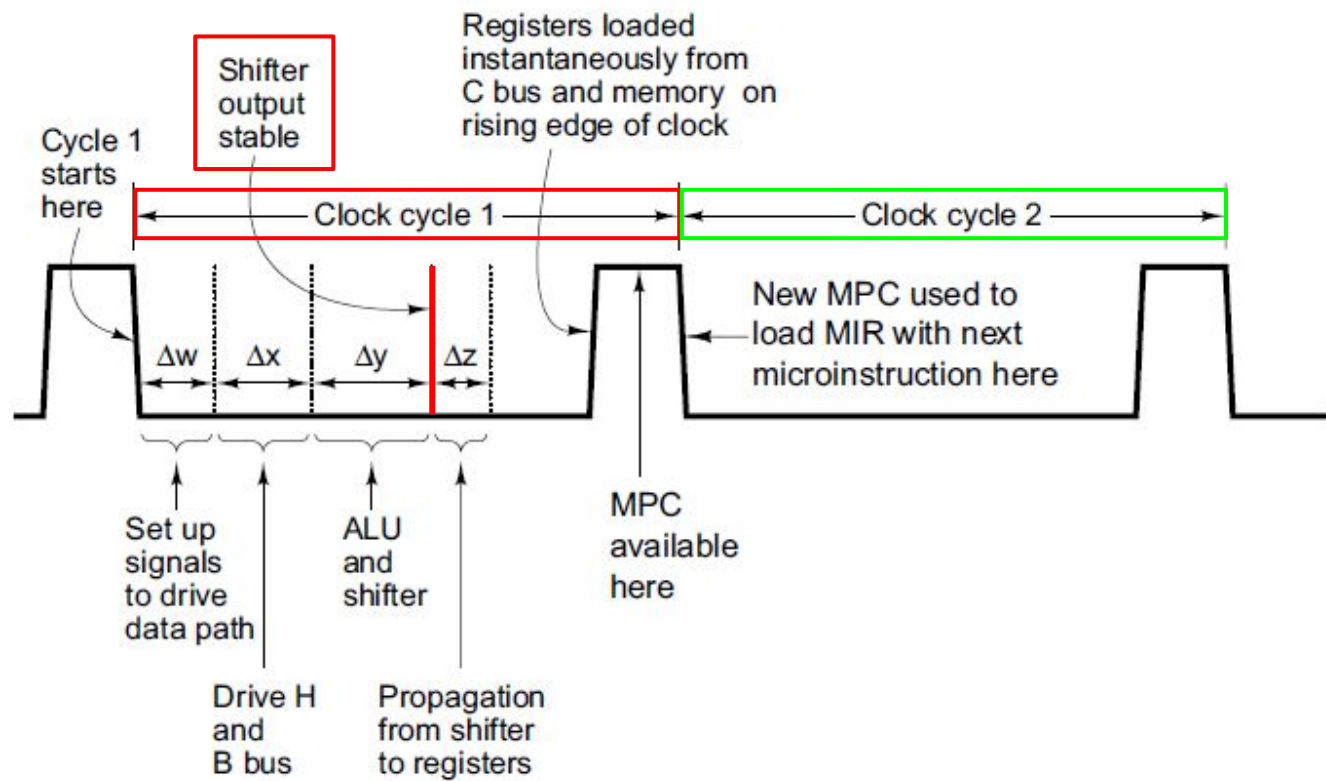
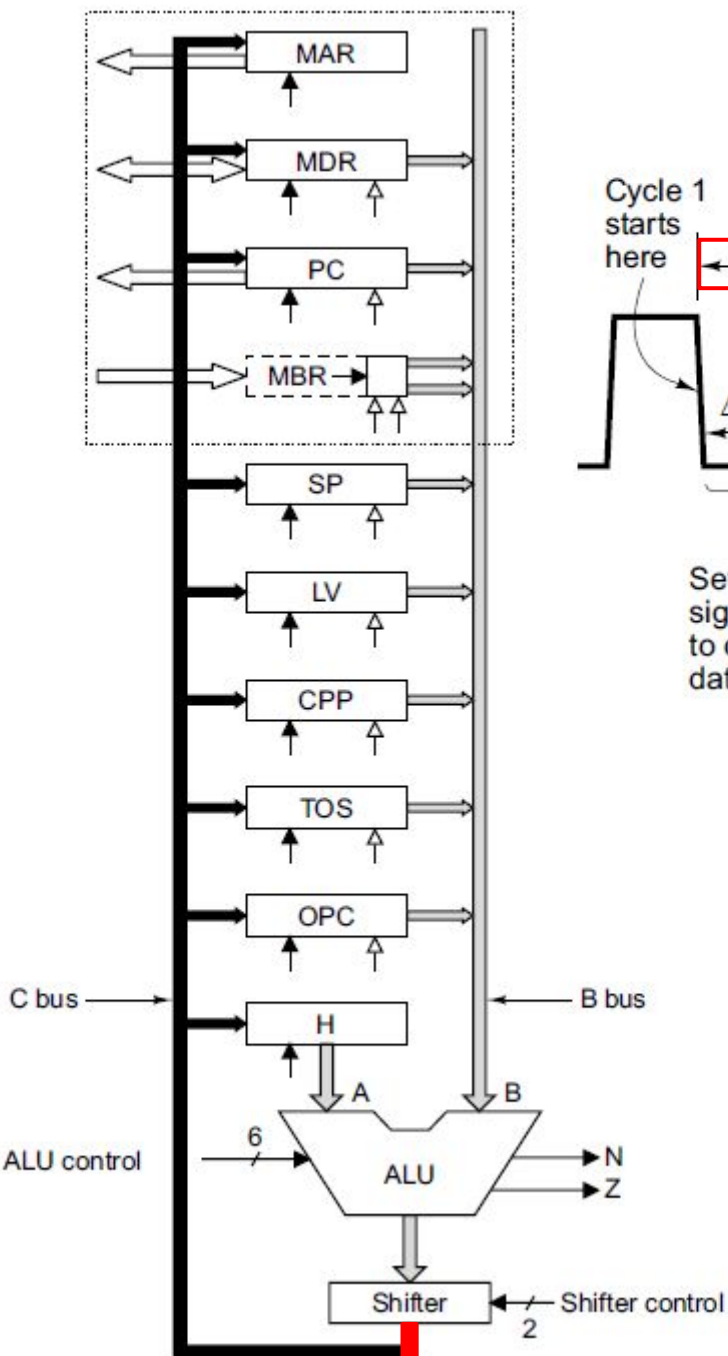
- MBR tiene 2 salidas al bus B y 2 líneas de control
- Unsigned y Signed

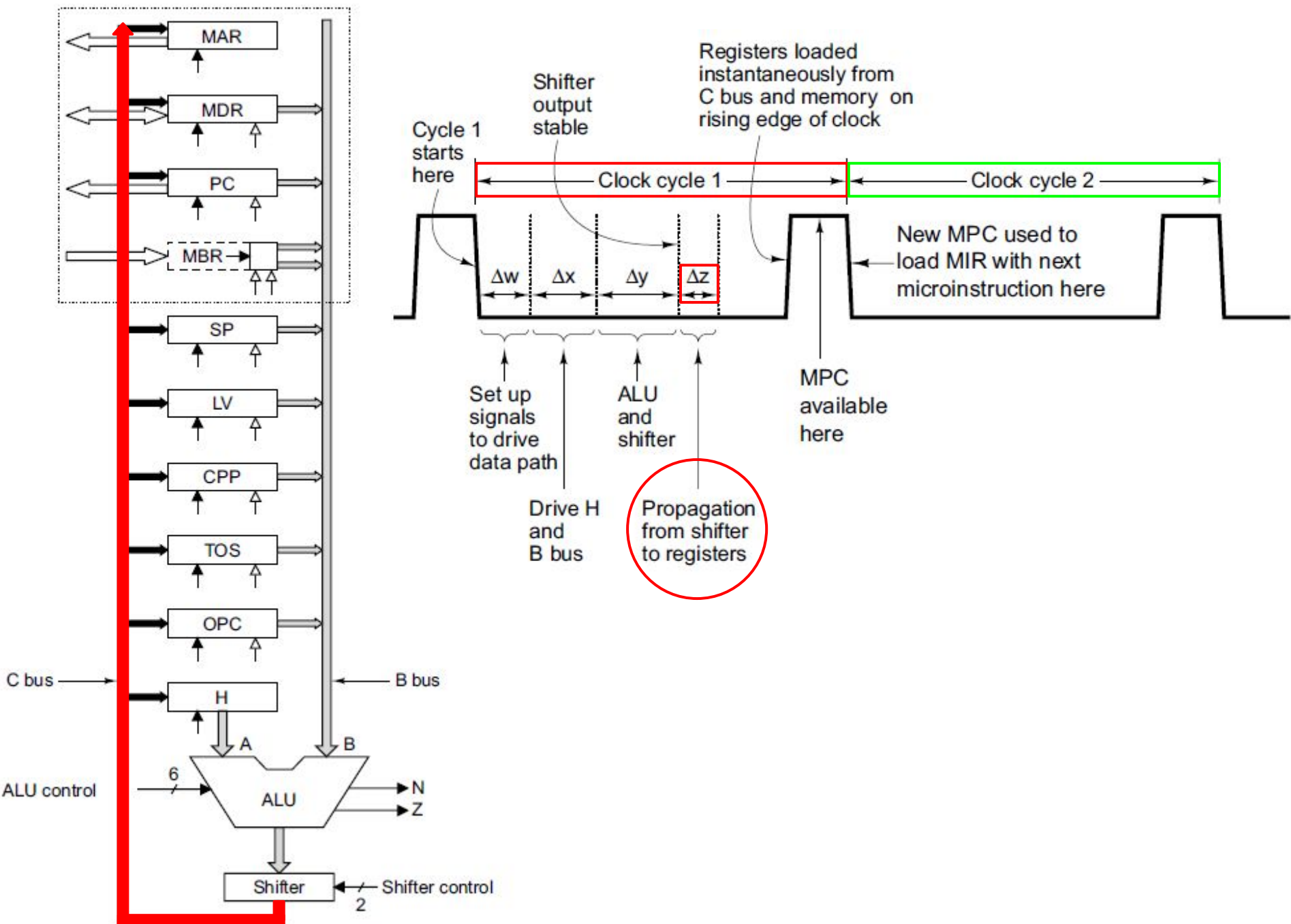


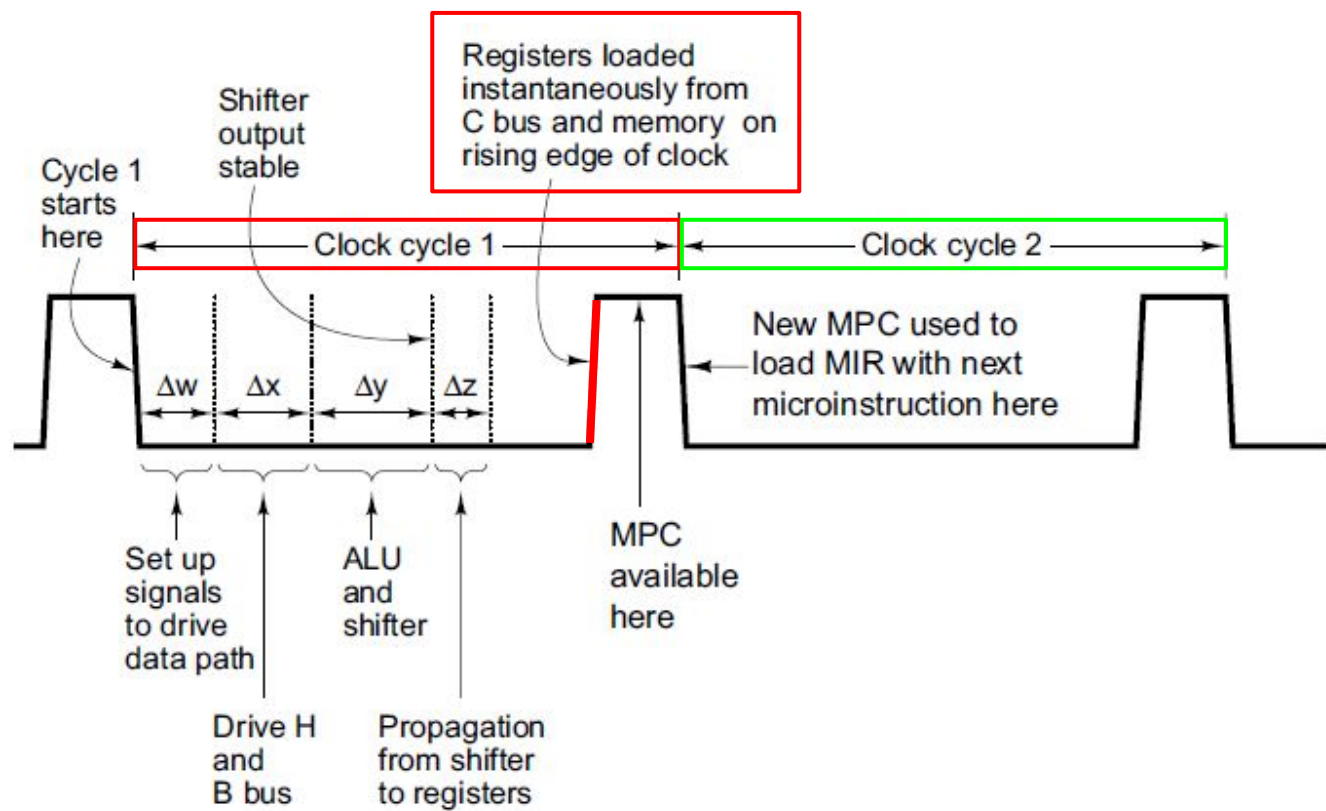
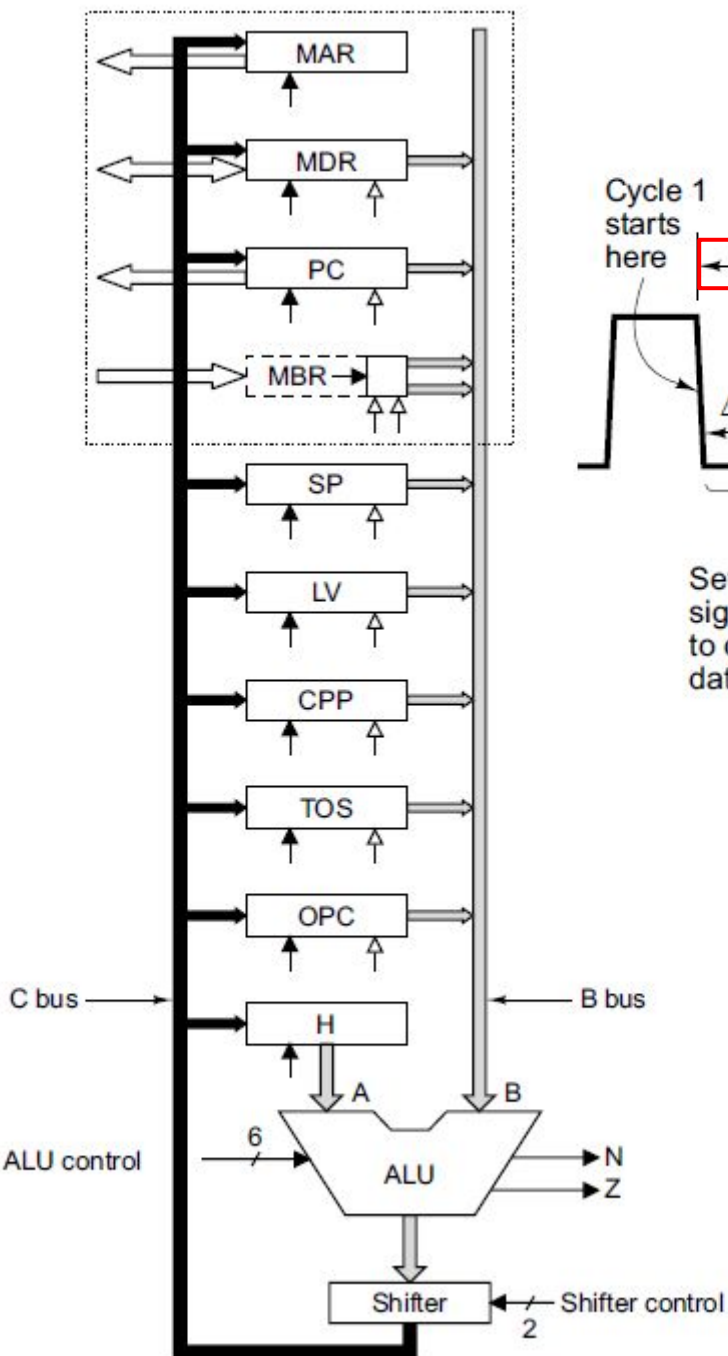


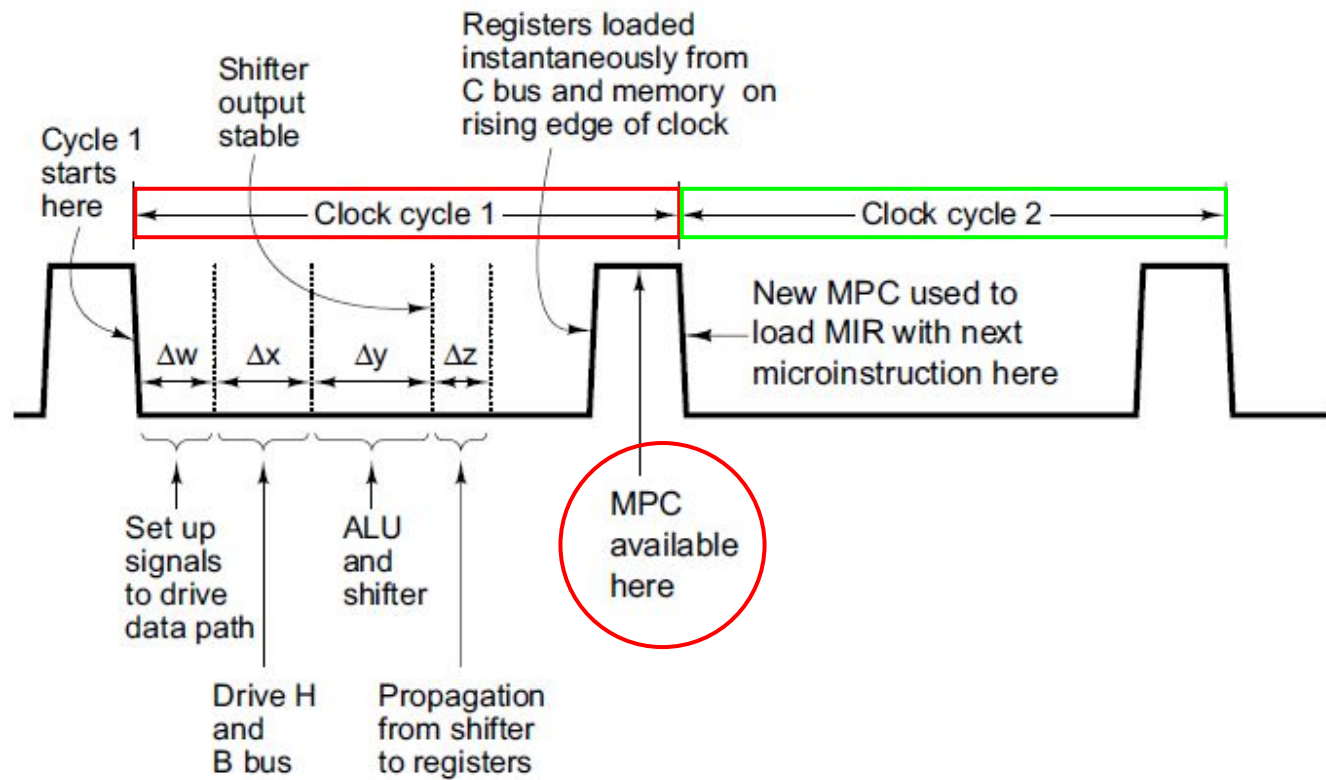
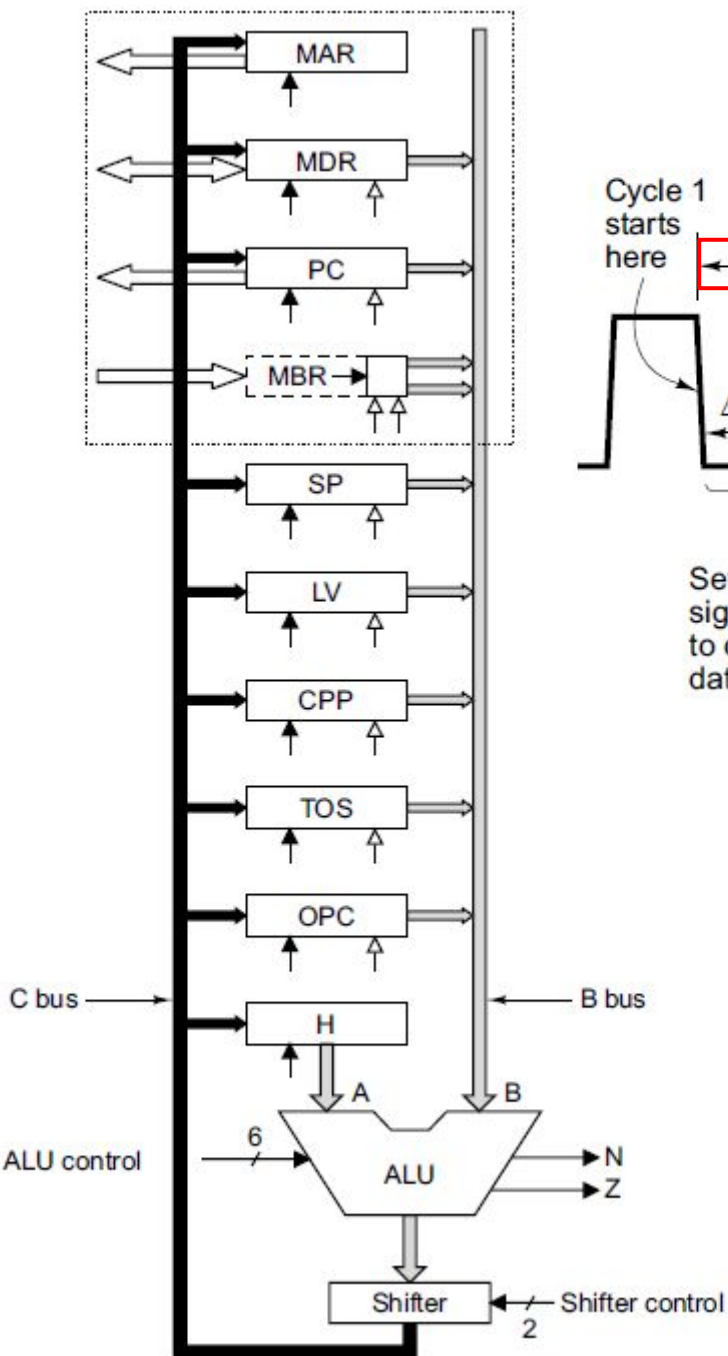


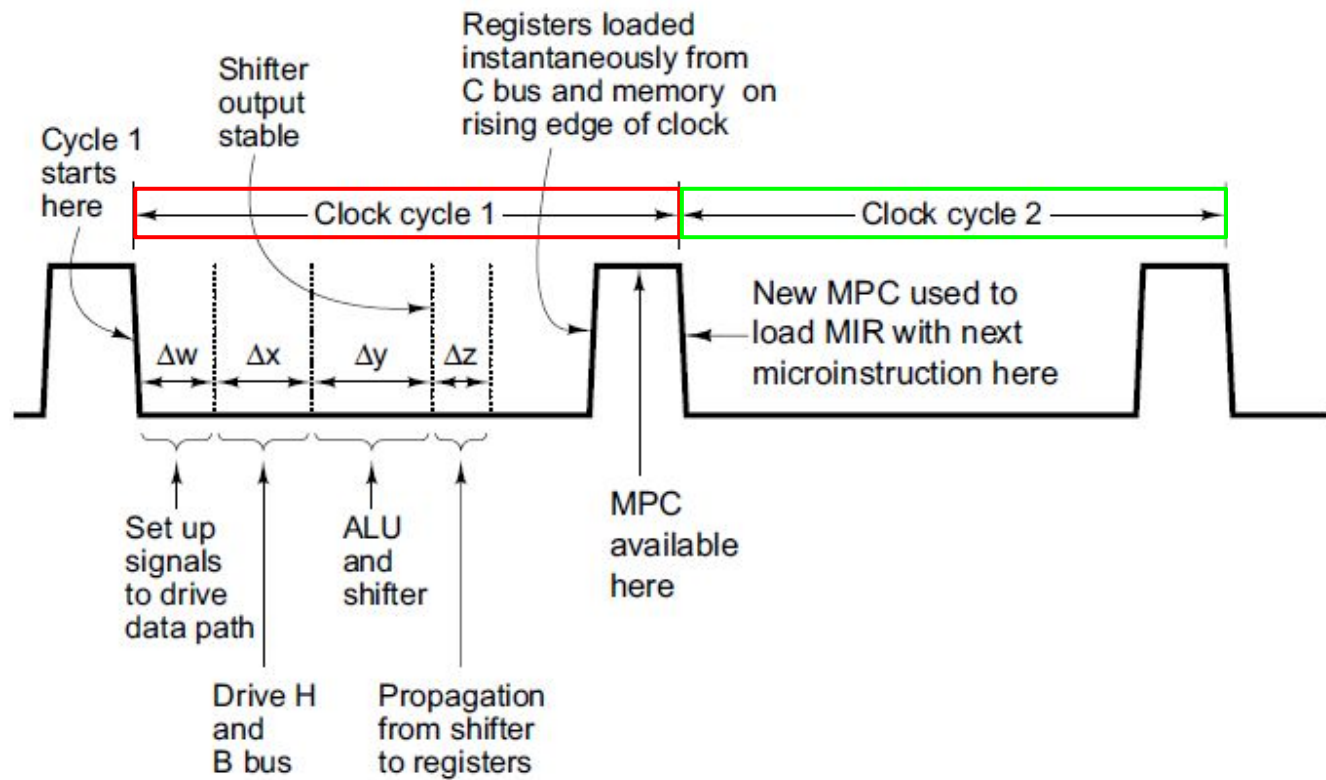
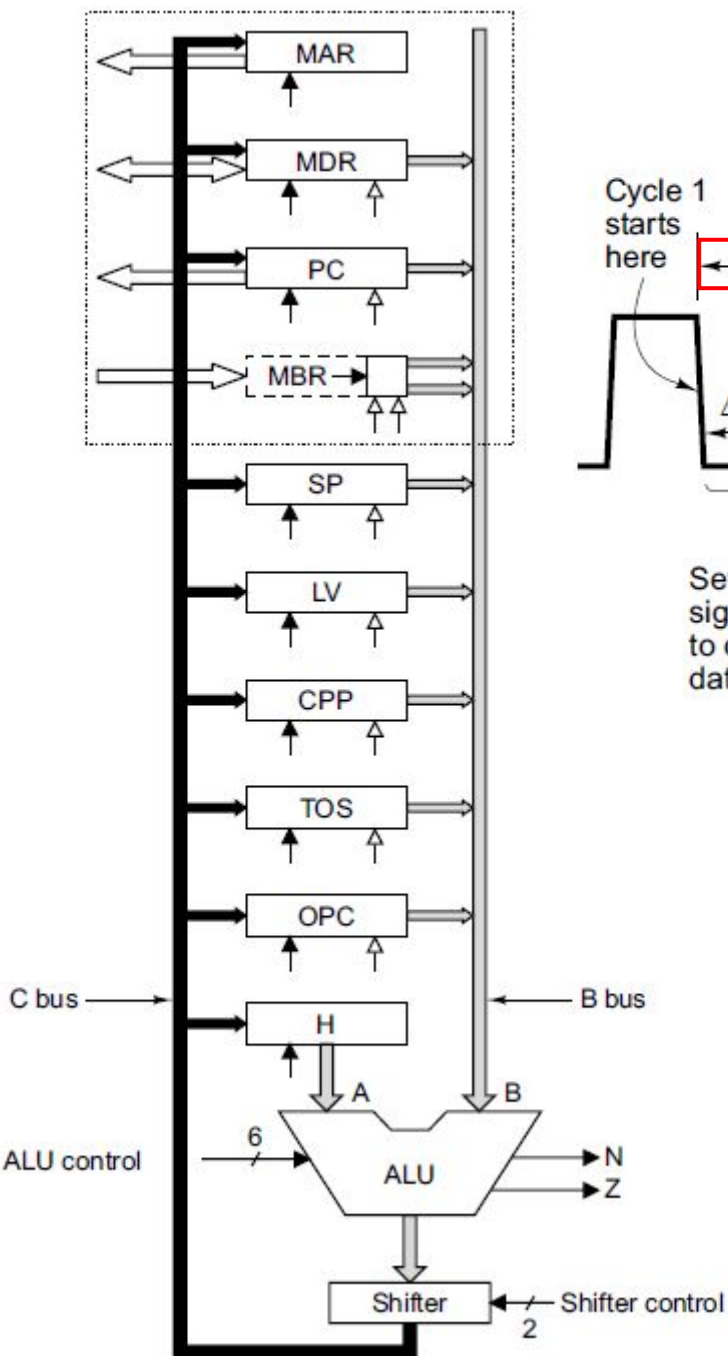










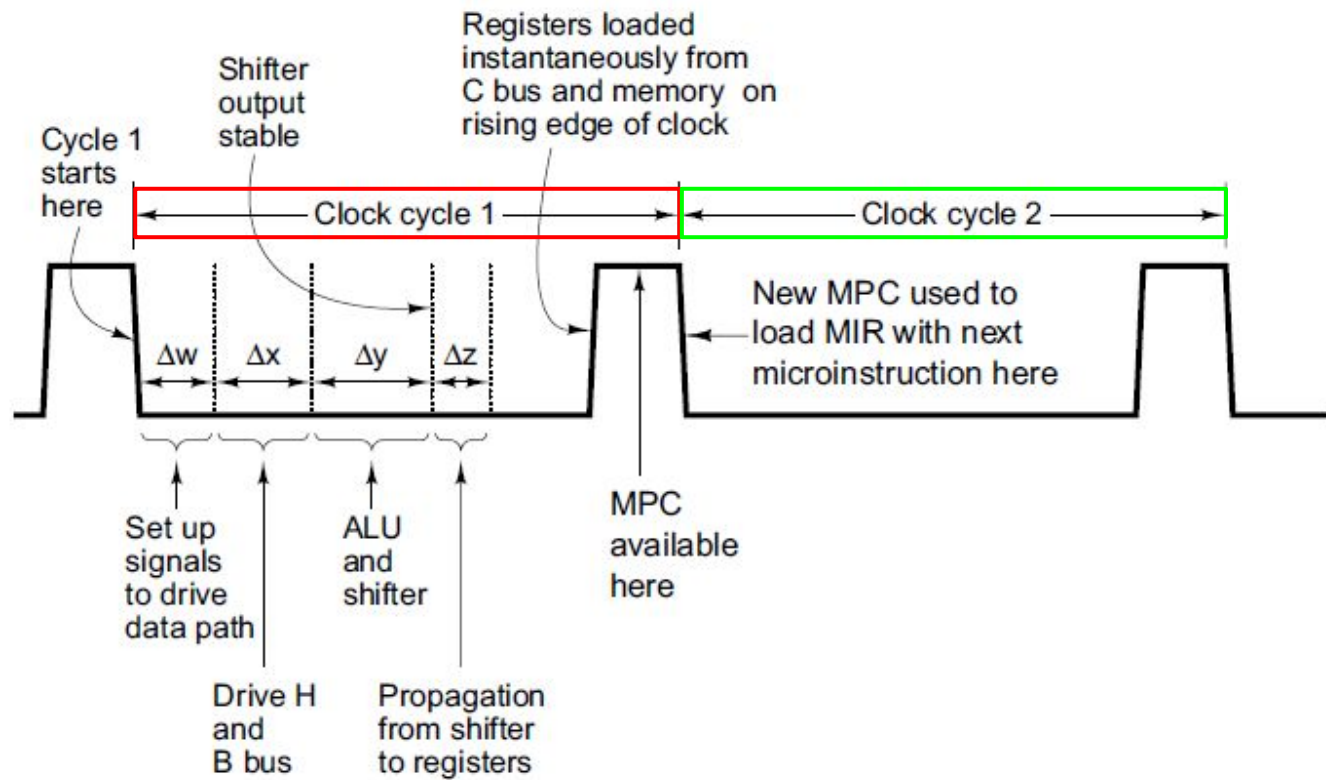
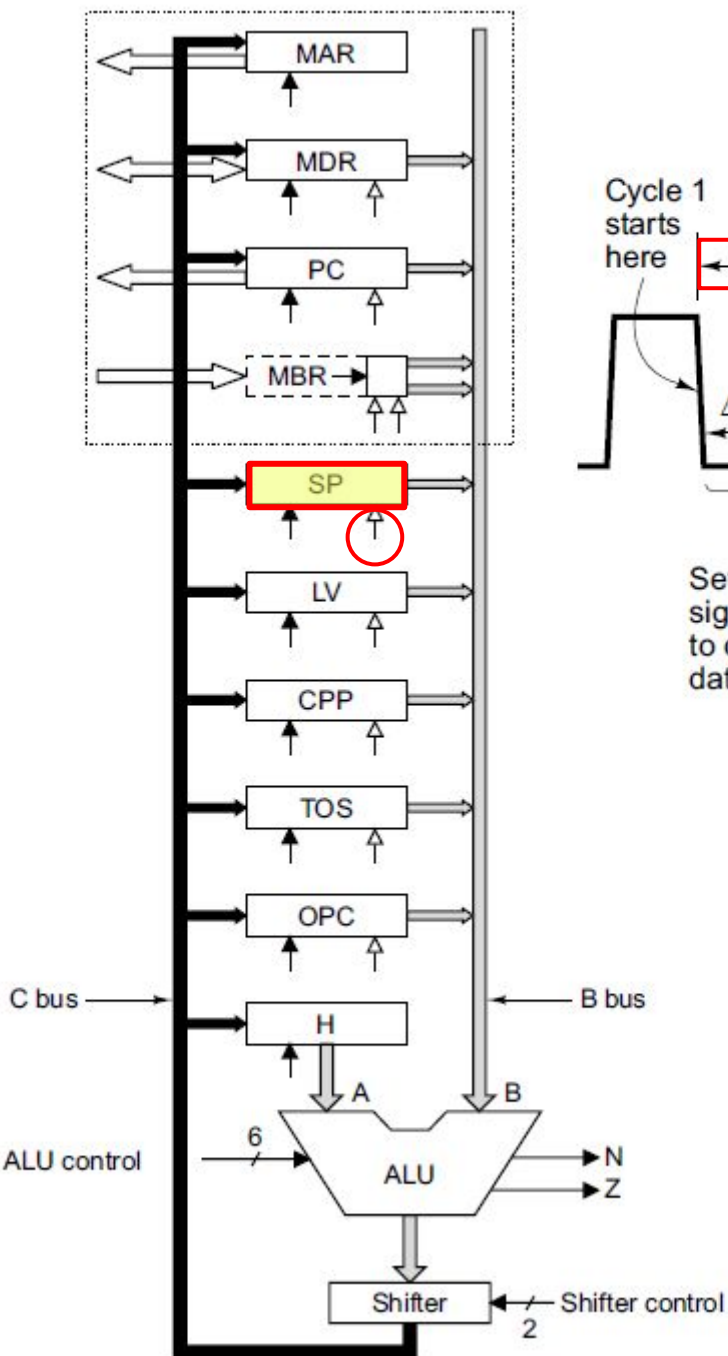


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

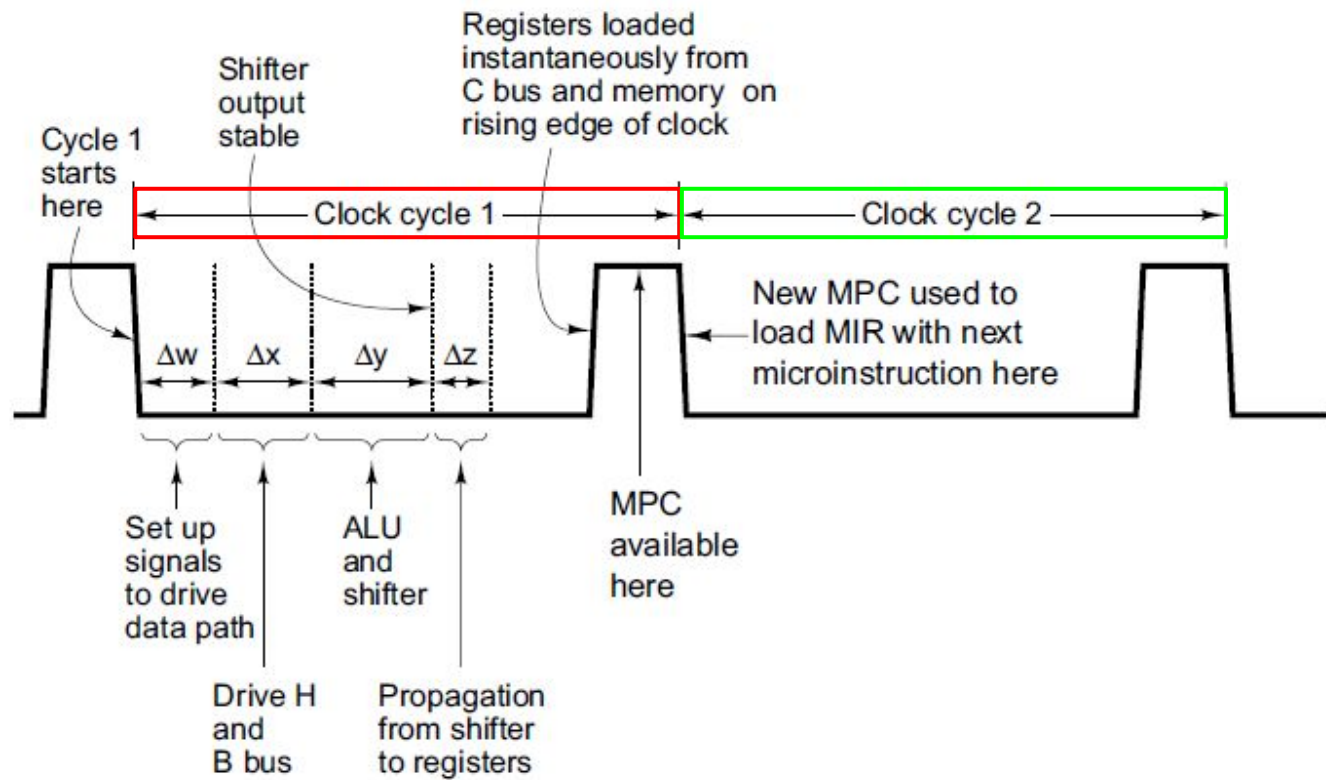
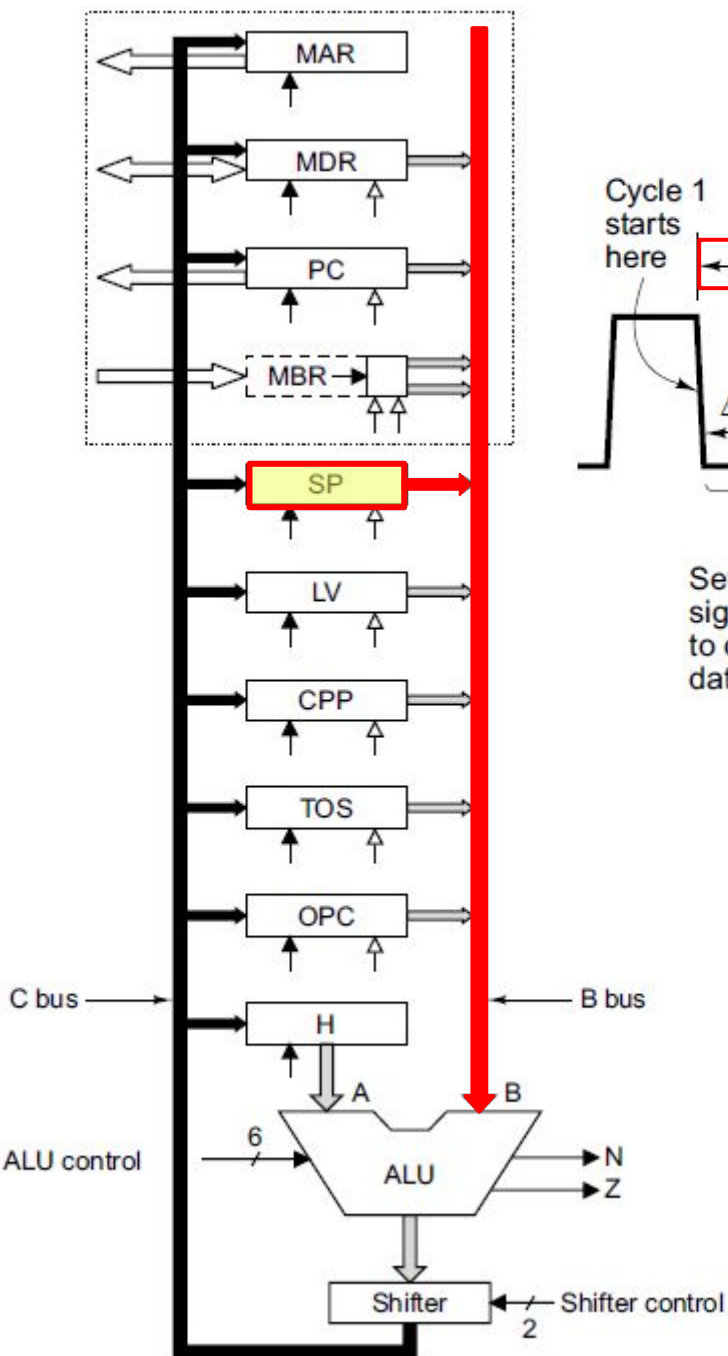


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

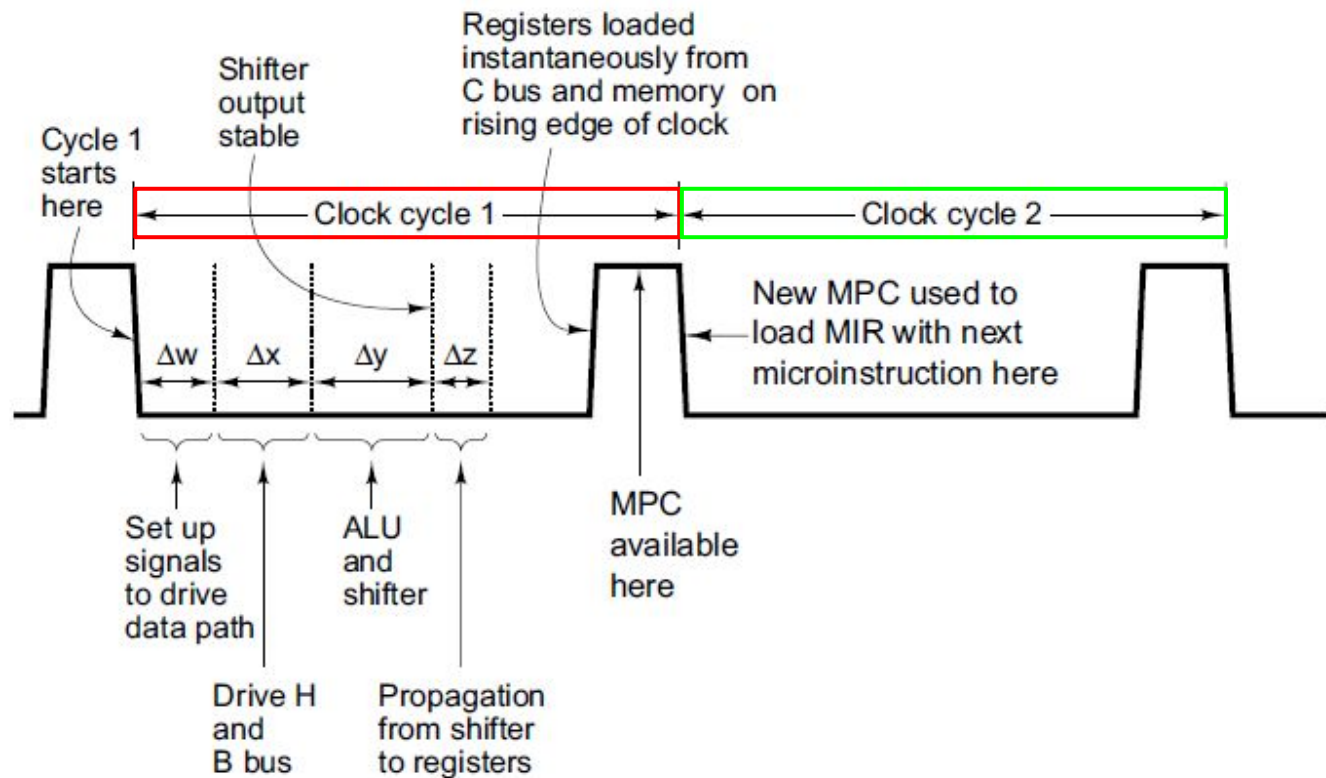
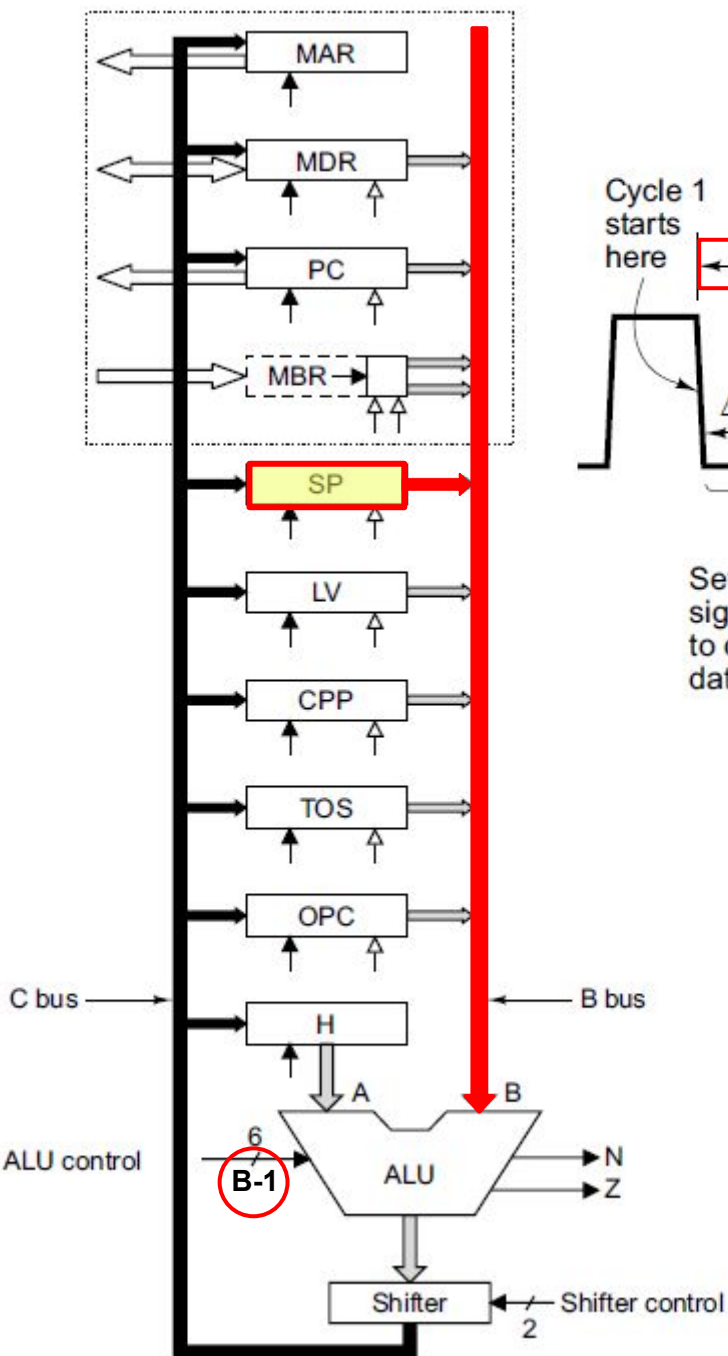


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

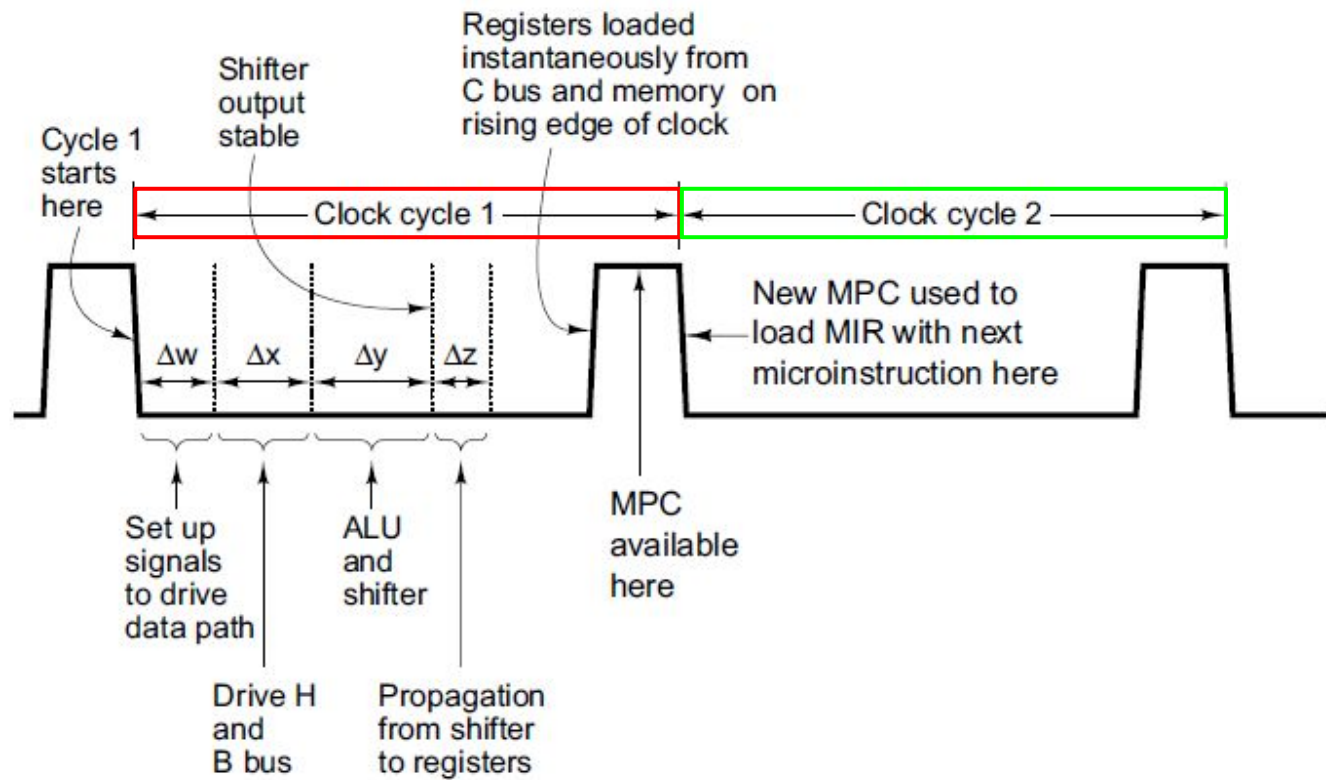
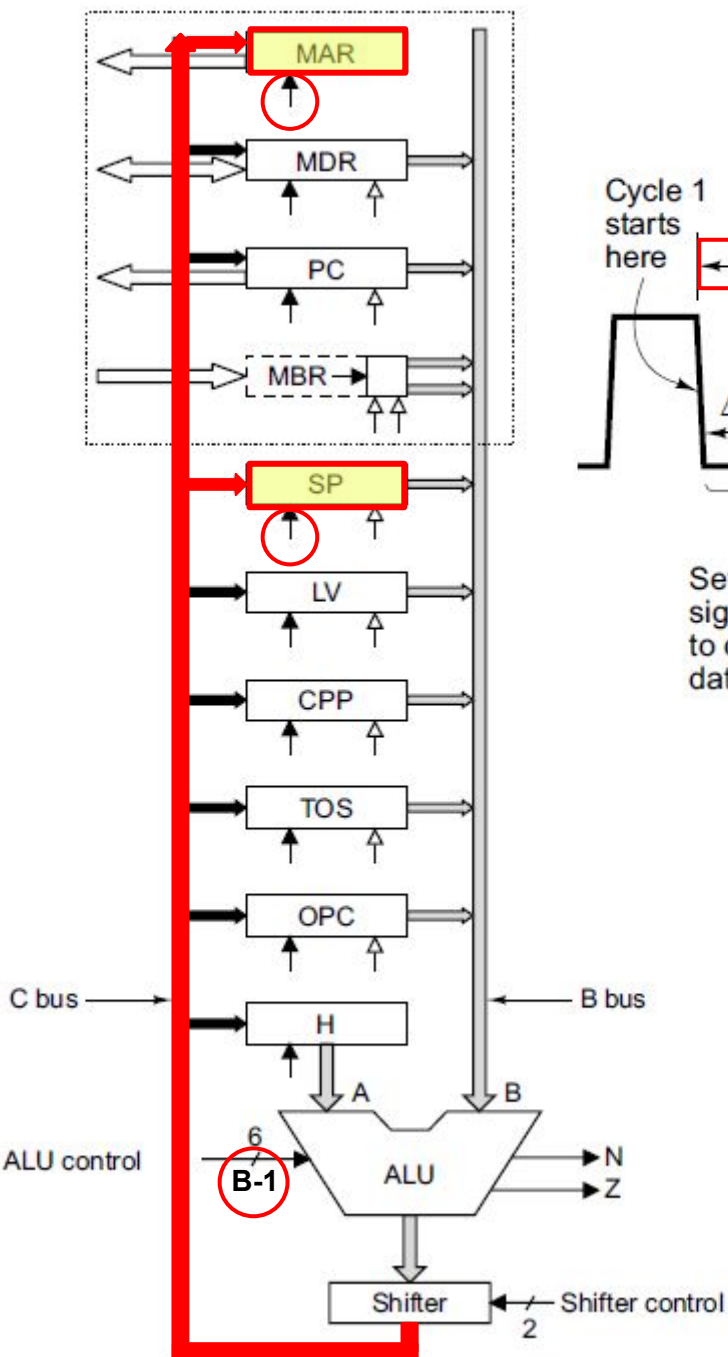


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

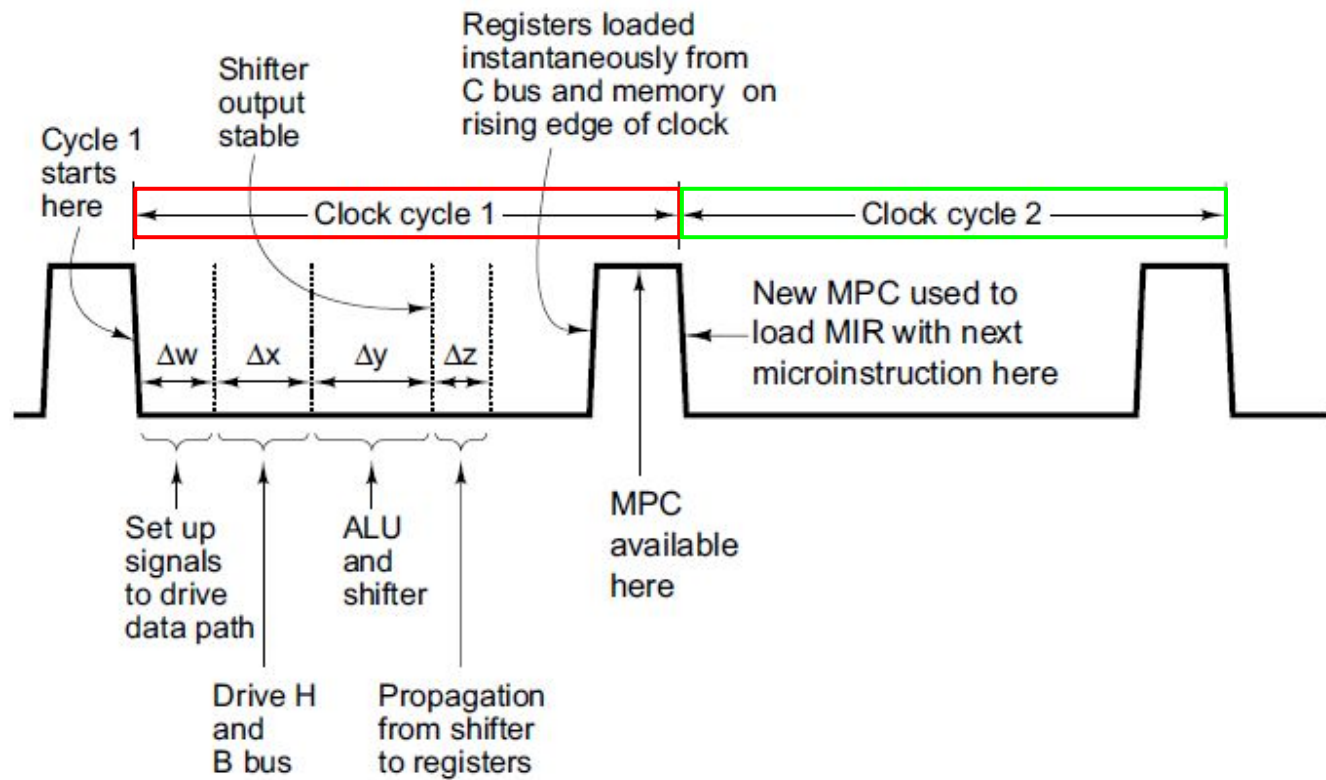
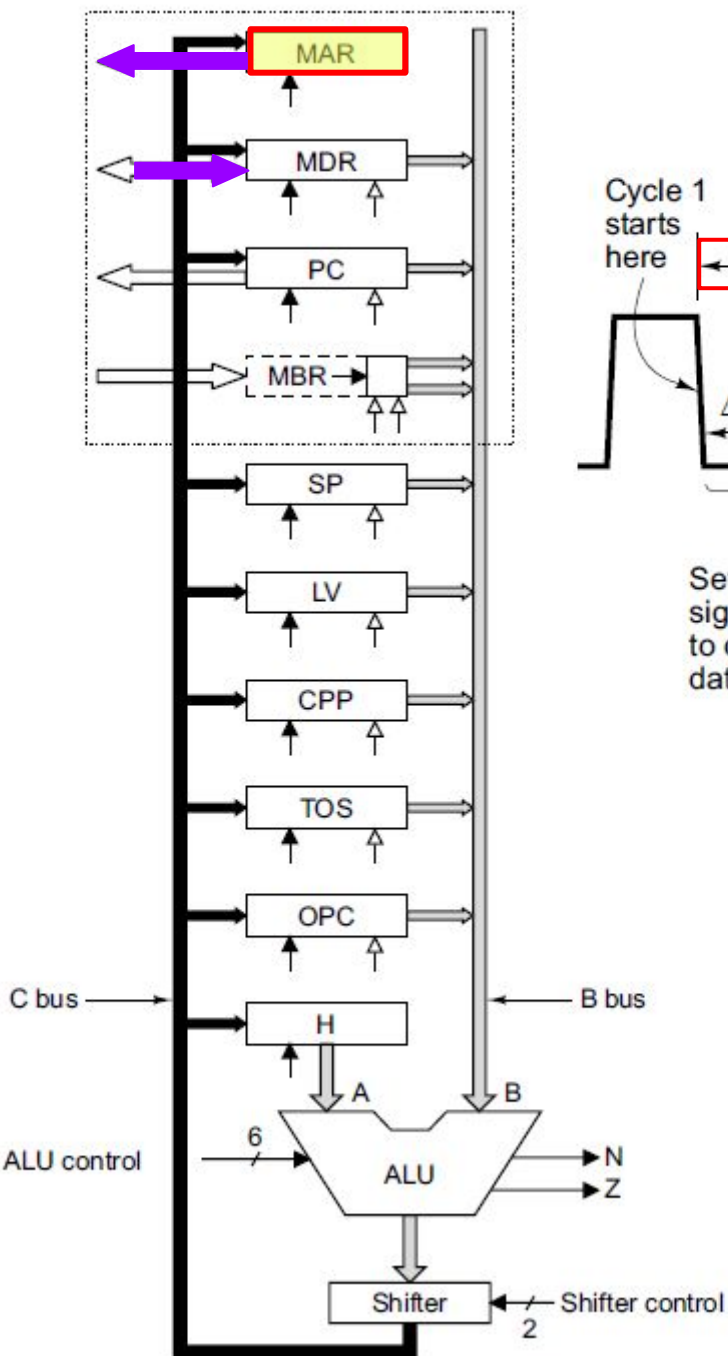


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

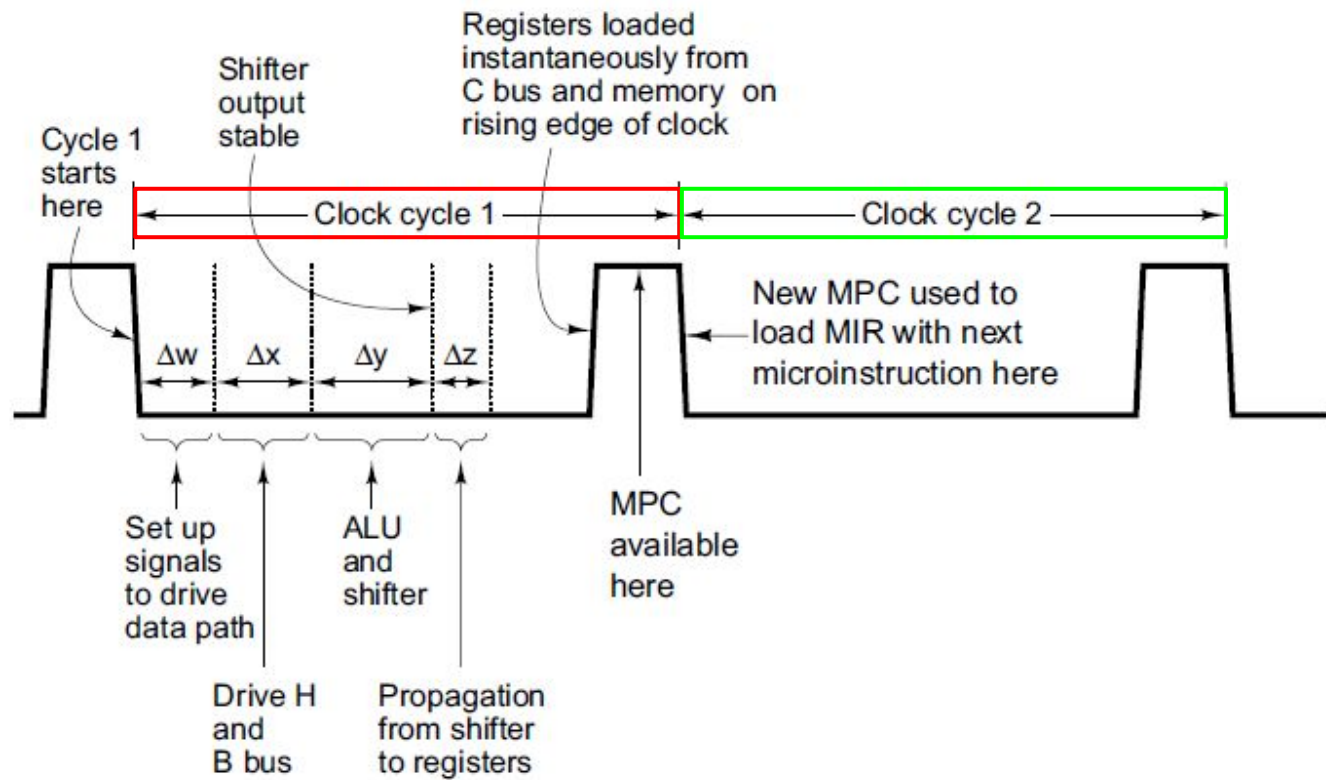
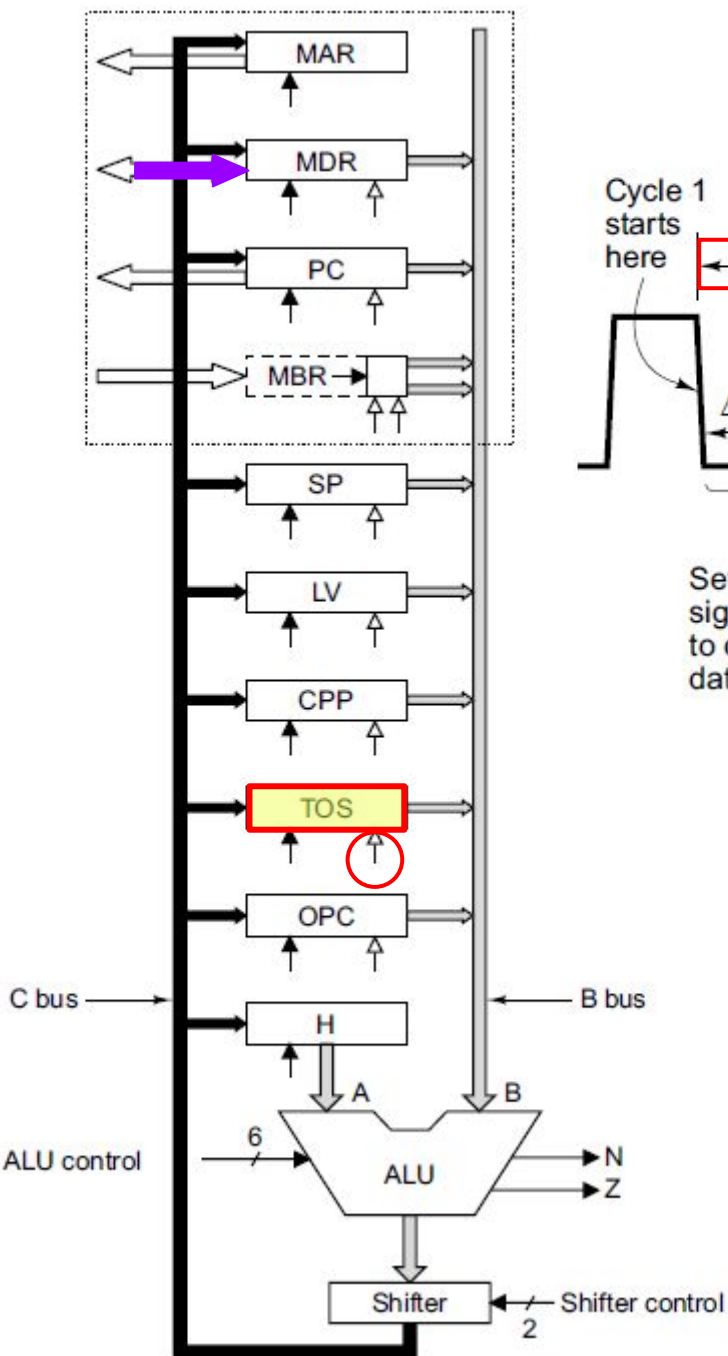


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

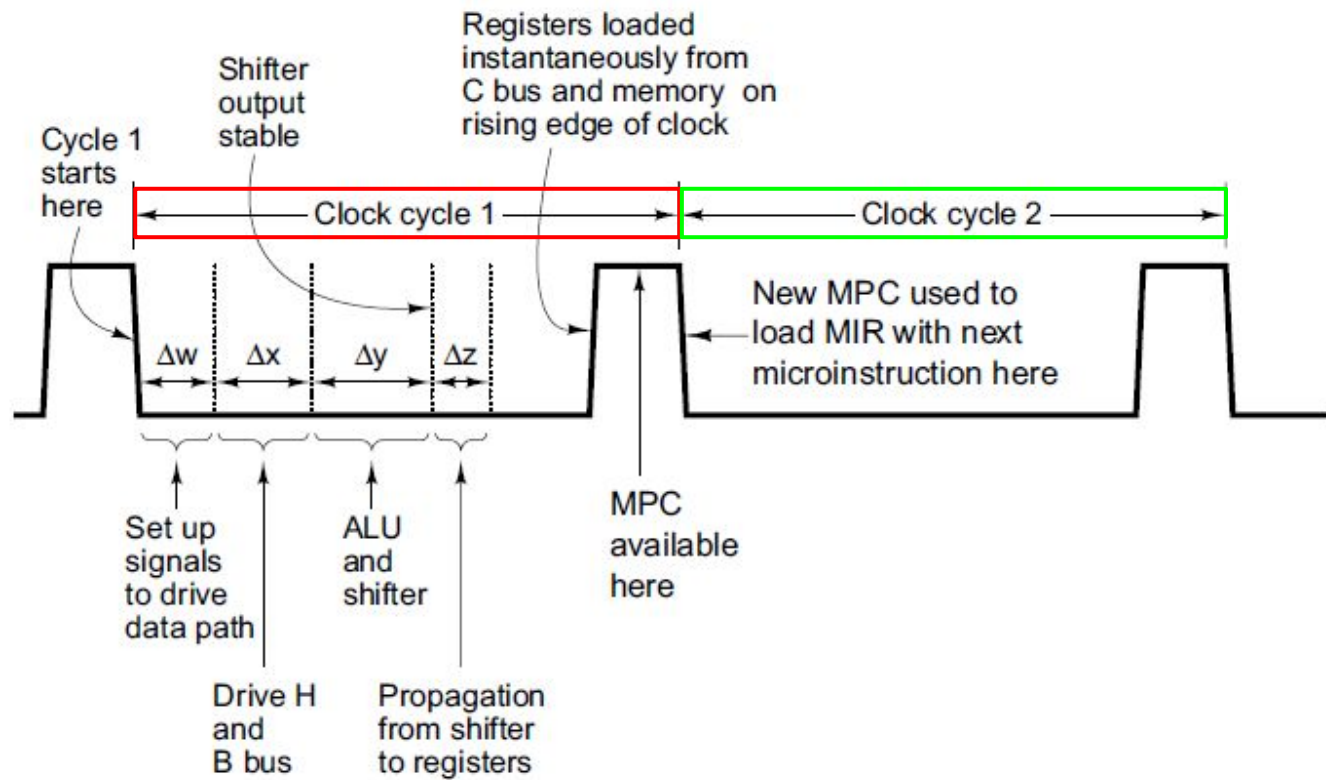
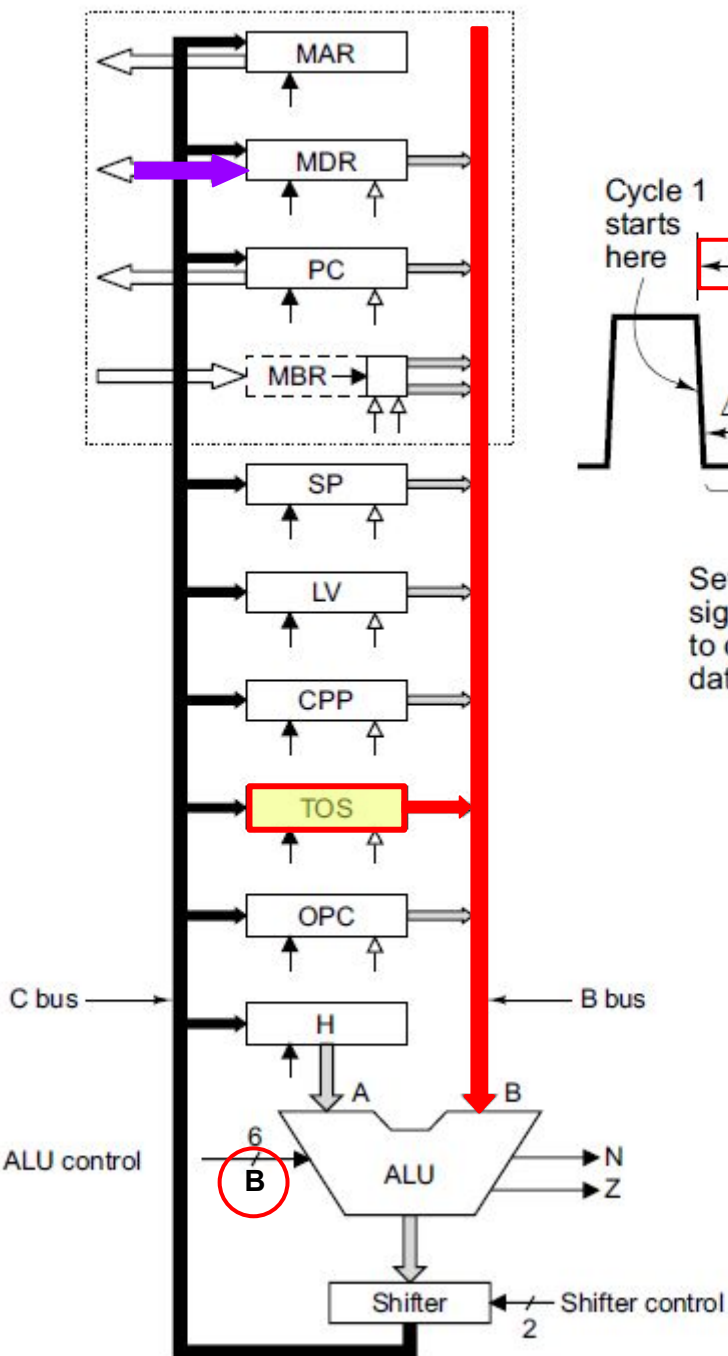


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

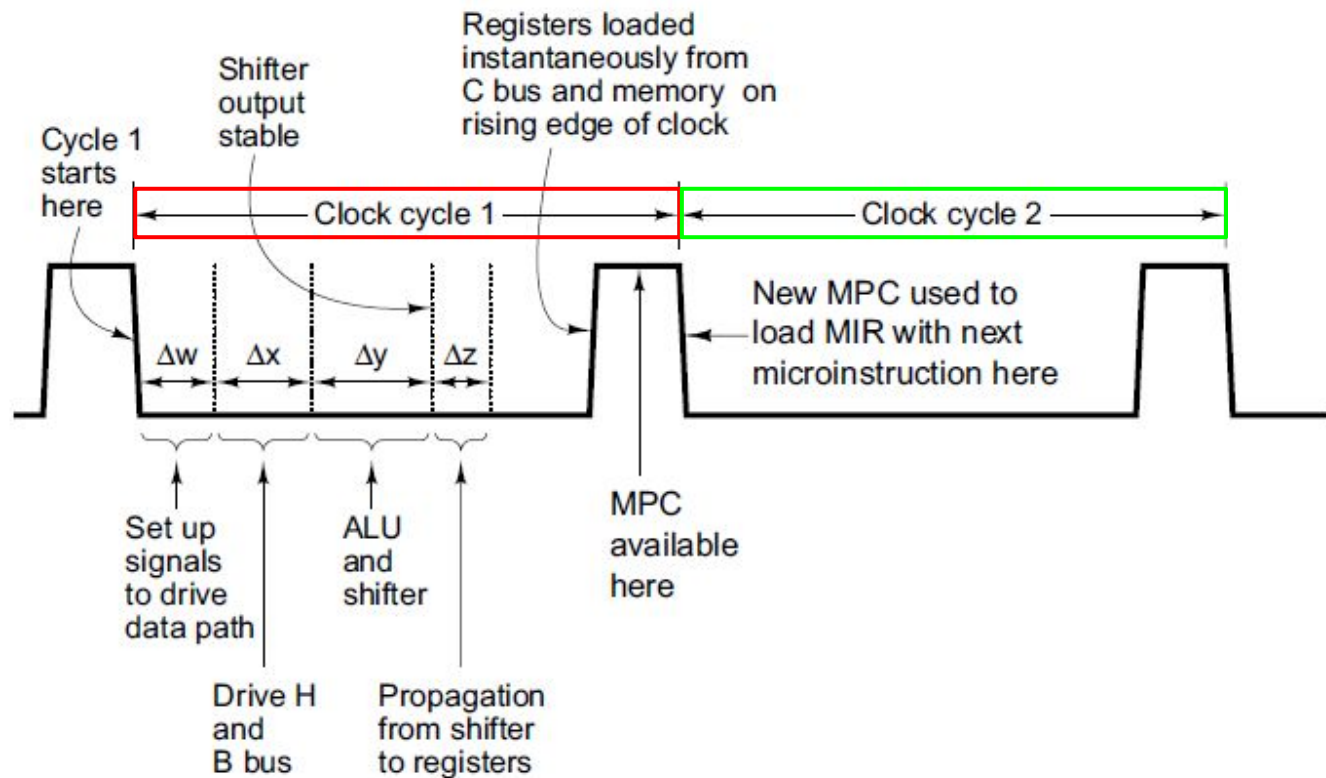
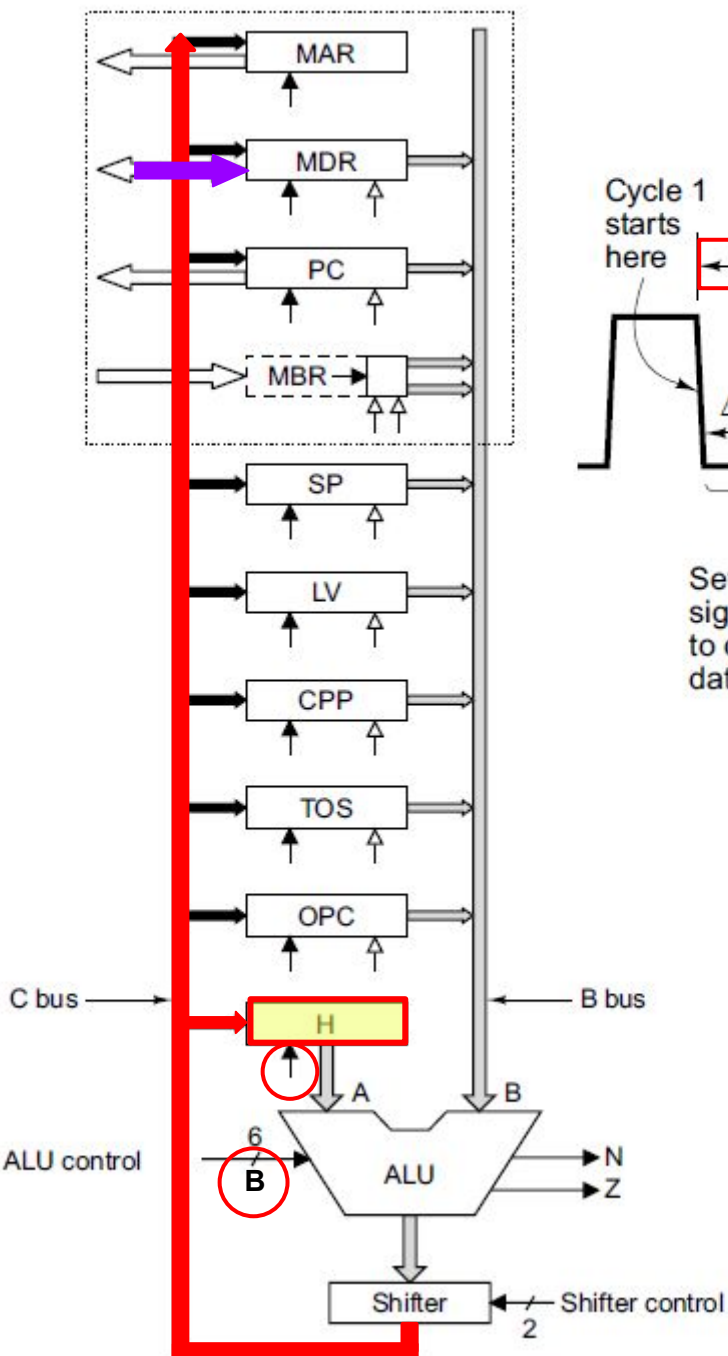


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

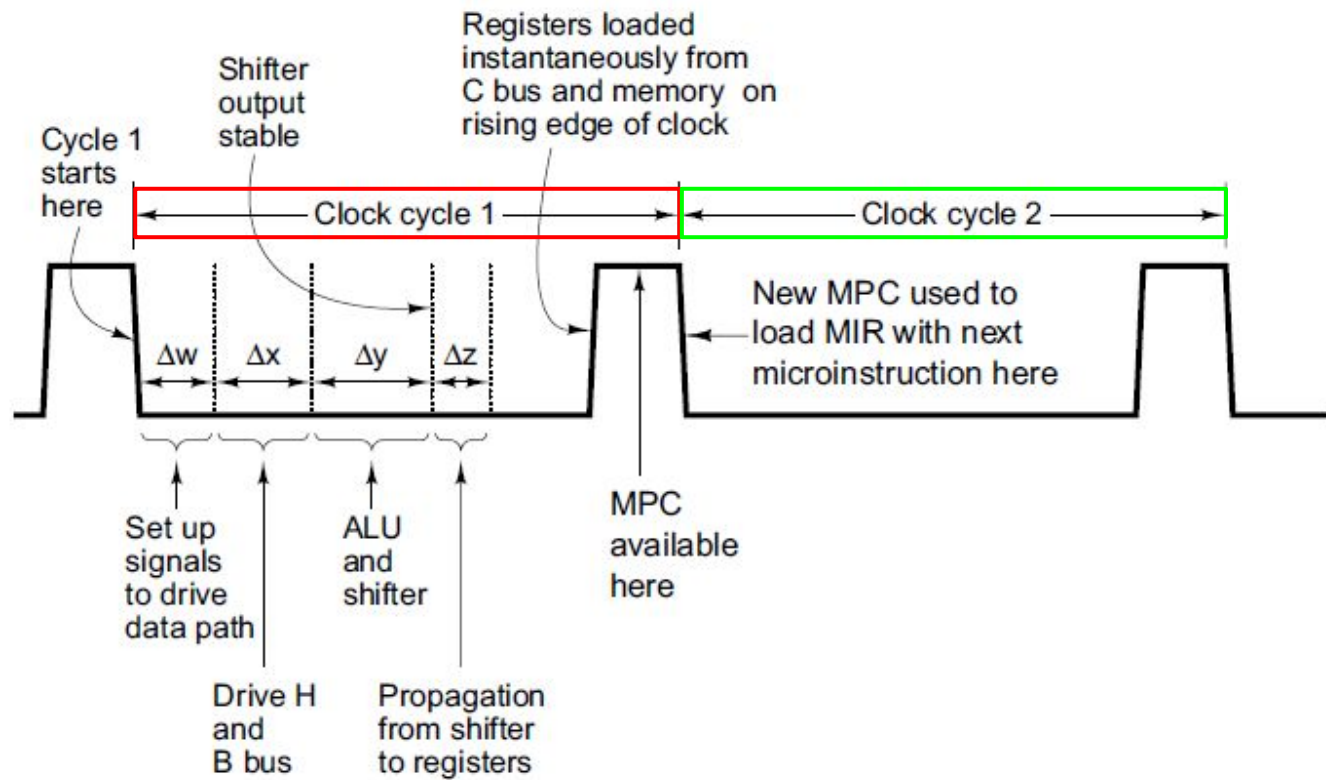
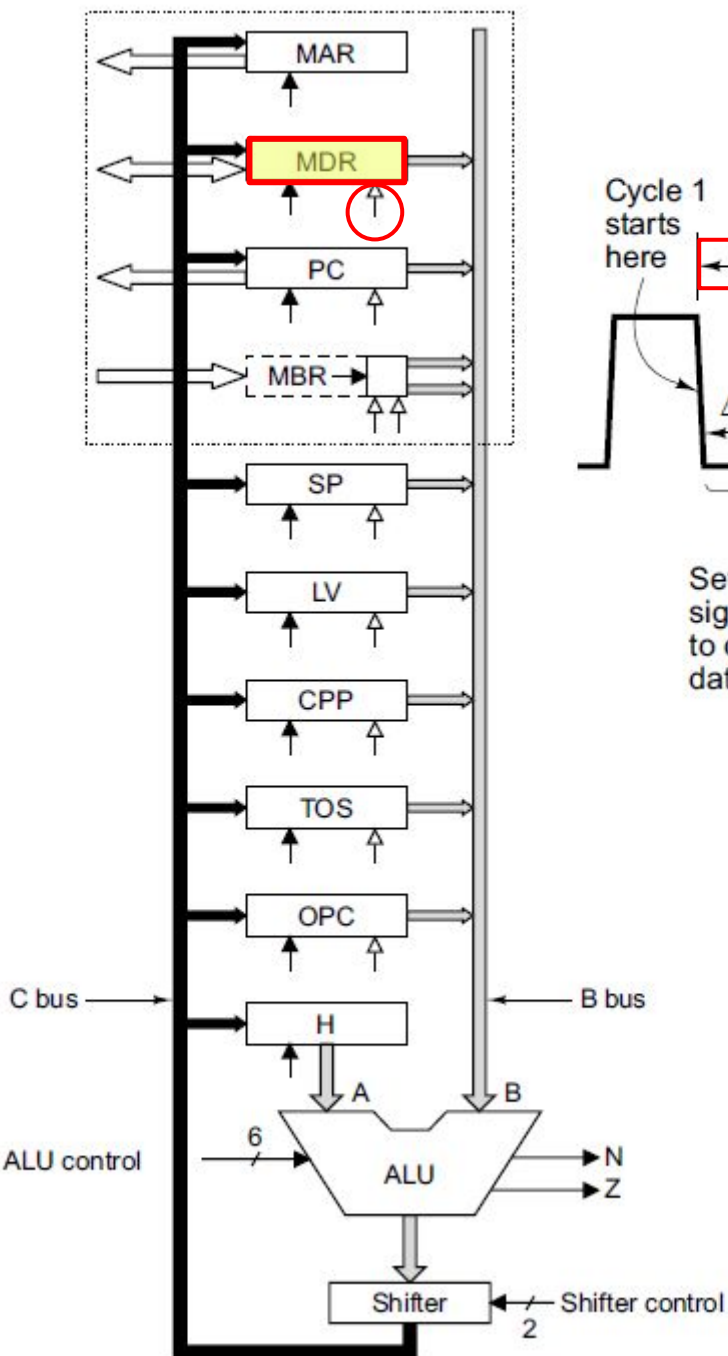


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

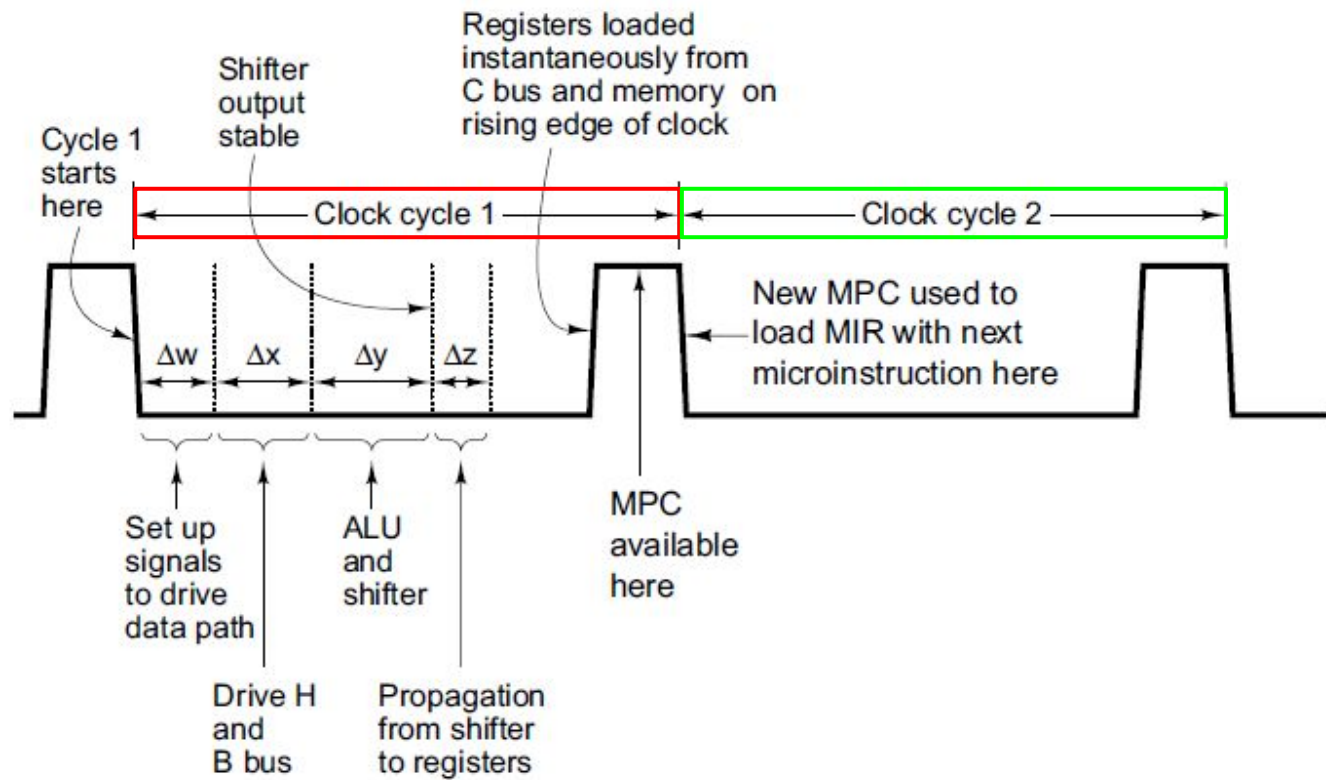
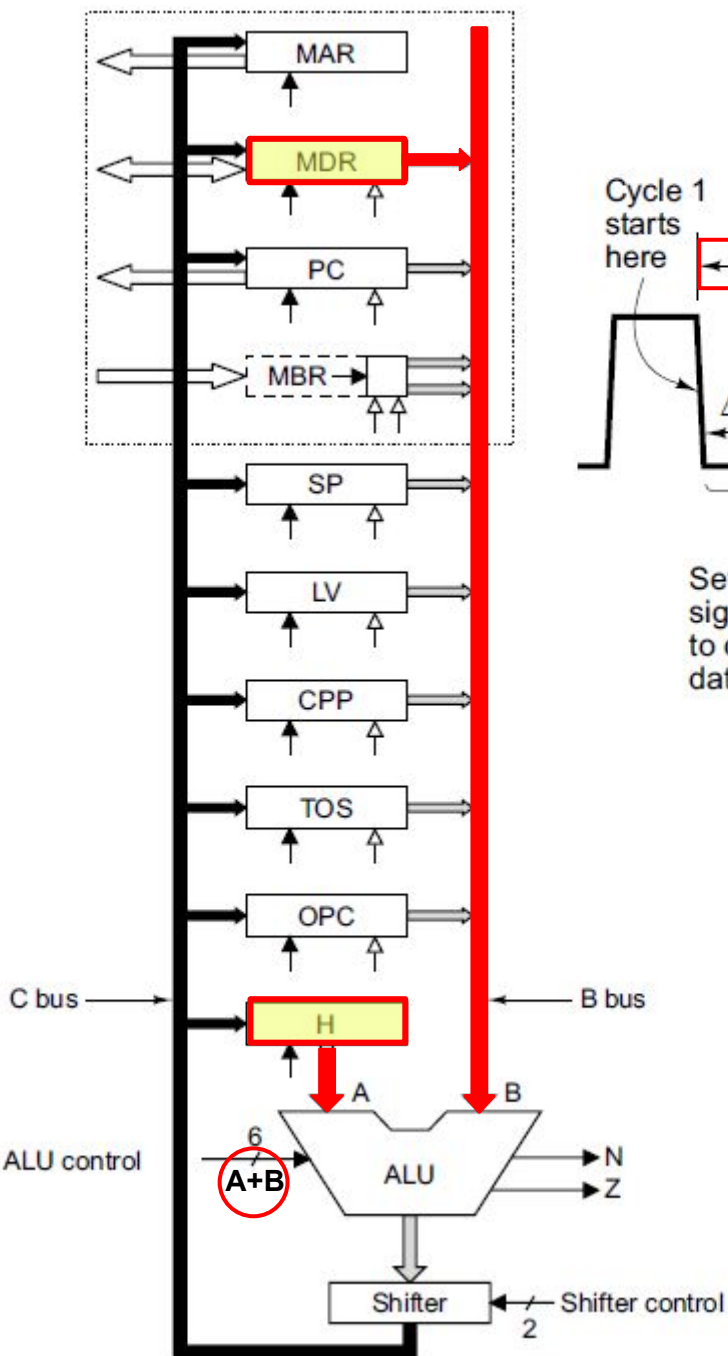


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

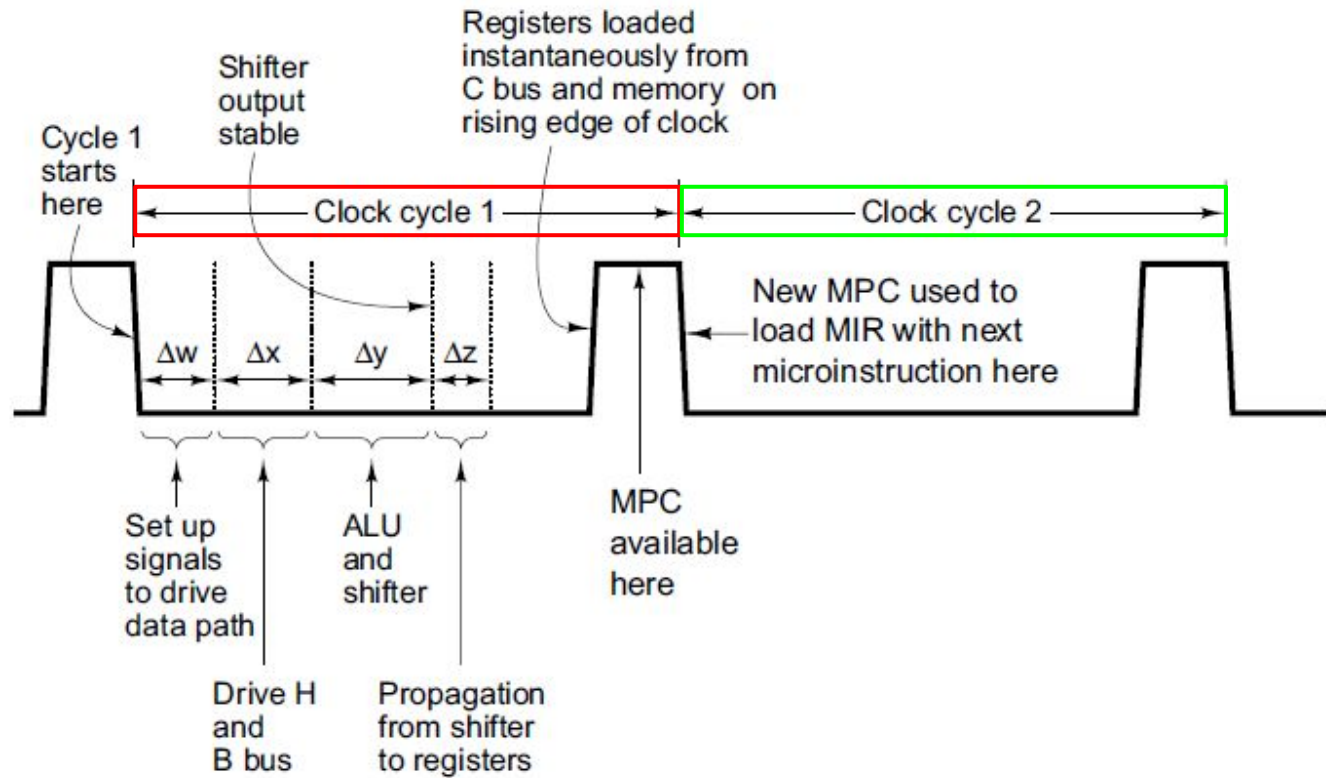
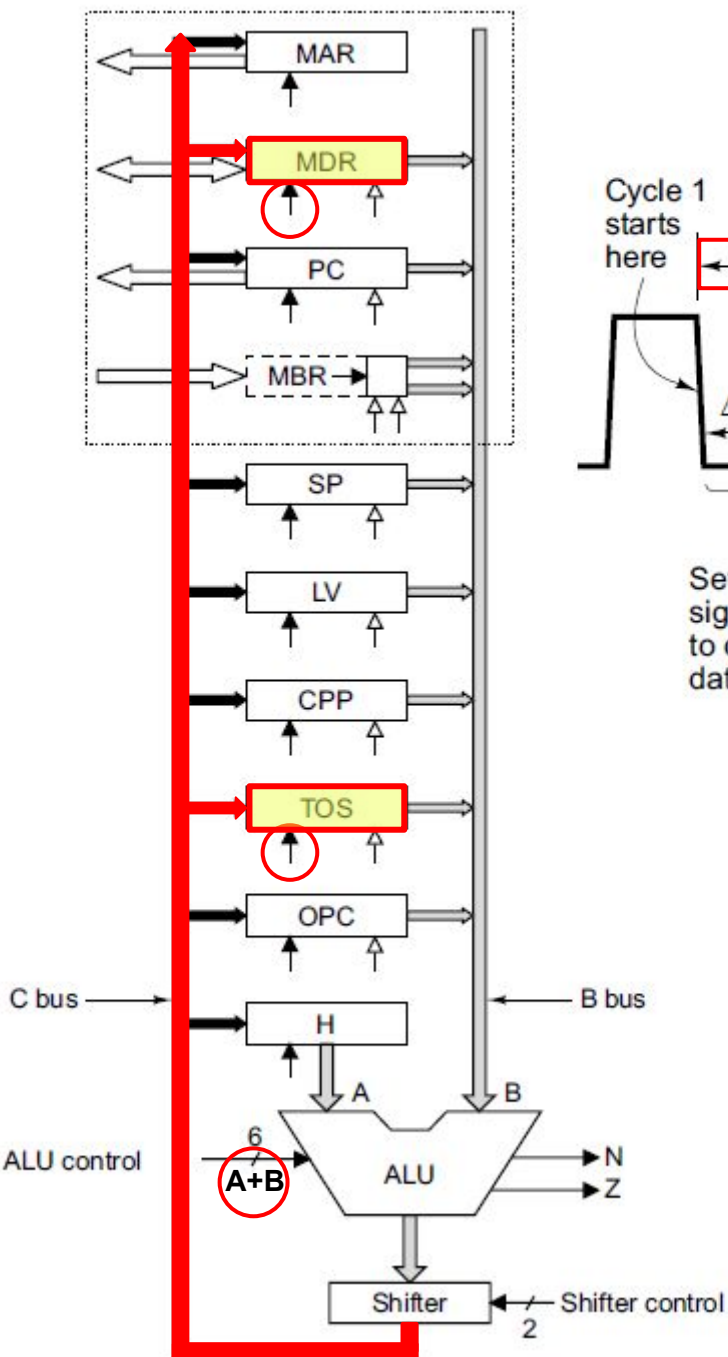


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

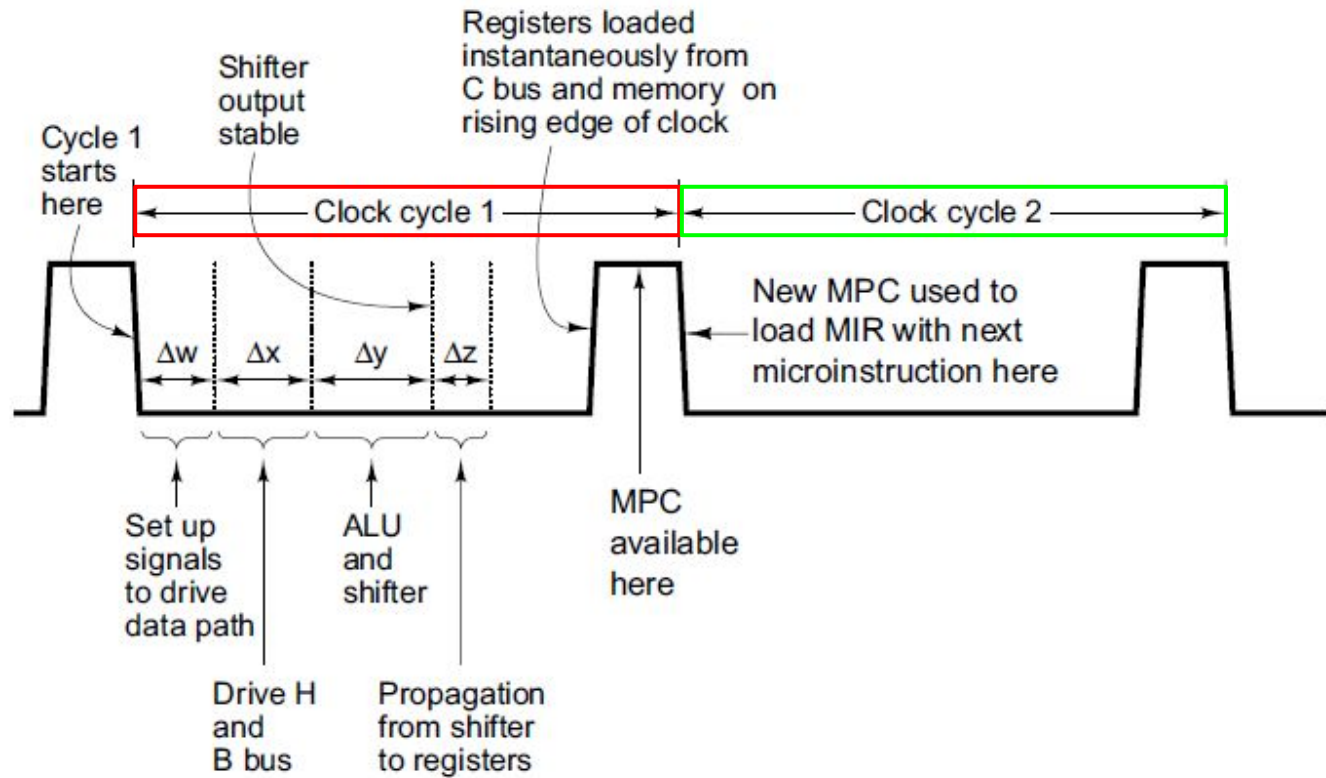
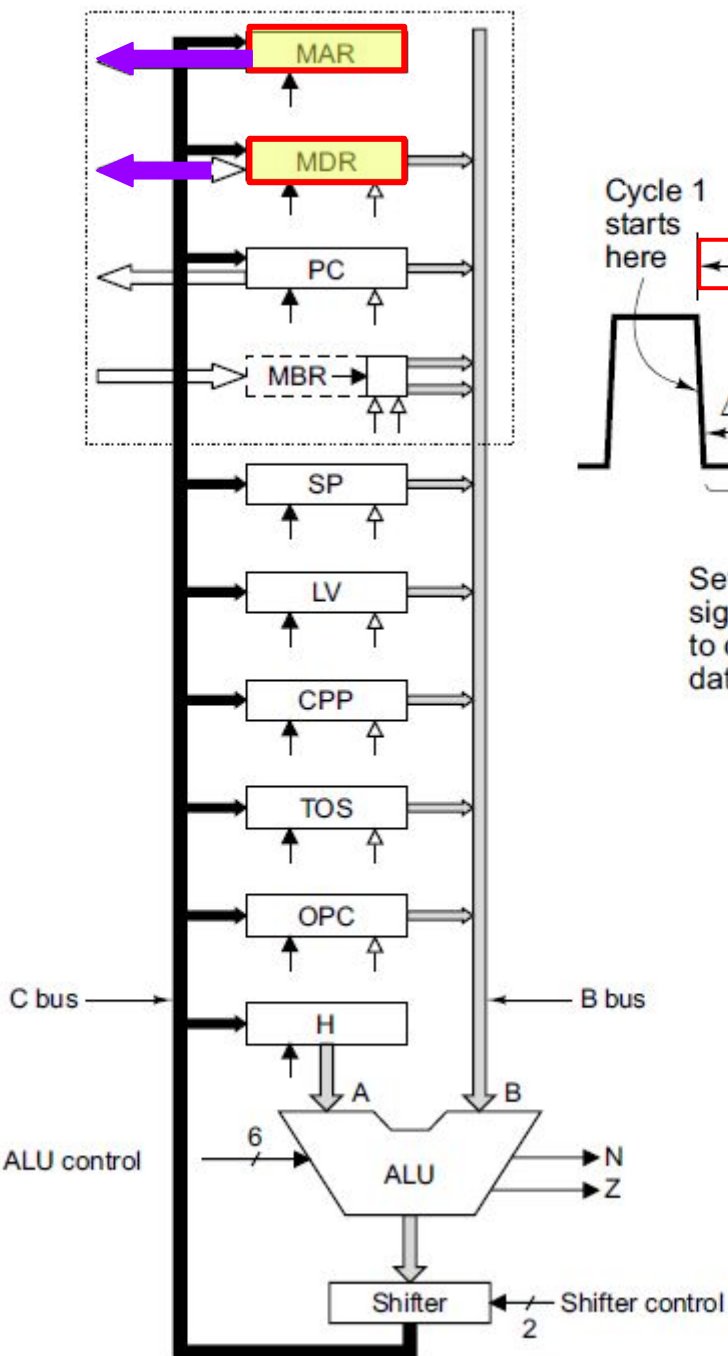


Acciones para resolver la instrucción **IADD**

MAR = SP = SP - 1; rd;

H = TOS;

MDR = TOS = H + MDR; wr;

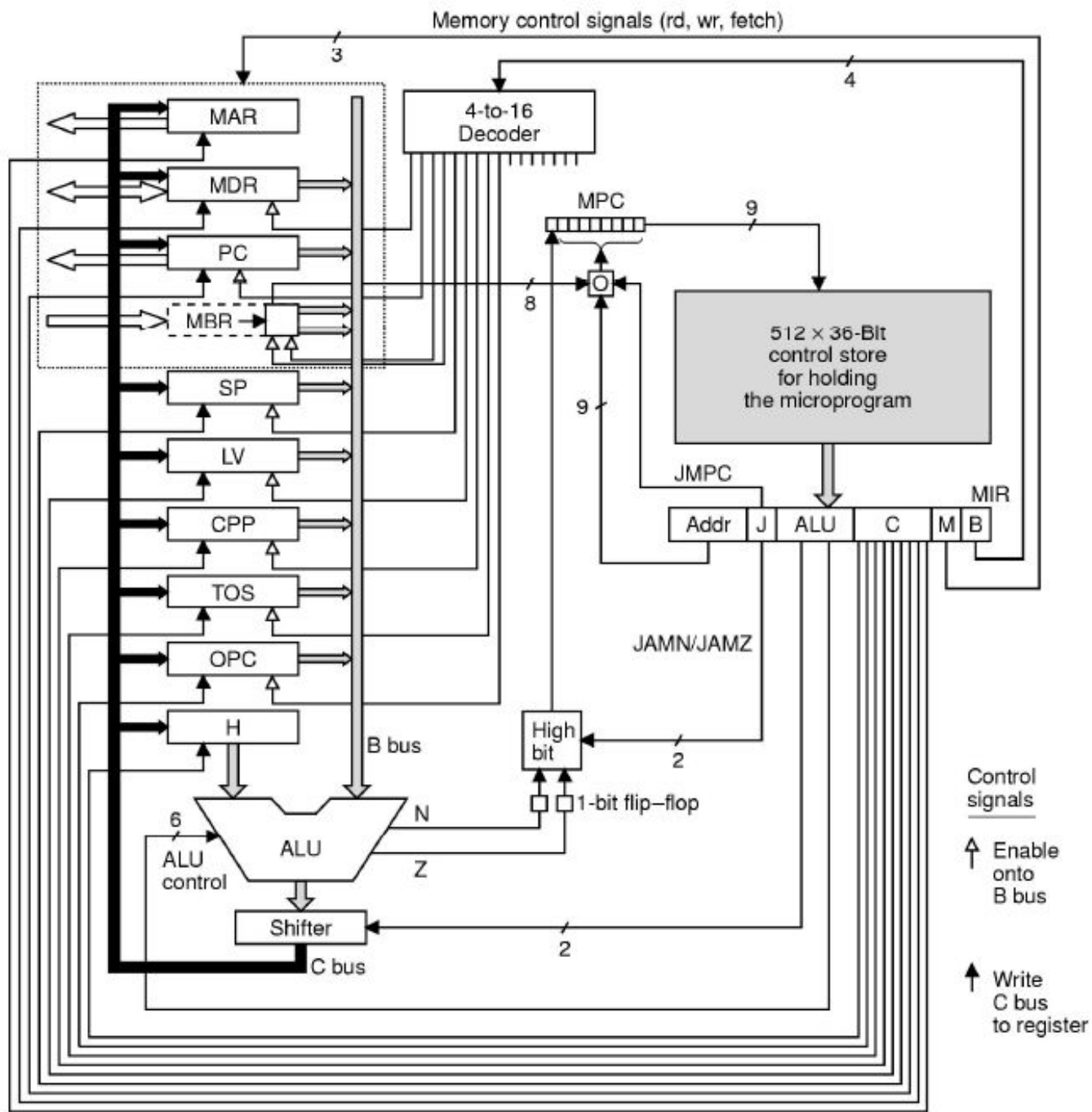


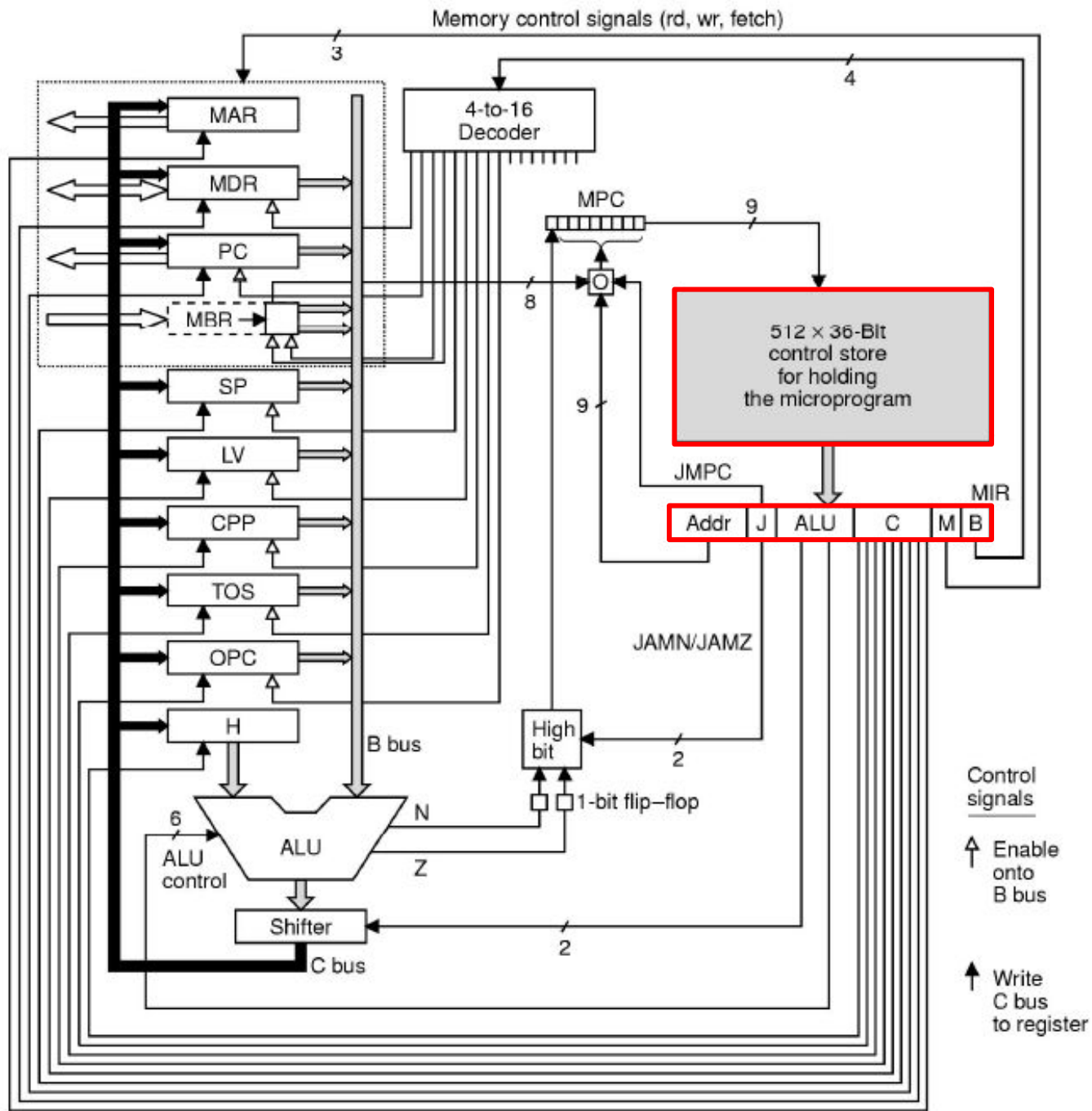
Acciones para resolver la instrucción **IADD**

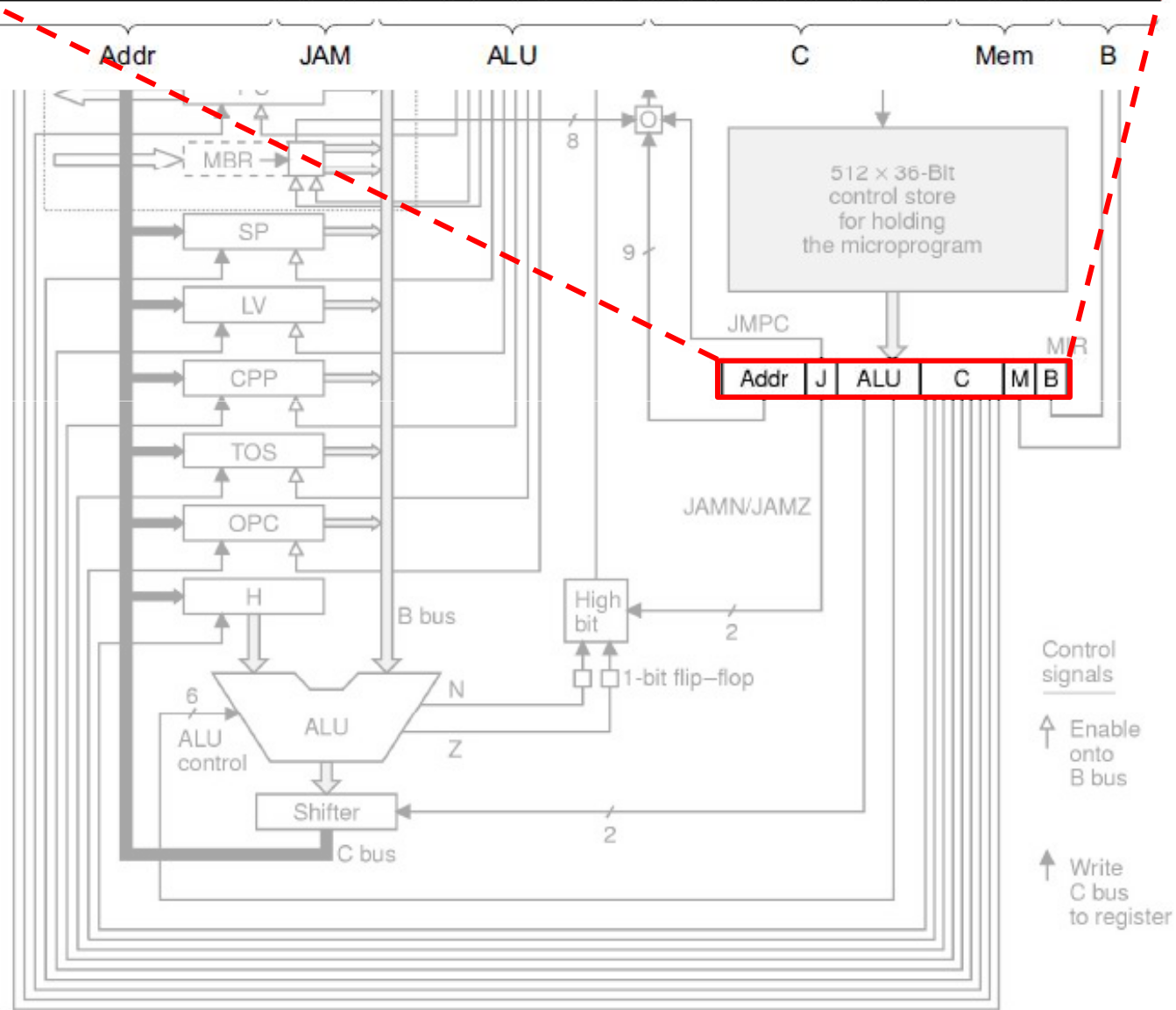
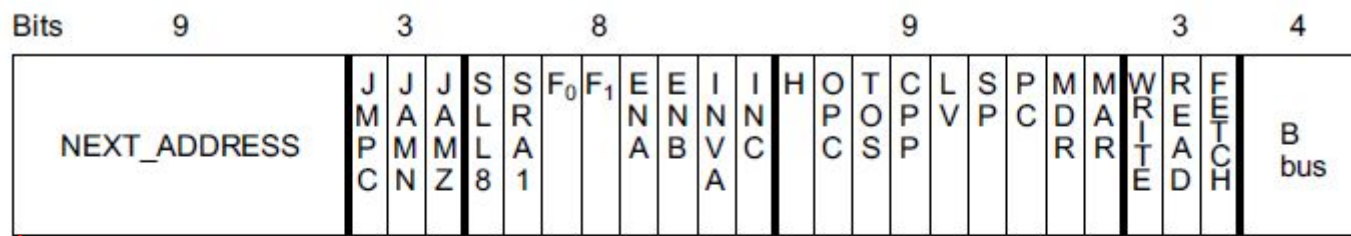
MAR = SP = SP - 1; rd;

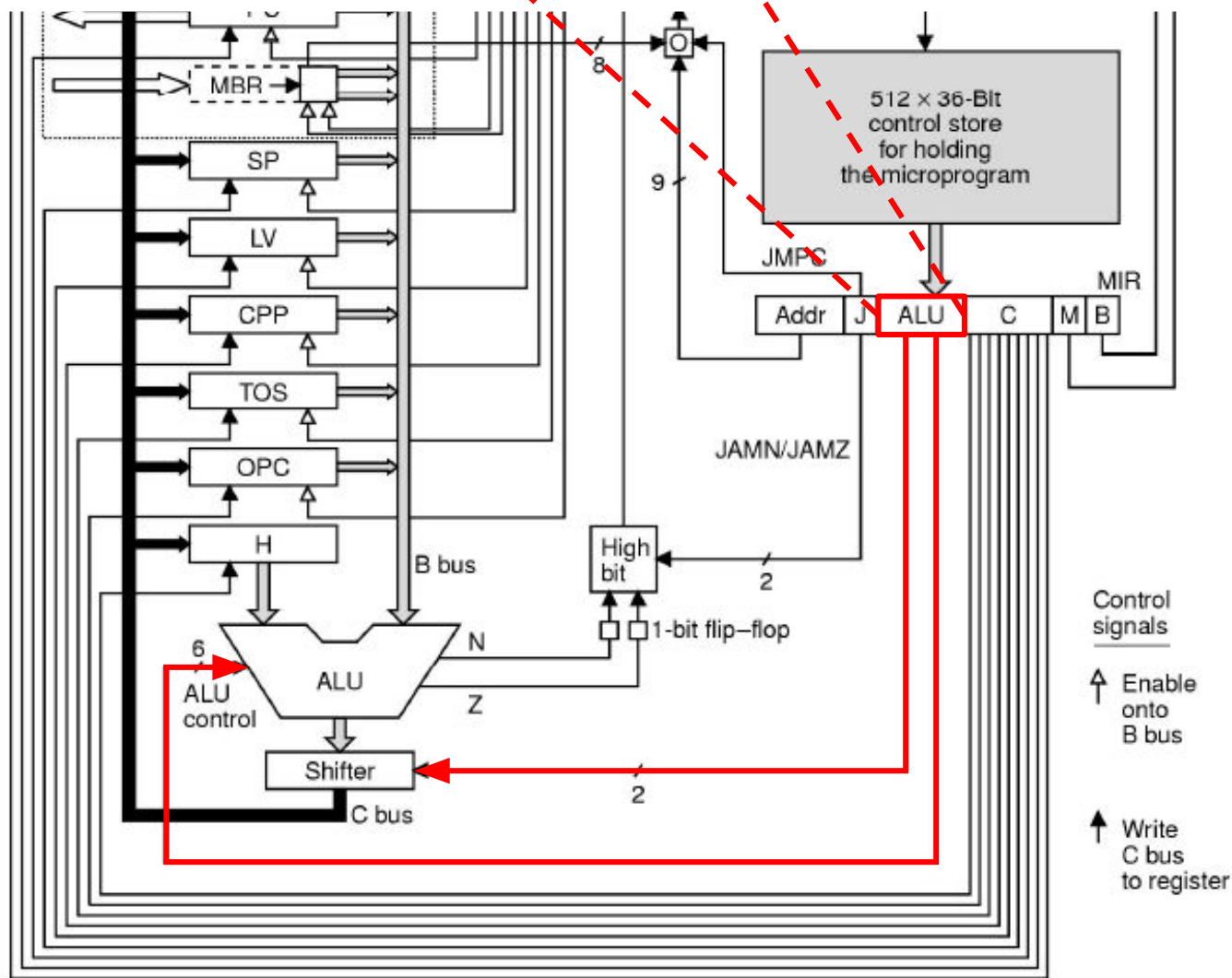
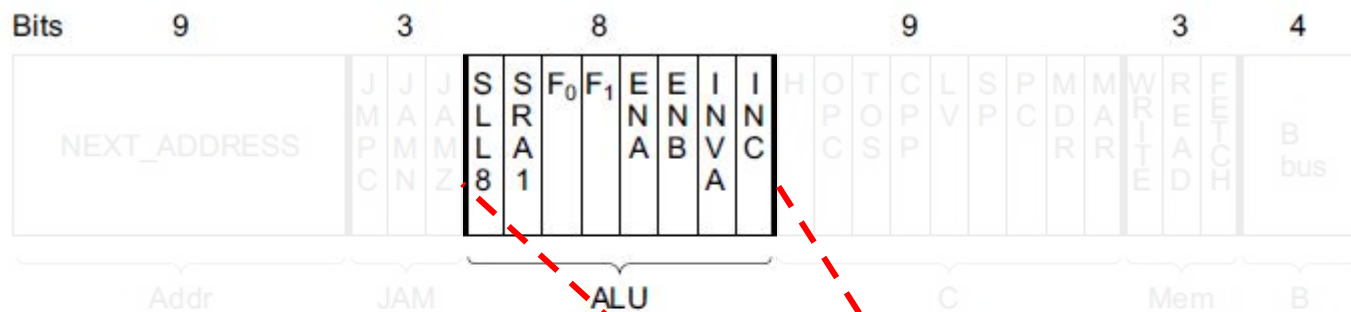
H = TOS;

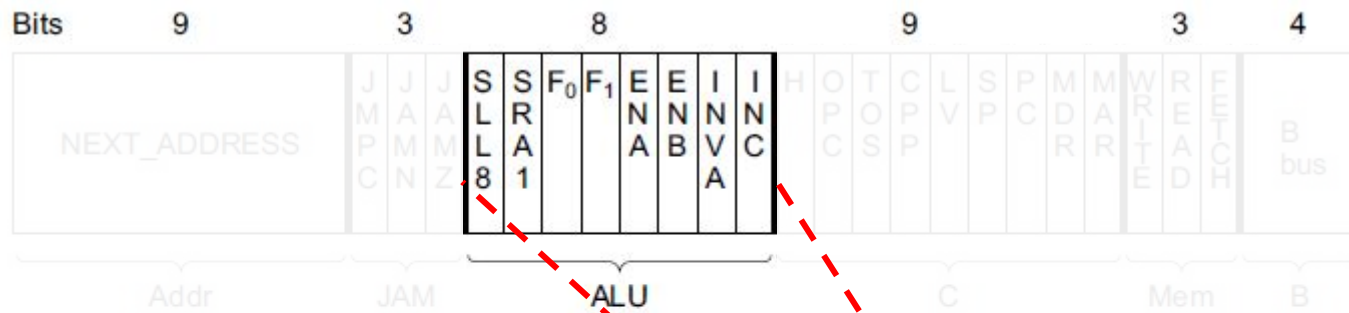
MDR = TOS = H + MDR; wr;



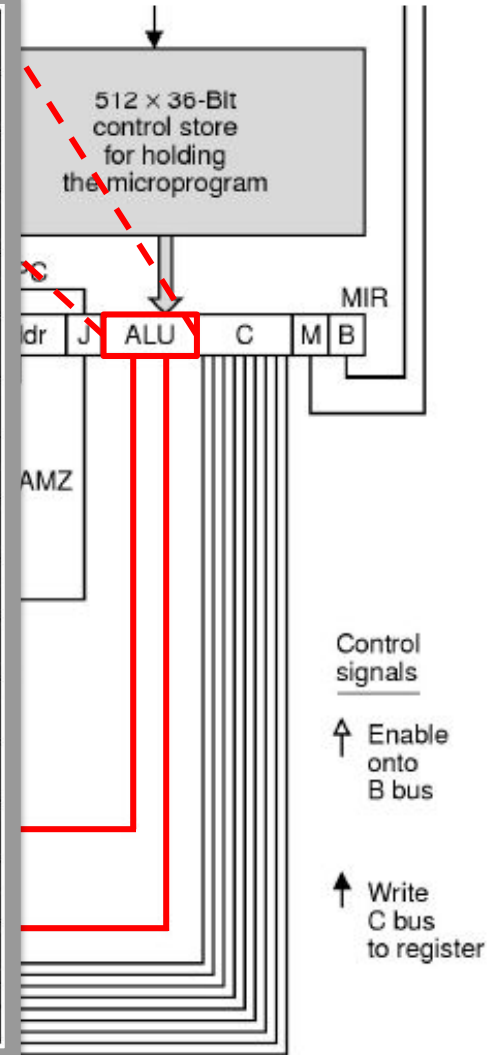


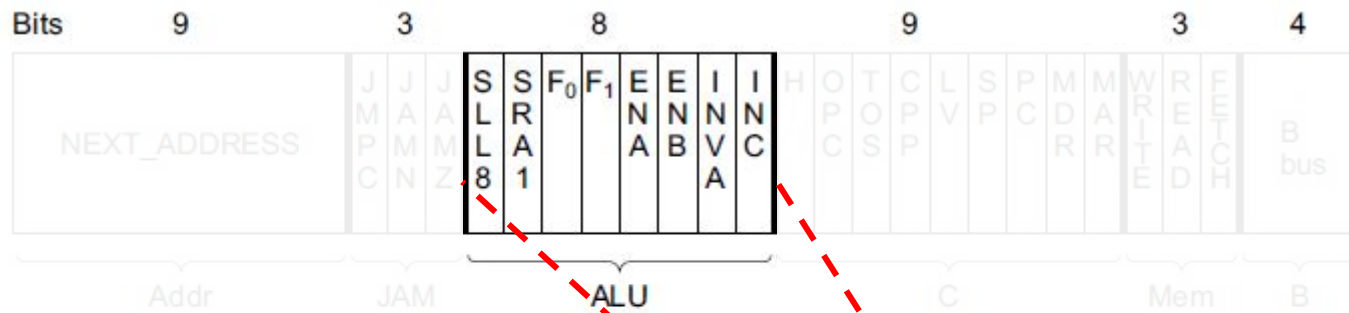




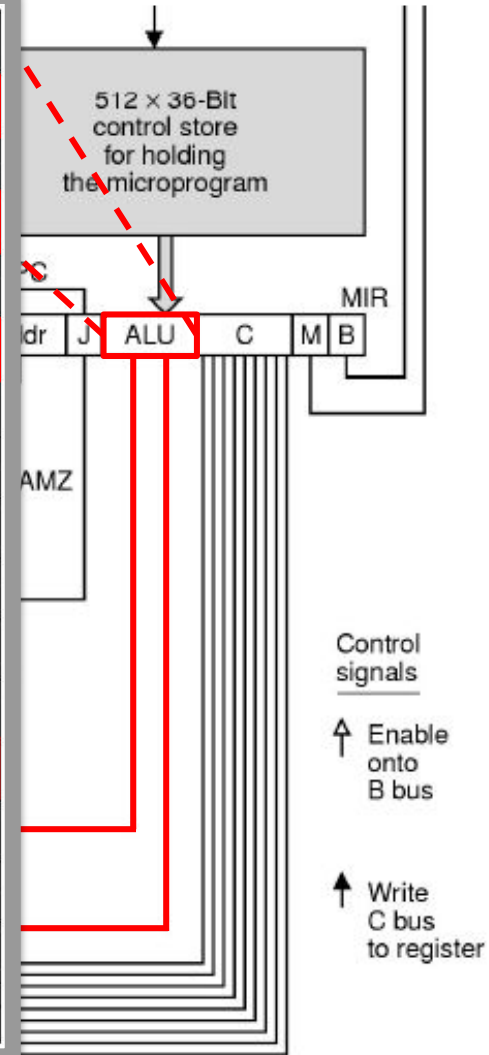


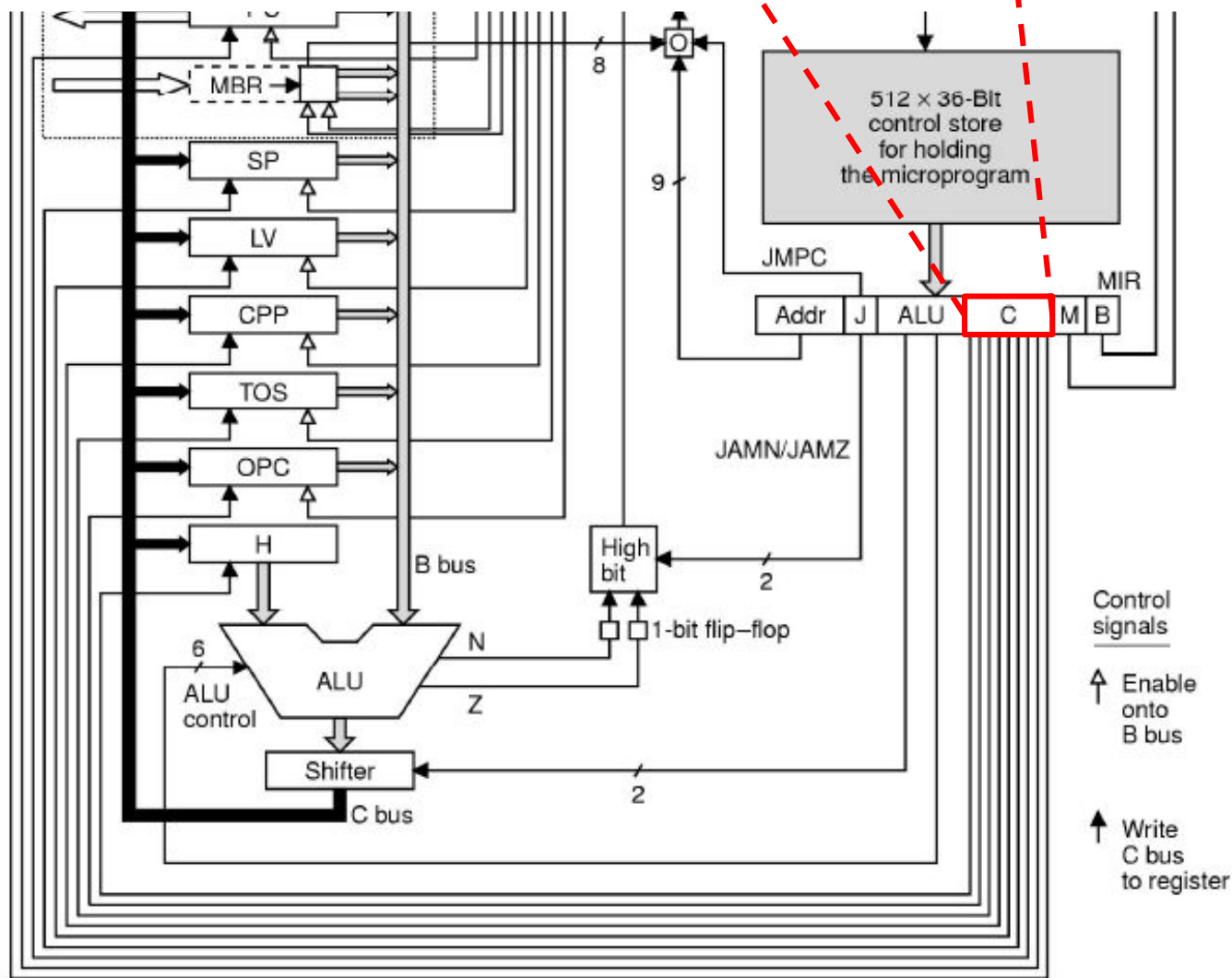
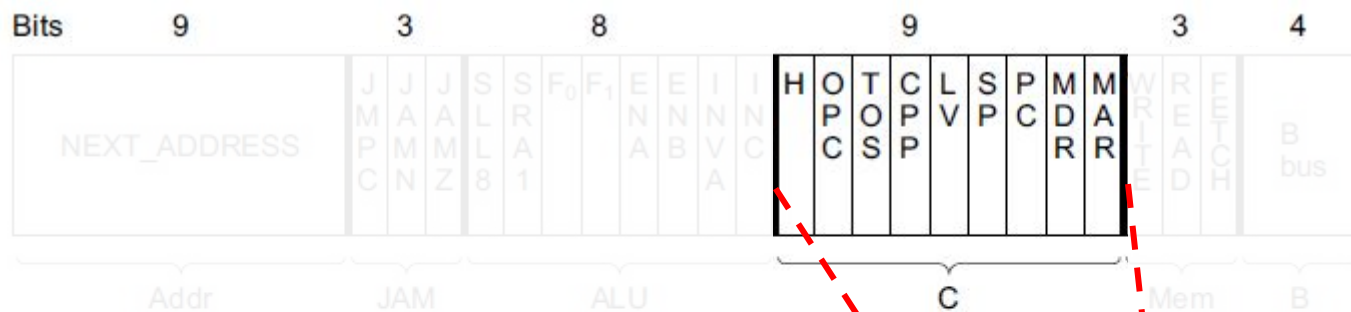
F ₀	F ₁	ENA	ENB	INVA	INC	Function
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
1	0	1	1	0	0	$\bar{B}$
1	1	1	1	0	0	A + B
1	1	1	1	0	1	A + B + 1
1	1	1	0	0	1	A + 1
1	1	0	1	0	1	B + 1
1	1	1	1	1	1	B - A
1	1	0	1	1	0	B - 1
1	1	1	0	1	1	-A
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
1	1	0	0	0	1	1
1	1	0	0	1	0	-1

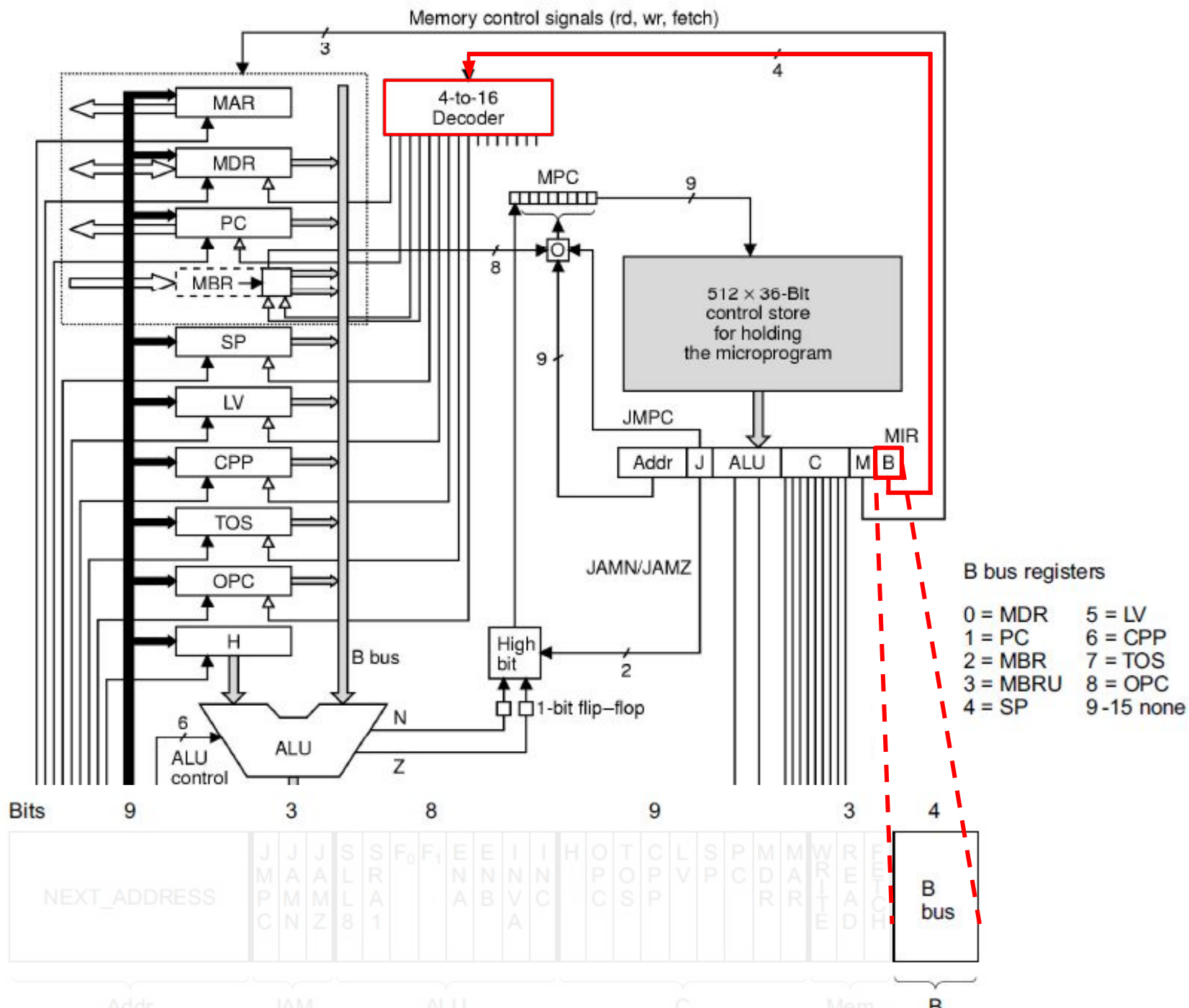


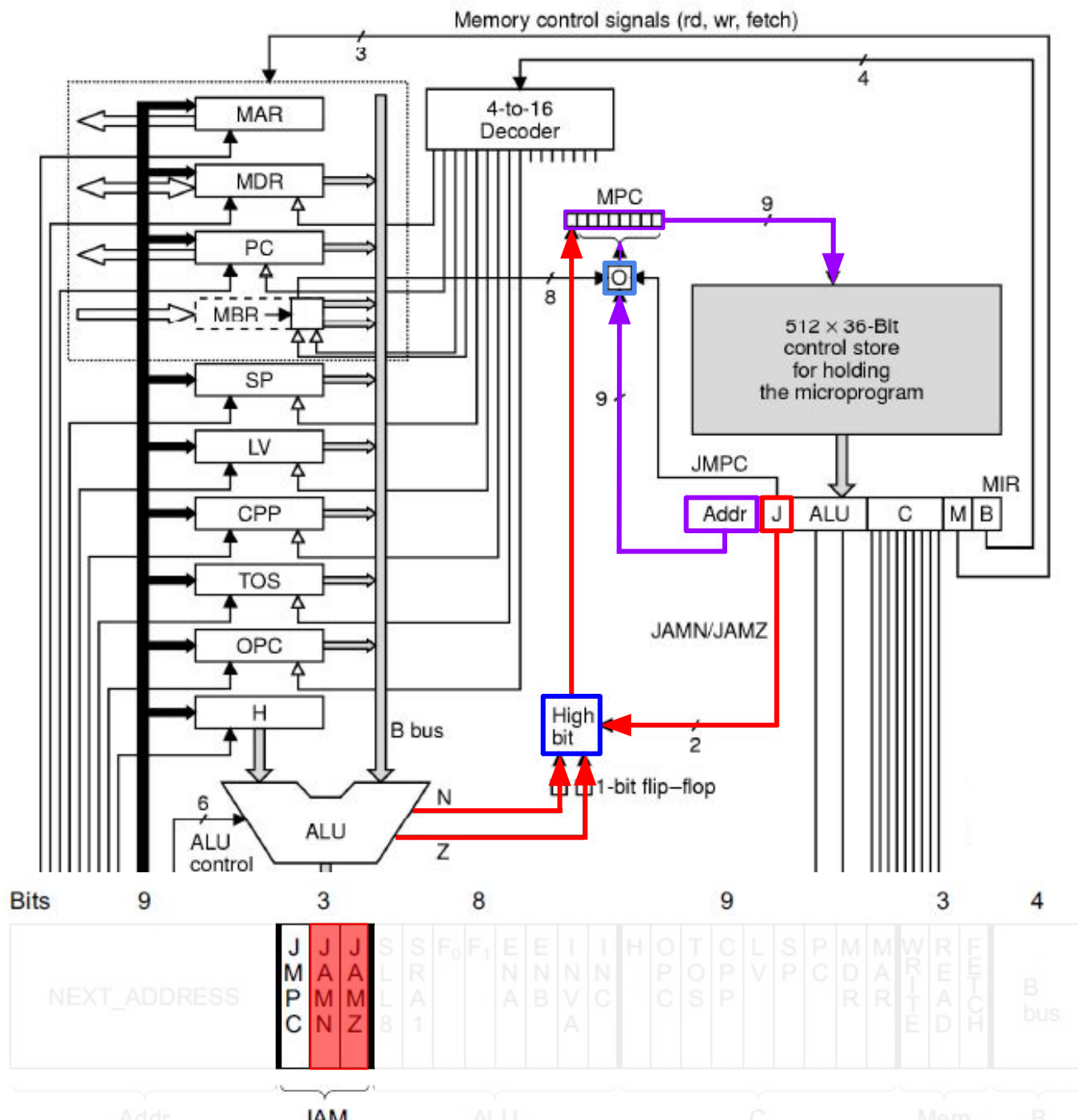


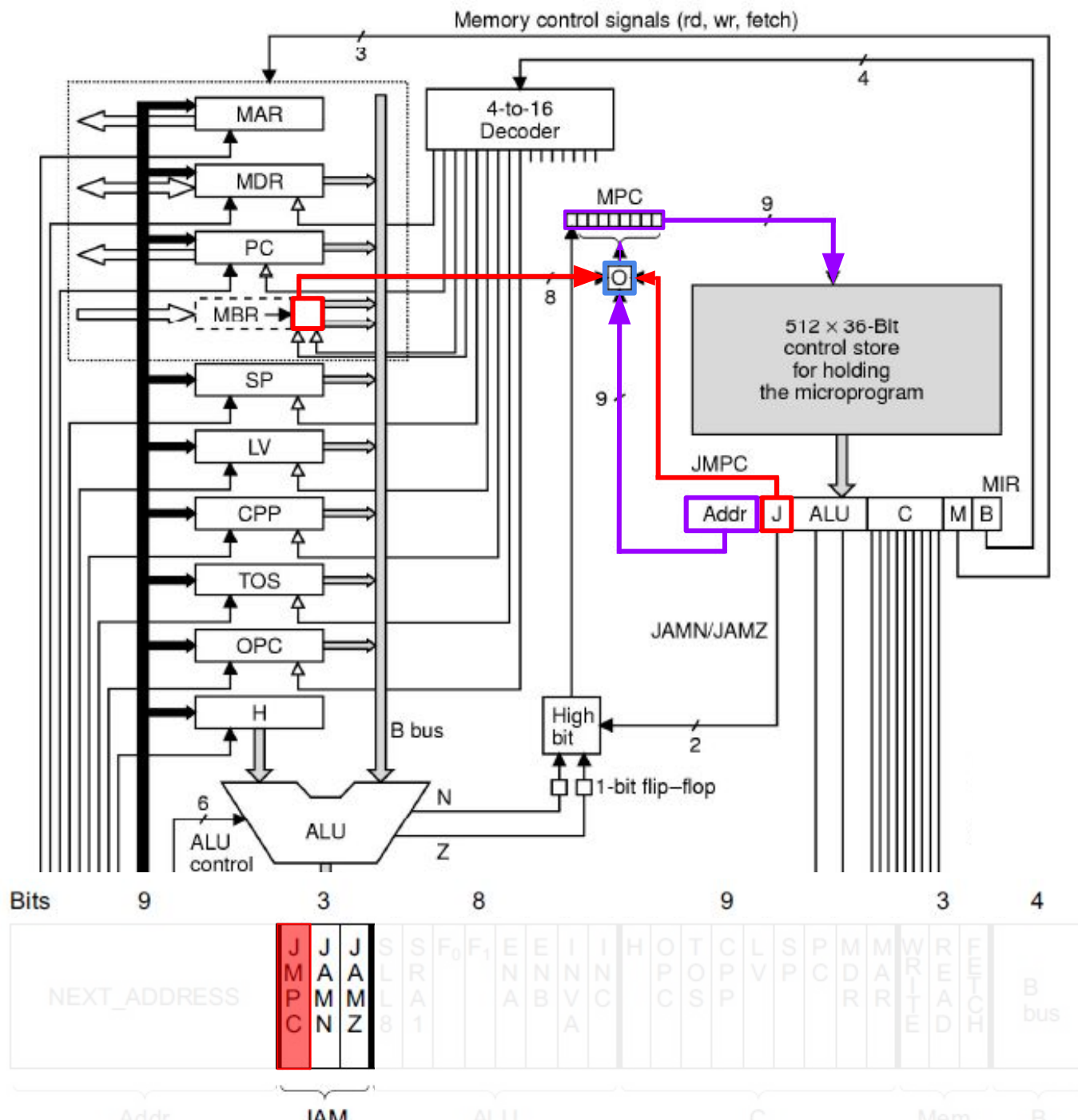
F ₀	F ₁	ENA	ENB	INVA	INC	Function
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
1	0	1	1	0	0	$\bar{B}$
1	1	1	1	0	0	A + B
1	1	1	1	0	1	A + B + 1
1	1	1	0	0	1	A + 1
1	1	0	1	0	1	B + 1
1	1	1	1	1	1	B - A
1	1	0	1	1	0	B - 1
1	1	1	0	1	1	-A
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
1	1	0	0	0	1	1
1	1	0	0	1	0	-1

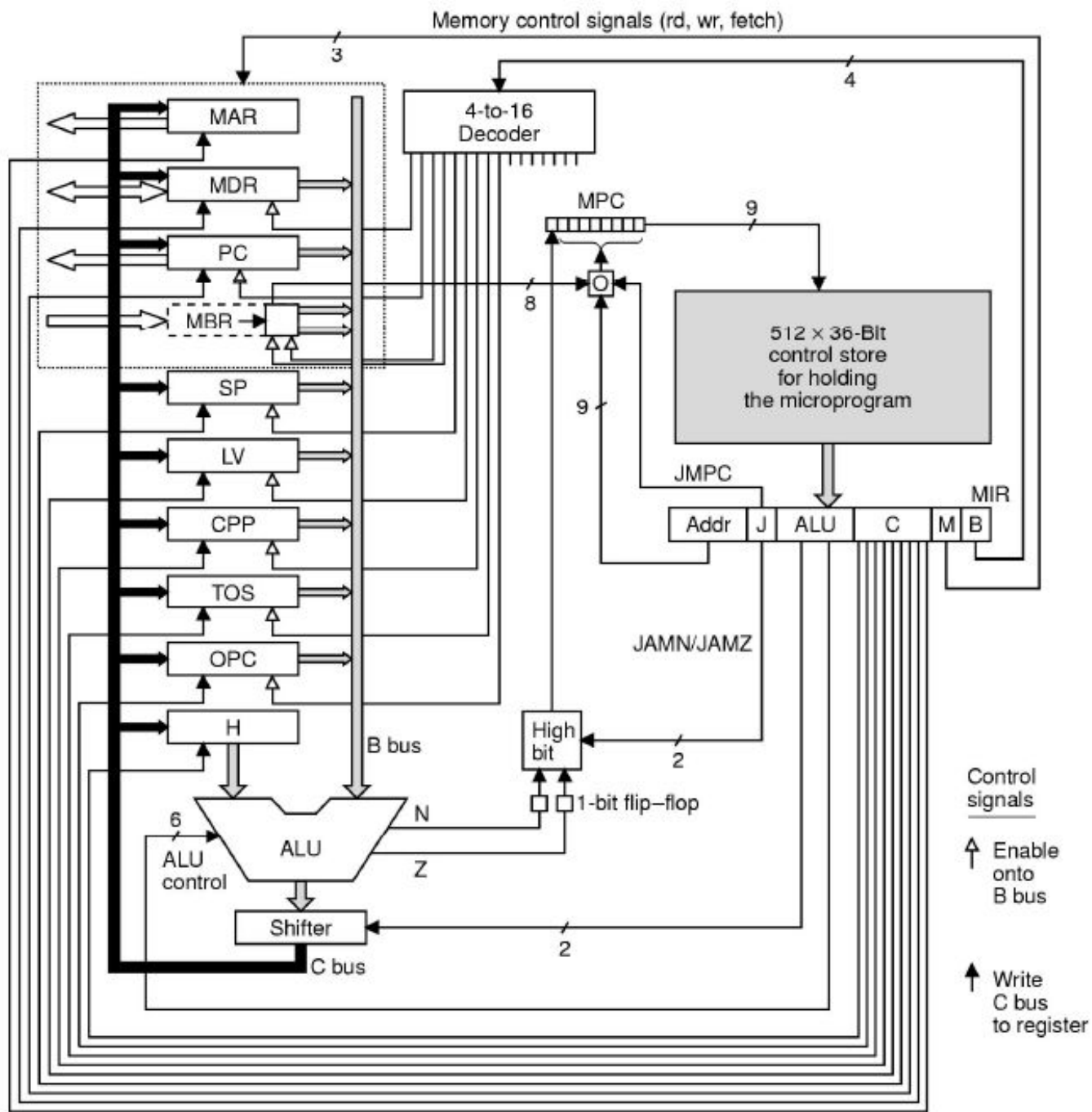


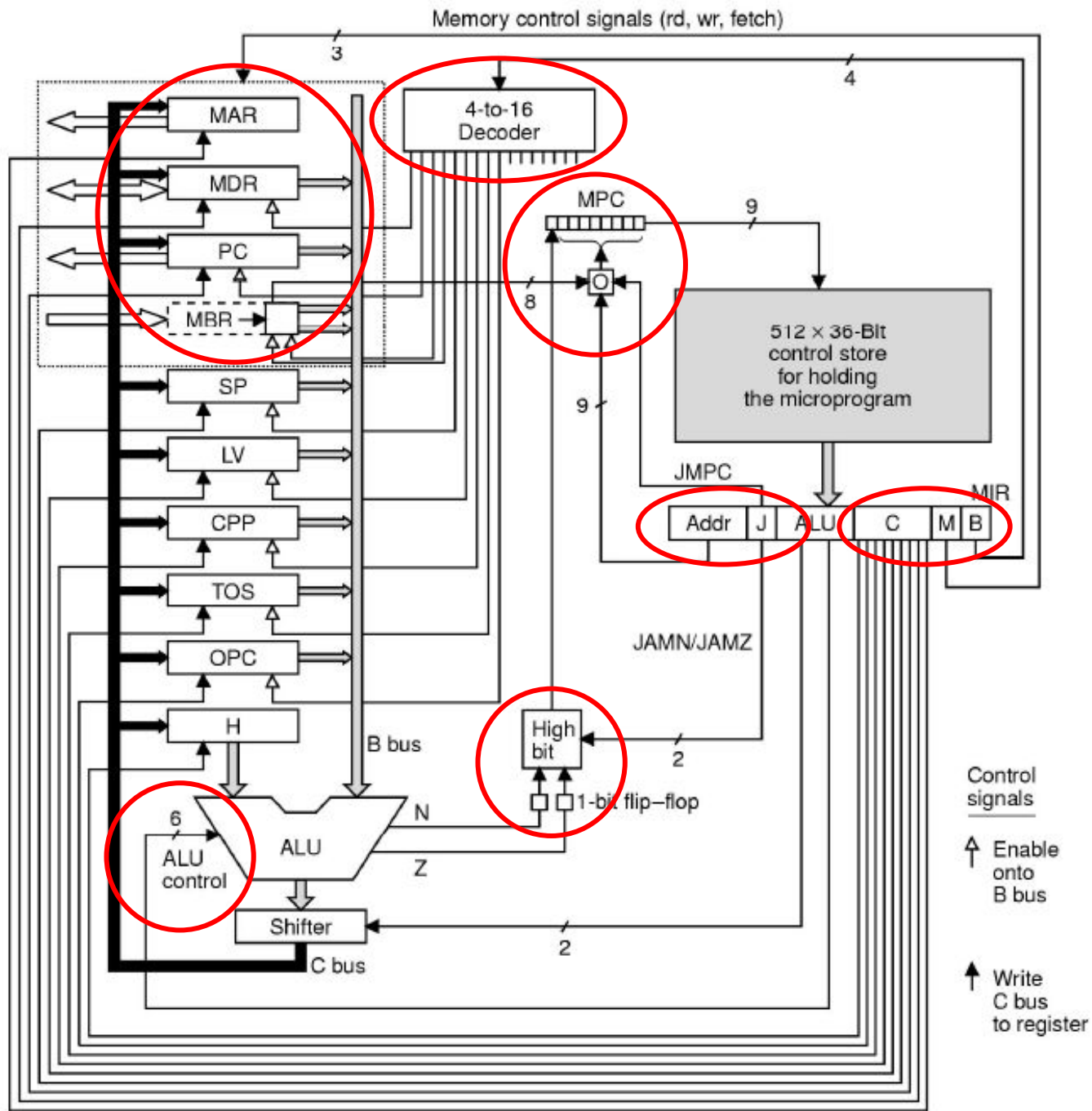












Java Virtual Machine

Microprograma (parte 1)

Label	Operations	Comments
Main1	PC = PC + 1; fetch; goto (MBR)	MBR holds opcode; get next byte; dispatch
nop1	goto Main1	Do nothing
iadd1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iadd2	H = TOS	H = top of stack
iadd3	MDR = TOS = MDR + H; wr; goto Main1	Add top two words; write to top of stack
isub1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
isub2	H = TOS	H = top of stack
isub3	MDR = TOS = MDR - H; wr; goto Main1	Do subtraction; write to top of stack
iband1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iband2	H = TOS	H = top of stack
iband3	MDR = TOS = MDR AND H; wr; goto Main1	Do AND; write to new top of stack
ior1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
ior2	H = TOS	H = top of stack
ior3	MDR = TOS = MDR OR H; wr; goto Main1	Do OR; write to new top of stack
dup1	MAR = SP = SP + 1	Increment SP and copy to MAR
dup2	MDR = TOS; wr; goto Main1	Write new stack word
pop1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
pop2		Wait for new TOS to be read from memory
pop3	TOS = MDR; goto Main1	Copy new word to TOS
swap1	MAR = SP - 1; rd	Set MAR to SP - 1; read 2nd word from stack
swap2	MAR = SP	Set MAR to top word
swap3	H = MDR; wr	Save TOS in H; write 2nd word to top of stack
swap4	MDR = TOS	Copy old TOS to MDR
swap5	MAR = SP - 1; wr	Set MAR to SP - 1; write as 2nd word on stack
swap6	TOS = H; goto Main1	Update TOS
bipush1	SP = MAR = SP + 1	MBR = the byte to push onto stack
bipush2	PC = PC + 1; fetch	Increment PC, fetch next opcode
bipush3	MDR = TOS = MBR; wr; goto Main1	Sign-extend constant and push on stack

Java Virtual Machine

Microprograma (parte 2)

Label	Operations	Comments
iload1	H = LV	MBR contains index; copy LV to H
iload2	MAR = MBRU + H; rd	MAR = address of local variable to push
iload3	MAR = SP = SP + 1	SP points to new top of stack; prepare write
iload4	PC = PC + 1; fetch; wr	Inc PC; get next opcode; write top of stack
iload5	TOS = MDR; goto Main1	Update TOS
istore1	H = LV	MBR contains index; Copy LV to H
istore2	MAR = MBRU + H	MAR = address of local variable to store into
istore3	MDR = TOS; wr	Copy TOS to MDR; write word
istore4	SP = MAR = SP - 1; rd	Read in next-to-top word on stack
istore5	PC = PC + 1; fetch	Increment PC; fetch next opcode
istore6	TOS = MDR; goto Main1	Update TOS
wide1	PC = PC + 1; fetch; goto (MBR OR 0x100)	Multiway branch with high bit set
wide_ild1	PC = PC + 1; fetch	MBR contains 1st index byte; fetch 2nd
wide_ild2	H = MBRU << 8	H = 1st index byte shifted left 8 bits
wide_ild3	H = MBRU OR H	H = 16-bit index of local variable
wide_ild4	MAR = LV + H; rd; goto iload3	MAR = address of local variable to push
wide_istore1	PC = PC + 1; fetch	MBR contains 1st index byte; fetch 2nd
wide_istore2	H = MBRU << 8	H = 1st index byte shifted left 8 bits
wide_istore3	H = MBRU OR H	H = 16-bit index of local variable
wide_istore4	MAR = LV + H; goto istore3	MAR = address of local variable to store into
ldc_w1	PC = PC + 1; fetch	MBR contains 1st index byte; fetch 2nd
ldc_w2	H = MBRU << 8	H = 1st index byte << 8
ldc_w3	H = MBRU OR H	H = 16-bit index into constant pool
ldc_w4	MAR = H + CPP; rd; goto iload3	MAR = address of constant in pool

Java Virtual Machine

Microprograma (parte 3)

Label	Operations	Comments
iinc1	H = LV	MBR contains index; Copy LV to H
iinc2	MAR = MBRU + H; rd	Copy LV + index to MAR; Read variable
iinc3	PC = PC + 1; fetch	Fetch constant
iinc4	H = MDR	Copy variable to H
iinc5	PC = PC + 1; fetch	Fetch next opcode
iinc6	MDR = MBR + H; wr; goto Main1	Put sum in MDR; update variable
goto1	OPC = PC - 1	Save address of opcode.
goto2	PC = PC + 1; fetch	MBR = 1st byte of offset; fetch 2nd byte
goto3	H = MBR << 8	Shift and save signed first byte in H
goto4	H = MBRU OR H	H = 16-bit branch offset
goto5	PC = OPC + H; fetch	Add offset to OPC
goto6	goto Main1	Wait for fetch of next opcode
iflt1	MAR = SP = SP - 1; rd	Read in next-to-top word on stack
iflt2	OPC = TOS	Save TOS in OPC temporarily
iflt3	TOS = MDR	Put new top of stack in TOS
iflt4	N = OPC; if (N) goto T; else goto F	Branch on N bit
ifeq1	MAR = SP = SP - 1; rd	Read in next-to-top word of stack
ifeq2	OPC = TOS	Save TOS in OPC temporarily
ifeq3	TOS = MDR	Put new top of stack in TOS
ifeq4	Z = OPC; if (Z) goto T; else goto F	Branch on Z bit
if_icmpeq1	MAR = SP = SP - 1; rd	Read in next-to-top word of stack
if_icmpeq2	MAR = SP = SP - 1	Set MAR to read in new top-of-stack
if_icmpeq3	H = MDR; rd	Copy second stack word to H
if_icmpeq4	OPC = TOS	Save TOS in OPC temporarily
if_icmpeq5	TOS = MDR	Put new top of stack in TOS
if_icmpeq6	Z = OPC - H; if (Z) goto T; else goto F	If top 2 words are equal, goto T, else goto F
T	OPC = PC - 1; fetch; goto goto2	Same as goto1; needed for target address
F	PC = PC + 1	Skip first offset byte
F2	PC = PC + 1; fetch	PC now points to next opcode
F3	goto Main1	Wait for fetch of opcode

Java Virtual Machine

Microprograma (parte 4)

Label	Operations	Comments
invokevirtual1	PC = PC + 1; fetch	MBR = index byte 1; inc. PC, get 2nd byte
invokevirtual2	H = MBRU << 8	Shift and save first byte in H
invokevirtual3	H = MBRU OR H	H = offset of method pointer from CPP
invokevirtual4	MAR = CPP + H; rd	Get pointer to method from CPP area
invokevirtual5	OPC = PC + 1	Save Return PC in OPC temporarily
invokevirtual6	PC = MDR; fetch	PC points to new method; get param count
invokevirtual7	PC = PC + 1; fetch	Fetch 2nd byte of parameter count
invokevirtual8	H = MBRU << 8	Shift and save first byte in H
invokevirtual9	H = MBRU OR H	H = number of parameters
invokevirtual10	PC = PC + 1; fetch	Fetch first byte of # locals
invokevirtual11	TOS = SP - H	TOS = address of OBJREF - 1
invokevirtual12	TOS = MAR = TOS + 1	TOS = address of OBJREF (new LV)
invokevirtual13	PC = PC + 1; fetch	Fetch second byte of # locals
invokevirtual14	H = MBRU << 8	Shift and save first byte in H
invokevirtual15	H = MBRU OR H	H = # locals
invokevirtual16	MDR = SP + H + 1; wr	Overwrite OBJREF with link pointer
invokevirtual17	MAR = SP = MDR;	Set SP, MAR to location to hold old PC
invokevirtual18	MDR = OPC; wr	Save old PC above the local variables
invokevirtual19	MAR = SP = SP + 1	SP points to location to hold old LV
invokevirtual20	MDR = LV; wr	Save old LV above saved PC
invokevirtual21	PC = PC + 1; fetch	Fetch first opcode of new method.
invokevirtual22	LV = TOS; goto Main1	Set LV to point to LV Frame

Java Virtual Machine

Arquitectura Básica

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta